Deep Learning

Mihir Rao

Contents

1 Introduction **5**
1.1 Representations: multilingual translation . . . 5
1.2 Machine learning and parameterized programs . . . 5
1.3 From shallow to deep learning . . . 6
1.4 A brief history . . . 6
1.5 What makes deep learning work . . . 7
1.6 The underlying themes . . . 8
1.7 Why "neural" networks . . . 8

2 ML Basics 1 **10**
2.1 Types of learning problems . . . 10
2.2 Supervised learning . . . 10
2.3 Predicting probabilities . . . 10
2.4 Representing $p(y \mid x)$: the softmax . . . 11
2.5 The machine learning method . . . 12
2.6 Maximum likelihood and the loss function . . . 12
2.7 Optimization: gradient descent . . . 13
2.8 Logistic regression . . . 13
2.9 Empirical risk and true risk . . . 14

3 ML Basics 2 **15**
3.1 Regression and the squared error . . . 15
3.2 Error across datasets . . . 15
3.3 Regularization . . . 16
3.4 Training, validation, and test sets . . . 18

4 Optimization **20**
4.1 How gradient descent behaves . . . 20
4.2 The loss surface . . . 20
4.3 Newton's method . . . 21
4.4 Momentum . . . 22
4.5 Gradient scale: RMSProp, AdaGrad, and Adam . . . 22
4.6 Stochastic gradient descent . . . 23
4.7 Learning rates and tuning . . . 24

5 Backpropagation **25**
5.1 Computation graphs . . . 25
5.2 Learning the features . . . 26
5.3 The chain rule . . . 26
5.4 The backpropagation algorithm . . . 27
5.5 Practical implementation . . . 27

6 Convolutional Nets **29**
6.1 Why convolutions . . . 29
6.2 Pooling . . . 30
6.3 Padding and stride . . . 30
6.4 Examples of convolutional networks . . . 31

7 Getting Neural Nets to Train **33**
7.1 Normalizing inputs and outputs . . . 33
7.2 Batch normalization . . . 33
7.3 Weight initialization . . . 34
7.4 Gradient clipping . . . 35
7.5 Ensembles . . . 36
7.6 Dropout . . . 37
7.7 Hyperparameters . . . 38

8 Computer Vision **39**
8.1 Object localization . . . 39
8.2 Sliding windows and OverFeat . . . 40
8.3 Object detection . . . 40
8.4 Semantic segmentation . . . 41

9 Generating Images from CNNs **43**
9.1 What do convolutional networks "see"? . . . 43
9.2 Visualizing neuron responses . . . 43
9.3 Optimizing the image . . . 44
9.4 DeepDream . . . 45
9.5 Style transfer . . . 45

10 Recurrent Neural Networks **47**
10.1 Variable-size inputs . . . 47
10.2 Training by backpropagation . . . 47
10.3 Variable-size outputs . . . 48
10.4 Vanishing and exploding gradients . . . 49
10.5 LSTM cells . . . 49
10.6 Autoregressive models and structured prediction . . . 50
10.7 Ways to use RNNs . . . 51

11 Sequence-to-Sequence Models **53**
11.1 Neural language models . . . 53
11.2 Conditional language models and sequence-to-sequence models . . . 53
11.3 Decoding the most likely sequence . . . 54
11.4 Beam search . . . 54
11.5 Attention . . . 56

12 Transformers **58**
12.1 Is attention all we need? . . . 58
12.2 Positional encoding . . . 58
12.3 Multi-head attention . . . 60
12.4 Adding nonlinearities . . . 60
12.5 Masked decoding . . . 60
12.6 The transformer . . . 61
12.7 Why transformers? . . . 63

13 Applications: NLP **65**
13.1 Unsupervised pretraining . . . 65

13.2 Word embeddings . . . 65
13.3 Contextual representations and ELMo . . . 67
13.4 BERT . . . 68
13.5 GPT and summary . . . 69

14 Learning-Based Control and Imitation **70**
14.1 From prediction to control . . . 70
14.2 Terminology . . . 70
14.3 Behavioral cloning . . . 71
14.4 Getting behavioral cloning to work . . . 71
14.5 DAgger . . . 72

15 Reinforcement Learning **74**
15.1 Rewards and Markov decision processes . . . 74
15.2 The goal of reinforcement learning . . . 74
15.3 Policy gradients . . . 75
15.4 Making policy gradients work . . . 76
15.5 Off-policy policy gradients . . . 77
15.6 Implementation with automatic differentiation . . . 78

16 Q-Learning **80**
16.1 Value functions and the policy gradient . . . 80
16.2 Fitting value functions . . . 80
16.3 Actor-critic algorithms . . . 81
16.4 From policy iteration to value iteration . . . 82
16.5 Fitted Q-iteration . . . 83
16.6 Q-learning . . . 84
16.7 Continuous actions and practice . . . 85

17 Autoencoders and Latent Variable Models **87**
17.1 Generative models . . . 87
17.2 Autoregressive generative models . . . 87
17.3 Autoencoders . . . 89
17.4 Latent variable models . . . 90
17.5 Latent variable models in deep learning . . . 91

18 Variational Autoencoders and Invertible Models **93**
18.1 The variational approximation . . . 93
18.2 Amortized variational inference . . . 94
18.3 The reparameterization trick . . . 95
18.4 The variational autoencoder . . . 95
18.5 VAEs in practice . . . 96
18.6 Invertible models and normalizing flows . . . 97

19 Generative Adversarial Networks **99**
19.1 Matching distributions at the population level . . . 99
19.2 The GAN game . . . 100
19.3 What does the GAN optimize? . . . 100
19.4 GAN architectures and variants . . . 101
19.5 Improved GAN training . . . 102
19.6 Wasserstein GAN . . . 103
19.7 Better discriminator regularization . . . 104

20 Adversarial Examples **105**
20.1 Do deep nets generalize? . . . 105

20.2 Adversarial examples . . . 106
20.3 Overfitting or linearity? . . . 106
20.4 Adversarial attacks . . . 107
20.5 Black-box attacks . . . 109
20.6 Defenses . . . 109

21 Meta-Learning **111**
21.1 What is meta-learning? . . . 111
21.2 Meta-learning with supervised learning . . . 111
21.3 Non-parametric meta-learning . . . 112
21.4 Model-agnostic meta-learning . . . 113
21.5 Meta-reinforcement learning . . . 114

Lecture 1: Introduction

1.1 Representations: multilingual translation

The world has over 6000 languages, and an automated translation system is trained on *paired* data: an English↔French model learns from a corpus of sentence pairs such as "I think, therefore I am" ↔ "Je pense, donc je suis." For most language pairs (say Maltese to Tibetan) there are almost no paired sentences.

Standard machine translation trains one model per pair. A multilingual translation model (Johnson et al., 2016) is a single model trained on all pairs at once, with the desired output language as an extra input. Two findings stand out.

- *Improved efficiency*: translating into and out of rare languages works better when the same model is also trained on common languages.
- *Zero-shot translation*: trained on English→French, French→English, and English→Spanish, the model can translate French→Spanish, a pair it never saw.

The desired-language input can even be a *mix*, for example 40% Spanish and 60% French. Translating an English sentence with Portuguese weight w and Spanish weight $1 - w$ gives Spanish for small w and Portuguese for large w, switching between $w = 0.3$ and $w = 0.4$. Mixing Japanese and Korean switches from one to the other near $w = 0.58$, where a single sentence mixes both, and mixing Russian and Belarusian at intermediate weights produces sentences that are neither Russian nor Belarusian.

The explanation is that the model does not map English words to French words directly. It maps the input to an internal "thought" and decodes the thought into the desired language. That thought is a representation. Classic machine learning predicts y from a given input x; for complex inputs such as sentences, images, or speech spectrograms, the hard part is choosing what x is. The power of deep learning lies in its ability to learn such representations automatically from data.

1.2 Machine learning and parameterized programs

A program that maps an image to an object label is a set of rules for transforming inputs into outputs. When we know the rules we can write them by hand, which is programming. When the rules are unknown, or too complex with too many exceptions and special cases, it is easier to provide data or examples. Machine learning defines a program that acquires the input-output relationship from data.

Example: *A line as a program.* With two input coordinates x_1, x_2, the rule "if x is below the line, output $\circ$, else $\times$" is the test $x_1\theta_1 + x_2\theta_2 + \theta_3 \leq 0$. Appending a constant coordinate, $\vec{x} = (x_1, x_2, 1)$, the test becomes $\vec{x}^T\vec{\theta} \leq 0$, and learning means choosing $\vec{\theta} = (\theta_1, \theta_2, \theta_3)$ so that this parameterized program gives the right answer.

In general we learn a PARAMETERIZED FUNCTION $f(x, \theta) = y$, also written $f_\theta(x) = y$, with input x, output y (for example a label), and learned parameters θ. Crucially, $f_\theta(x)$ can be almost any expression of x and θ. The question is which parameterization to use: for an image, a line through raw pixel values does not separate dogs from cats.

1.3 From shallow to deep learning

SHALLOW LEARNING is a compromise: instead of hand-programming the rules, hand-program a fixed FEATURE extractor $\phi(x)$ (for example histograms of edge orientations), and learn a simple rule on top of it, such as $\phi(x)^T\theta \leq 0$, just like the two-dimensional example. Coming up with good features is very hard.

Deep learning also learns the parameters of the feature extractor. Rather than a single learned map, it stacks several layers: each layer is a simple parameterized transformation of the preceding one, and higher levels of representation (Figure 1) are more abstract, more invariant to nuisances, and easier to use for predicting the label. In networks trained on images, for example, the lowest layer responds to colors and edges, the next to parts, and the top to whole objects.

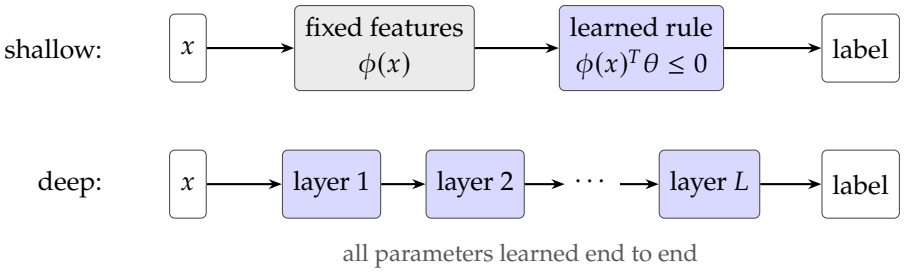


Figure 1: Shallow learning applies a learned linear rule to hand-designed features; deep learning learns every layer of the representation.

Definition 1.1 [DEEP LEARNING]. DEEP LEARNING is machine learning with multiple layers of learned representations. The function from input to internal representation to output is usually a DEEP NEURAL NETWORK, although almost any multi-layer parametric function with learned parameters can be called a neural network. The parameters of every layer are usually (but not always) trained with respect to the overall task objective, such as accuracy; this is called END-TO-END learning.

1.4 A brief history

- 1950: Turing describes learning as a path to machine intelligence.

- 1957: Rosenblatt's *perceptron* is proposed as a practical learning method.
- 1969: Minsky and Papert describe fundamental limitations of neural networks; most (but not all) mainstream research turns to shallow learning.
- 1986: backpropagation becomes a practical method for training deep networks; 1989: LeNet recognizes handwriting.
- 1990s–2000s: a wave of interest in probabilistic methods and convex optimization, mostly with shallow models.
- Around 2006: deep neural networks start gaining attention again.
- 2012: Krizhevsky's AlexNet beats all other methods on ImageNet.

1.5 What makes deep learning work

Three ingredients came together.

1. *Big models with many layers*: LeNet (1989) has 7 layers, AlexNet (2012) 8, and ResNet-152 (2015) 152. On the ImageNet challenge the error fell as depth grew (Figure 2).
2. *Large datasets*: MNIST (handwritten digits, 1990s to today) has 60,000 images, Caltech 101 (2003) about 9,000, CIFAR-10 (2009) about 60,000, and ImageNet (ILSVRC, 2009) 1.5 million.
3. *Enough compute*: accuracy on ImageNet rises with the number of operations per forward pass, and training BERT, a large language model, takes about 54 hours on about 16 TPUs.

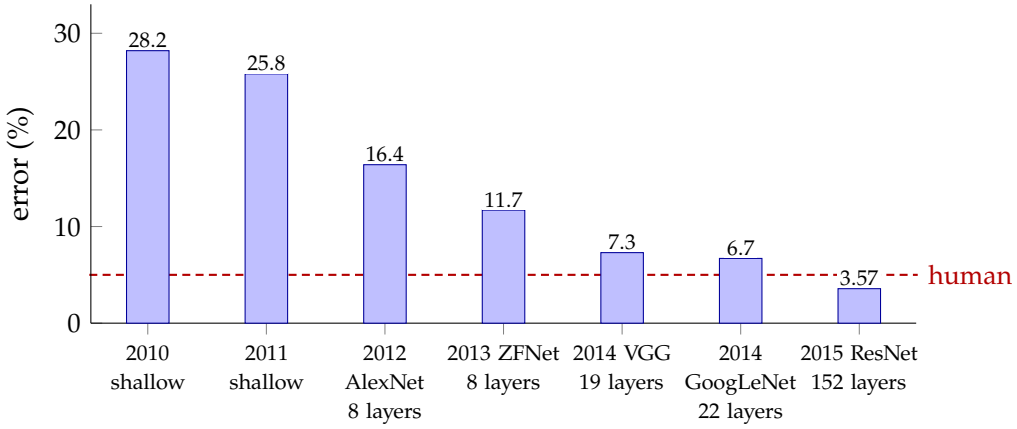


Figure 2: Error rate (%) of leading ImageNet (ILSVRC) classification entries by year, labeled with network depth. The dashed line marks human performance, about 5% error, as measured by one human (Andrej Karpathy) competing against a ConvNet.

One perspective is that deep learning is a bad idea, since it requires huge models, huge datasets, and huge compute. The other is that it is great, since the models keep getting better as we add data, layers, and compute.

1.6 The underlying themes

Definition 1.2 [CAPACITY, INDUCTIVE BIAS, SCALING]. Informally:

- MODEL CAPACITY is how many different functions a model class can represent (for example, all linear decision boundaries versus non-linear ones);
- INDUCTIVE BIAS is the built-in knowledge or bias in a model, designed to help it learn; all such knowledge is "bias" in the sense that it makes some solutions more likely and others less likely;
- SCALING is the ability of an algorithm to work better as more data and model capacity are added.

Deep learning acquires representations with high-capacity models and lots of data, without manual engineering of features. This brings automation (we need not know what the good features are) and better performance (representations learned end to end are tailored to the task). It also shifts the balance between learning and inductive bias ("nature versus nurture"): the models get most of their performance from data rather than from designer insight. Inductive bias can never be fully removed, which raises the recurring question of whether to build in knowledge or better machinery for learning and scale. *Algorithms that scale* are methods that get better and better as we add data, representational capacity, and compute.

1.7 Why "neural" networks

Neural networks were first proposed as a rudimentary model of the brain. A biological neuron receives signals from other neurons through its dendrites, "decides" whether to fire based on them, and transmits a signal along its axon to downstream neurons. An artificial neuron, or UNIT, sums the activations of upstream units,

$$z = \sum_i a_i,$$

"decides" how much to fire through an ACTIVATION FUNCTION, $a = \sigma(z)$, and transmits a to the units of the next layer. Common activation functions are the sigmoid $1/(1 + e^{-z})$, tanh, and the ReLU $\max(0, z)$ (Figure 3).

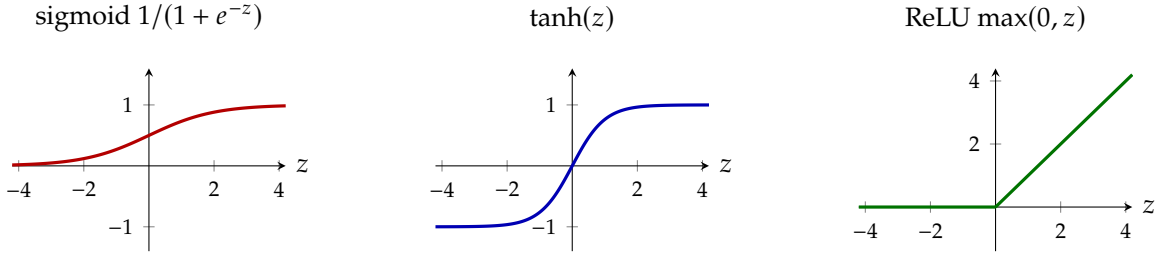


Figure 3: Three activation functions, plotted from their formulas.

This crudely models some neuron function but misses many anatomical details, so it should not be taken too seriously. Still, unsupervised learning models trained on natural images, sounds, and touch develop receptive fields whose statistics resemble those measured in primary sensory cortex (Saxe et al.). This could mean that the brain does something like deep learning, or that any sufficiently powerful learning machine arrives at basically the same solution, as birds and airplanes both use wings.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 1 (Spring 2021).

Lecture 2: ML Basics 1

2.1 Types of learning problems

- Supervised learning: given $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, learn $f_\theta(x) \approx y$; for example, predict an object label from an image.
- Unsupervised learning: from unlabeled data, learn a representation. What that means varies: *generative modeling* (GANs, VAEs, pixel RNNs) learns to produce new samples such as faces or animals, while *self-supervised representation learning* (for example BERT, or solving puzzles made from image patches) trains on tasks built from the data itself.
- Reinforcement learning: an agent observes a state s_t, takes an action a_t, and receives a reward; choose $f_\theta(s_t) = a_t$ to maximize $\sum_{t=1}^{H} r(s_t, a_t)$. Supervised learning gets $f_\theta(x_i)$ to match y_i; reinforcement learning gets $f_\theta(s_t)$ to maximize a reward that could be anything, so it subsumes (generalizes) supervised learning. For an animal the actions are muscle contractions, the observations sight and smell, and the reward food; for a robot, motor torques, camera images, and a task success measure such as running speed; for inventory management, what to purchase, inventory levels, and profit. Other applications include education, video and ad recommendations, and healthcare.

2.2 Supervised learning

The overwhelming majority of machine learning used in industry is supervised learning: all prediction and recognition models trained from ground-truth data, with simple basic principles. Examples are predicting the category of an object from an image, a French sentence from an English one, the presence of disease from an X-ray, or the text of a phrase from an audio utterance. Three questions must be answered.

1. How do we represent $f_\theta(x)$? For example $f_\theta(x) = \theta_1 x_1 + \theta_2 x_2 + \theta_3$, or $f_\theta(x) = \theta_1 x + \theta_2 x^2 + \theta_3 x^3$.
2. How do we measure the difference between $f_\theta(x_i)$ and y_i? For example $\|f_\theta(x_i) - y_i\|^2$, or $\delta(f_\theta(x_i) \neq y_i)$, or through probabilities.
3. How do we find the best θ? For example by least squares, random search, or gradient descent.

2.3 Predicting probabilities

Prediction is difficult: a sloppy handwritten digit might be a 5 with probability 90%, a 6 with 8%, and an 8 with 2%, and another might be a 3 with 40%, a 5 with 30%, and a 6 with 20%. Predicting

probabilities often makes more sense than predicting discrete labels, and it is also easier to learn because of smoothness: a discrete label cannot be changed "a tiny bit," but a probability can. So instead of $f_\theta(x) \approx y$ we learn $p_\theta(y \mid x)$.

Here x is a random variable representing the input and y one representing the output. The chain rule and the definition of conditional probability give

$$p(x, y) = p(x)\, p(y \mid x), \qquad p(y \mid x) = \frac{p(x, y)}{p(x)}.$$

2.4 Representing $p(y \mid x)$: the softmax

With 10 possible labels the program outputs 10 numbers that are positive and sum to 1. For two classes, try one linear function per class, $f_{\text{dog}}(x) = x^T \theta_{\text{dog}}$ and $f_{\text{cat}}(x) = x^T \theta_{\text{cat}}$, with $\vec{\theta} = \{\theta_{\text{dog}}, \theta_{\text{cat}}\}$, and set $p(y \mid x) = \text{softmax}(f_{\text{dog}}(x), f_{\text{cat}}(x))$, where the softmax could be any function (ideally one to one and onto) that turns these inputs into probabilities that are positive and sum to 1.

A number z can be made positive by z^2, $|z|$, $\max(0, z)$, or $\exp(z)$. The exponential is especially convenient because it is one to one and onto, mapping the whole real line onto the positive reals (though any of these would work). A collection of positive numbers is made to sum to 1 by dividing by their sum, $z_1 / \sum_{i=1}^n z_i$. Thus

$$\text{softmax}_{\text{dog}}(f_{\text{dog}}(x), f_{\text{cat}}(x)) = \frac{\exp(f_{\text{dog}}(x))}{\exp(f_{\text{dog}}(x)) + \exp(f_{\text{cat}}(x))}.$$

There is nothing magical about this, and it is not the only way; the numbers just need to be positive and sum to 1.

Definition 2.1 [Softmax]. With N possible labels, $p(y \mid x)$ is a vector with N elements and $f_\theta(x)$ a vector-valued function with N outputs. The softmax sets

$$p(y = i \mid x) = \text{softmax}(f_\theta(x))[i] = \frac{\exp(f_{\theta,i}(x))}{\sum_{j=1}^N \exp(f_{\theta,j}(x))}.$$

With two linear classes, $P(\text{red} \mid x) = e^{\theta_{\text{red}}^T x} / (e^{\theta_{\text{red}}^T x} + e^{\theta_{\text{blue}}^T x})$. The decision boundary $P(\text{red} \mid x) = P(\text{blue} \mid x) = 0.5$ is a hyperplane, and as $\theta_y^T x$ grows, $p(y \mid x)$ grows. In one dimension (Figure 4), points far on one side are "definitely blue," points far on the other "definitely red," and points near the boundary "not sure." The numerator makes the probability increase exponentially as we move away from the boundary, and the denominator is the normalizer. Scaling the parameters by 5 or 100 sharpens the curve toward a step: the result looks like $\max_y \theta_y^T x$, which is why the function is called a *soft*max.

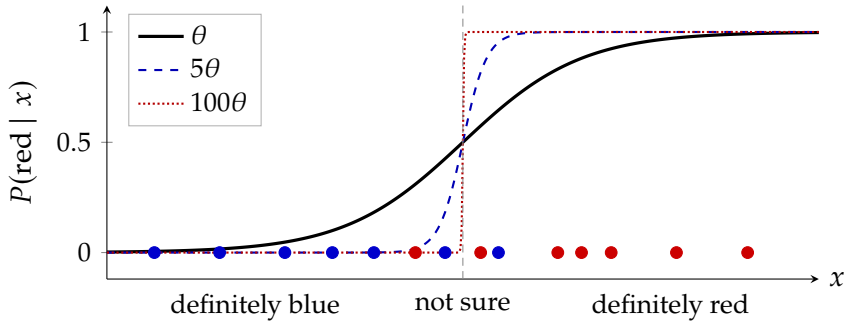


Figure 4: $P(\text{red} \mid x)$ for a one-dimensional input, with the parameters scaled by 1, 5, and 100. Larger scales approach a hard max. Schematic.

2.5 The machine learning method

The machine learning method for solving any problem ever:

1. define your *model class*: how to represent the "program";
2. define your *loss function*: how to measure whether one model in the class is better than another;
3. pick your *optimizer*: how to search the model class for the model that minimizes the loss;
4. run it on a big GPU.

This mirrors Marr's levels of analysis: the computational level ("why?", the loss function), the algorithmic level ("what?", the model), and the implementation level ("how?", the optimization algorithm), with "on which GPU?" below them. There are many variants on this basic idea.

2.6 Maximum likelihood and the loss function

How is the dataset "generated"? A photo is drawn from a distribution over photos, $x \sim p(x)$, and its label from a conditional distribution over labels, $y \sim p(y \mid x)$, so $(x, y) \sim p(x, y)$. The key assumption about the training set $\mathcal{D}$ is that its examples are INDEPENDENT AND IDENTICALLY DISTRIBUTED (I.I.D.): every (x_i, y_i) is independent of every other (x_j, y_j), and $(x_i, y_i) \sim p(x, y)$ with exactly the same distribution for all i. Then

$$p(\mathcal{D}) = \prod_i p(x_i, y_i) = \prod_i p(x_i)\, p(y_i \mid x_i).$$

We learn $p_\theta(y \mid x)$, a "model" of the true $p(y \mid x)$. A good model should make the data look probable, so choose θ to maximize $p(\mathcal{D}) = \prod_i p(x_i) p_\theta(y_i \mid x_i)$. Multiplying together many numbers ≤ 1 is numerically problematic, so take logarithms:

$$\log p(\mathcal{D}) = \sum_i \big[\log p(x_i) + \log p_\theta(y_i \mid x_i)\big] = \sum_i \log p_\theta(y_i \mid x_i) + \text{const},$$

since $p(x_i)$ does not depend on θ. Hence

$$\theta^\star \leftarrow \arg\max_\theta \sum_i \log p_\theta(y_i \mid x_i) \qquad \text{(MAXIMUM LIKELIHOOD ESTIMATION, MLE)},$$
$$\theta^\star \leftarrow \arg\min_\theta -\sum_i \log p_\theta(y_i \mid x_i) \qquad \text{(NEGATIVE LOG-LIKELIHOOD, NLL)},$$

and the NLL is our loss function.

In general a LOSS FUNCTION quantifies how *bad* θ is, and we want the least bad (best) θ. Examples:

- negative log-likelihood $-\sum_i \log p_\theta(y_i \mid x_i)$, also called the CROSS-ENTROPY. The cross-entropy $H(p, p_\theta) = -\sum_y p(y \mid x_i) \log p_\theta(y \mid x_i)$ measures how similar two distributions are; with $y_i \sim p(y \mid x_i)$ it is estimated by the single sample, $H(p, p_\theta) \approx -\log p_\theta(y_i \mid x_i)$;
- the zero-one loss $\sum_i \delta(f_\theta(x_i) \neq y_i)$;
- the mean squared error $\sum_i \frac{1}{2} \| f_\theta(x_i) - y_i \|^2$, which is also a negative log-likelihood: if $p_\theta(y \mid x) = \mathcal{N}(f_\theta(x), I)$, then $-\log p_\theta(y_i \mid x_i) = \frac{1}{2} \| f_\theta(x_i) - y_i \|^2 + \text{const}$.

2.7 Optimization: gradient descent

The loss $\mathcal{L}(\theta) = -\sum_i \log p_\theta(y_i \mid x_i)$ defines a "landscape" over parameter space. A generic algorithm repeats: find a direction v in which $\mathcal{L}(\theta)$ decreases, then set $\theta \leftarrow \theta + \alpha v$, where the small constant α is the LEARNING RATE or STEP SIZE. In one dimension, a negative slope means go right and a positive slope go left; in general, go in each dimension opposite the slope along that dimension, $v_k = -d\mathcal{L}(\theta)/d\theta_k$. Collecting the slopes into the GRADIENT $\nabla_\theta \mathcal{L}(\theta) = \left(d\mathcal{L}/d\theta_1, \ldots, d\mathcal{L}/d\theta_n \right)^T$ gives GRADIENT DESCENT:

$$\text{repeat:} \quad \text{compute } \nabla_\theta \mathcal{L}(\theta), \qquad \theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta).$$

2.8 Logistic regression

Putting the pieces together, LOGISTIC REGRESSION has model class

$$f_\theta(x) = \begin{bmatrix} x^T \theta_{y_1} \\ \vdots \\ x^T \theta_{y_m} \end{bmatrix} = x^T \theta, \qquad p_\theta(y = i \mid x) = \text{softmax}(f_\theta(x))[i] = \frac{\exp(f_{\theta,i}(x))}{\sum_{j=1}^m \exp(f_{\theta,j}(x))},$$

where θ is the matrix with columns $\theta_{y_1}, \ldots, \theta_{y_m}$; loss $\mathcal{L}(\theta) = -\sum_{i=1}^n \log p_\theta(y_i \mid x_i)$; and gradient descent as the optimizer.

With only two classes the parameterization is redundant: since $P(y_1 \mid x) + P(y_2 \mid x) = 1$, knowing

$P(y_1 \mid x)$ determines $P(y_2 \mid x)$. Multiplying the numerator and denominator by $e^{-\theta_{y_1}^T x}$,

$$P(y_1 \mid x) = \frac{e^{\theta_{y_1}^T x}}{e^{\theta_{y_1}^T x} + e^{\theta_{y_2}^T x}} = \frac{e^{\theta_{y_1}^T x - \theta_{y_1}^T x}}{e^{\theta_{y_1}^T x - \theta_{y_1}^T x} + e^{\theta_{y_2}^T x - \theta_{y_1}^T x}} = \frac{1}{1 + e^{-\theta_+^T x}}, \qquad \theta_+ = \theta_{y_1} - \theta_{y_2}.$$

This is the LOGISTIC EQUATION, also called a SIGMOID.

2.9 Empirical risk and true risk

The zero-one loss $\sum_i \delta(f_\theta(x_i) \neq y_i)$ is 1 if wrong and 0 if right. The RISK is the probability of getting it wrong, quantified by the expected loss, and generalizes to other losses such as the NLL:

$$\text{Risk} = E_{x \sim p(x),\, y \sim p(y|x)}\big[\mathcal{L}(x, y, \theta)\big].$$

During training we cannot sample $x \sim p(x)$; we only have $\mathcal{D}$, so we use the EMPIRICAL RISK

$$\text{Empirical risk} = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(x_i, y_i, \theta) \approx E_{x \sim p(x),\, y \sim p(y|x)}\big[\mathcal{L}(x, y, \theta)\big].$$

Supervised learning is (usually) EMPIRICAL RISK MINIMIZATION. Whether this matches true risk minimization depends on whether the approximation is good.

- OVERFITTING: the empirical risk is low but the true risk is high. It can happen if the dataset is too small or the model too powerful (too many parameters, too much capacity).
- UNDERFITTING: both the empirical and the true risk are high. It can happen if the model is too weak (too few parameters, too little capacity) or the optimizer is poorly configured (for example a wrong learning rate).

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 2 (Spring 2021).

Lecture 3: ML Basics 2

Supervised learning minimizes the empirical risk, a sample estimate of the true risk. Overfitting (low empirical risk, high true risk) and underfitting (both high) describe how this can fail. This lecture makes the two notions precise, introduces regularization to control them, and describes how data is split to diagnose them.

3.1 Regression and the squared error

The analysis is simplest for regression, where the program outputs a continuous distribution over a continuous number, though everything applies to classification too. With a normal (Gaussian) model $p_\theta(y \mid x) = \mathcal{N}(f_\theta(x), \Sigma_\theta(x))$,

$$\log p_\theta(y \mid x) = -\frac{1}{2}\big(f_\theta(x) - y\big)^T \Sigma_\theta(x)^{-1}\big(f_\theta(x) - y\big) - \frac{1}{2}\log|\Sigma_\theta(x)| + \text{const},$$

which for $\Sigma_\theta(x) = I$ is $-\frac{1}{2}\|f_\theta(x) - y\|^2 + \text{const}$. The negative log-likelihood is therefore the mean squared error loss $\mathcal{L}(\theta, x, y) = \frac{1}{2}\|f_\theta(x) - y\|^2$. It is a bit easier to analyze, but other losses can be analyzed too.

3.2 Error across datasets

To understand overfitting and underfitting more formally, ask how the error changes for different training sets (Figure 5). When overfitting, the training data is fitted well, the true function poorly, and the learned function looks different each time. When underfitting, the training data and the true function are both fitted poorly, and the learned function looks similar across datasets, even if we pool all the datasets together.

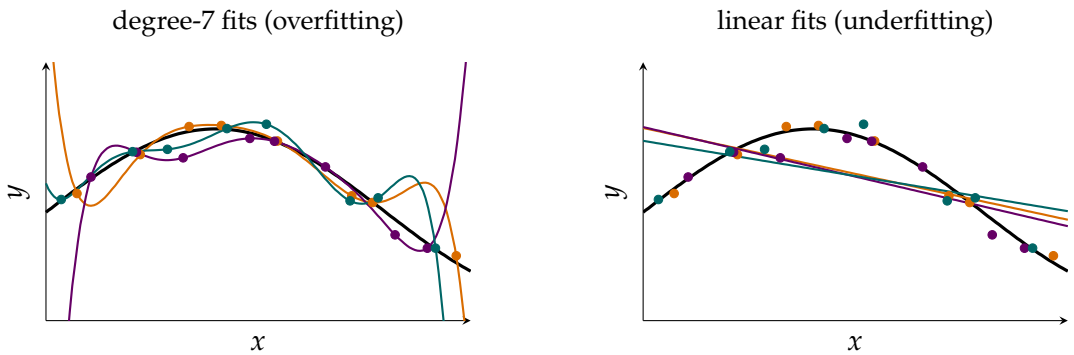


Figure 5: Fits to three random datasets (colors) drawn from the same true function (black). Schematic, with synthetic data.

A dataset $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ is random, with $p(\mathcal{D}) = \prod_{i=1}^{N} p(x_i)p(y_i \mid x_i)$. Let $f_{\mathcal{D}}$ be the function learned from $\mathcal{D}$ and f the true function. The expected error with respect to the data distribution, a sum over all possible datasets, is

$$E_{\mathcal{D}\sim p(\mathcal{D})}\left[\|f_{\mathcal{D}}(x) - f(x)\|^2\right] = \sum_{\mathcal{D}} p(\mathcal{D})\,\|f_{\mathcal{D}}(x) - f(x)\|^2.$$

It measures how well the *algorithm* does, independently of the particular random choice of dataset, which is what matters for improving the algorithm.

Theorem 3.1 [Bias-variance decomposition]. *Let* $\bar{f}(x) = E_{\mathcal{D}\sim p(\mathcal{D})}[f_{\mathcal{D}}(x)]$. *Then*

$$E_{\mathcal{D}\sim p(\mathcal{D})}\left[\|f_{\mathcal{D}}(x) - f(x)\|^2\right] = \underbrace{E_{\mathcal{D}\sim p(\mathcal{D})}\left[\|f_{\mathcal{D}}(x) - \bar{f}(x)\|^2\right]}_{\textit{Variance}} + \underbrace{\|\bar{f}(x) - f(x)\|^2}_{\textit{Bias}^2}.$$

Proof. Write $f_{\mathcal{D}} - f = (f_{\mathcal{D}} - \bar{f}) + (\bar{f} - f)$ and expand the square:

$$E\left[\|f_{\mathcal{D}} - \bar{f}\|^2\right] + E\left[\|\bar{f} - f\|^2\right] + 2\,E\left[f_{\mathcal{D}}(x) - \bar{f}(x)\right]^T\left(\bar{f}(x) - f(x)\right).$$

The cross term vanishes because $\bar{f} - f$ does not depend on $\mathcal{D}$ and $E[f_{\mathcal{D}}(x)] - \bar{f}(x) = 0$; for the same reason the second expectation is just $\|\bar{f}(x) - f(x)\|^2$. □

The variance asks how much the prediction changes with the dataset, regardless of what the true function is. The bias is the error of the average prediction, and it does not go away no matter how much data we have. If the variance is too high (too little data, too complex a function class), we are overfitting; if the bias is too high (an insufficiently complex function class), we are underfitting.

3.3 Regularization

How do we regulate the bias-variance tradeoff? Getting more data addresses the variance but has no effect on the bias. Changing the model class (for example from 12th-degree polynomials to linear functions) trades one for the other, but abruptly. We would like to restrict the model class smoothly, with a continuous knob for complexity.

Definition 3.2 [Regularization]. A regularizer is something we add to the loss function to reduce variance. In machine learning, any "heuristic" term added to the loss that does not depend on the data is generally called a regularizer.

The intuition: high variance means the data does not give enough information to identify the parameters. Many fits may all have zero training error; if the loss supplies enough extra information to disambiguate between (almost) equally good models, we can pick the best one, for example the smoothest.

The Bayesian perspective. One interpretation (not the only one) is a prior on parameters. Given $\mathcal{D}$, the most likely θ maximizes

$$p(\theta \mid \mathcal{D}) = \frac{p(\theta, \mathcal{D})}{p(\mathcal{D})} \propto p(\theta, \mathcal{D}) = p(\mathcal{D} \mid \theta)\, p(\theta),$$

where $p(\mathcal{D} \mid \theta) = \prod_i p(x_i) p_\theta(y_i \mid x_i)$ is the likelihood from before ($p_\theta(y_i \mid x_i)$ is shorthand for $p(y_i \mid x_i, \theta)$) and the PRIOR $p(\theta)$ says how likely θ is before we have seen $\mathcal{D}$. Taking negative logarithms gives the new loss function, in which we choose the last term:

$$\mathcal{L}(\theta) = -\Big(\sum_{i=1}^{N} \log p(y_i \mid x_i, \theta)\Big) - \log p(\theta).$$

Example: *Regularized linear regression.* Take polynomial features, $f_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots$. Wildly oscillating fits typically require large coefficients; if only small coefficients are allowed, the best fit is smoother. A distribution that assigns higher probability to small numbers is the Gaussian prior $p(\theta) = \mathcal{N}(0, \sigma^2)$ on each coordinate, for which

$$\log p(\theta) = \sum_{i=1}^{D} \Big(-\frac{1}{2}\frac{\theta_i^2}{\sigma^2} - \log \sigma - \frac{1}{2}\log 2\pi\Big) = -\lambda \|\theta\|^2 + \text{const}, \qquad \lambda = \frac{1}{2\sigma^2},$$

since the last two terms do not influence θ. With $\Sigma_\theta(x) = I$ the loss becomes

$$\mathcal{L}(\theta) = \sum_{i=1}^{N} \frac{1}{2}\| f_\theta(x_i) - y_i\|^2 + \lambda \|\theta\|^2.$$

The HYPERPARAMETER λ is selected directly rather than through σ.

Example: *Regularized logistic regression.* On separable data the maximum-likelihood boundary can contort so that technically every point is classified correctly. Since $P(\text{red} \mid x) = e^{\theta_{\text{red}}^T x}/(e^{\theta_{\text{red}}^T x} + e^{\theta_{\text{blue}}^T x})$, larger θ gives sharper probabilities (Lecture 2), so the solution is to prefer smaller θ, with the same Gaussian prior:

$$\mathcal{L}(\theta) = -\Big(\sum_{i=1}^{N} \log p(y_i \mid x_i, \theta)\Big) + \lambda \|\theta\|^2.$$

For classification this is sometimes called WEIGHT DECAY.

Other regularizers include L1 REGULARIZATION $\lambda \sum_{i=1}^{D} |\theta_i|$, whose corners create a preference for zeroing out dimensions, as opposed to L2 REGULARIZATION $\lambda \sum_{i=1}^{D} \theta_i^2$ (Figure 6); dropout, a special regularizer for neural networks; and the gradient penalty, a special regularizer for GANs.

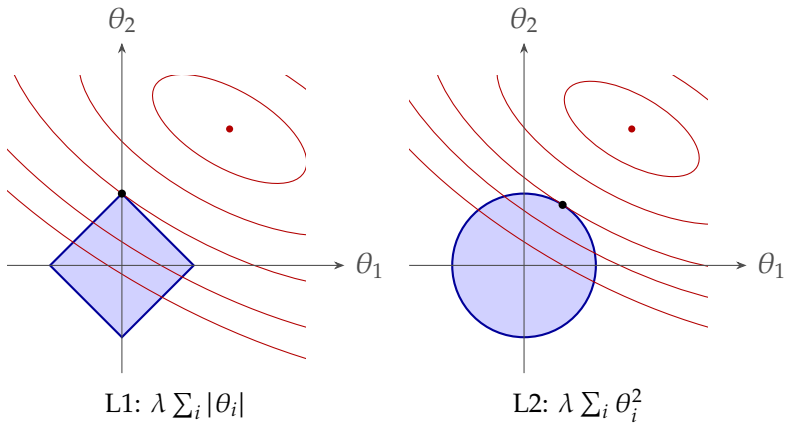


Figure 6: Contours of the data loss (red) first touch the L1 ball at a corner, where $\theta_1 = 0$, but touch the L2 ball at a point with both coordinates non-zero. Schematic.

Besides the Bayesian perspective (the regularizer is prior knowledge about the parameters), there are others. In the *numerical* perspective, the regularizer makes underdetermined problems well-determined. In the *optimization* perspective, it makes the loss landscape easier to search; paradoxically, a regularizer can then reduce underfitting if that was due to poor optimization, which is especially common with GANs. Regularizers introduce hyperparameters that must be selected for them to work well.

3.4 Training, validation, and test sets

How do we know whether we are overfitting or underfitting, which algorithm to use, and which hyperparameters? Choosing whatever makes the training loss low does not work: an overfit model can reach $\mathcal{L}(\theta) = 0$, so overfitting cannot be diagnosed from the training loss.

Instead, split the dataset into a TRAINING SET, used for training, and a VALIDATION SET, reserved for selecting hyperparameters, adding or removing features, and tweaking the model class. The workflow alternates two checks.

1. Train θ with $\mathcal{L}(\theta, \mathcal{D}_{\text{train}})$. If $\mathcal{L}(\theta, \mathcal{D}_{\text{train}})$ is not low enough, we are underfitting: decrease regularization or improve the optimizer.
2. Look at $\mathcal{L}(\theta, \mathcal{D}_{\text{val}})$. If $\mathcal{L}(\theta, \mathcal{D}_{\text{val}}) \gg \mathcal{L}(\theta, \mathcal{D}_{\text{train}})$, we are overfitting: increase regularization.

The training set selects θ (by optimization) and the optimizer's hyperparameters, such as the learning rate α. The validation set selects the model class (for example logistic regression versus a neural network), the regularizer's hyperparameters, and which features to use.

LEARNING CURVES plot both losses against the number of gradient-descent steps, which raises the question of how we know when to stop. When both curves level off close together, the training loss that remains measures the bias.

Once the recipe is followed, how good is the final classifier? $\mathcal{L}(\theta, \mathcal{D}_{\text{val}})$ is no good as an answer, since the validation set was already used to pick hyperparameters. So reserve a third set as a final exam (a kind of validation validation set): the TEST SET, used only to report final performance and nothing else.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 3 (Spring 2021).

Lecture 4: Optimization

4.1 How gradient descent behaves

Gradient descent minimizes the loss $\mathcal{L}(\theta) = -\sum_i \log p_\theta(y_i \mid x_i)$ by repeating $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$. Picture it on the level set contours of the loss: curves of θ values along which $\mathcal{L}(\theta)$ takes the same value. The gradient is perpendicular to the contours, so in a long narrow valley the first step $\theta_1 = \theta_0 - \alpha \nabla_\theta \mathcal{L}(\theta_0)$ points mostly across the valley rather than along it. Gradient descent does not always move toward the optimum: it zigzags across the valley and makes slow progress along it (Figure 7). The steepest direction is not always the best.

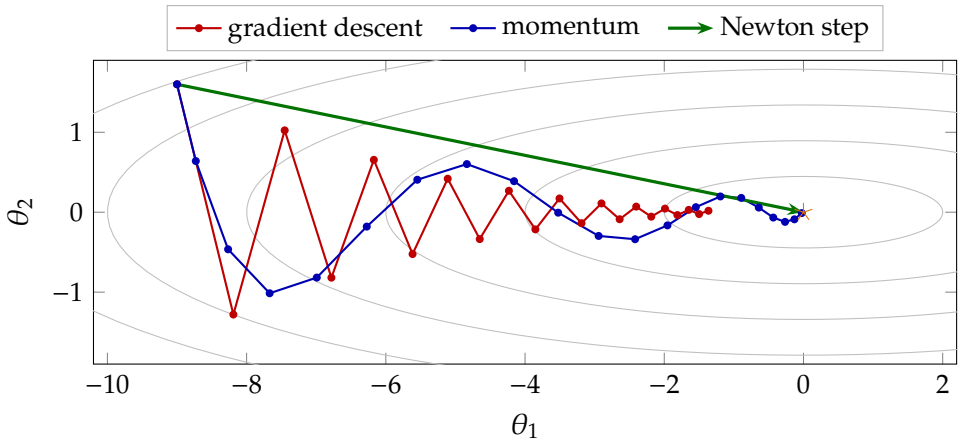


Figure 7: Twenty steps of gradient descent ($\alpha = 0.09$) and of momentum ($\alpha = 0.03$, $\mu = 0.75$) from $\theta_0 = (-9, 1.6)$ on $\mathcal{L}(\theta) = \frac{1}{2}(\theta_1^2 + 20\theta_2^2)$, computed from the update rules. The Newton step jumps directly to the optimum (star).

4.2 The loss surface

Definition 4.1 [Convexity]. A function is convex if the line segment between any two points on its graph lies entirely above the graph.

Convex functions are "nice": every road leads to the global minimum, and simple algorithms like gradient descent have strong guarantees (although that does not mean gradient descent works well for all convex functions). The negative log-likelihood of logistic regression, $p_\theta(y = i \mid x) = \exp(x^T \theta_i) / \sum_{j=1}^m \exp(x^T \theta_j)$, is guaranteed to be convex, which is not an obvious or trivial statement.

The loss of a neural network is not. It is hard to visualize because networks have very many parameters, but low-dimensional slices (Li et al., *Visualizing the Loss Landscape of Neural Nets*) show a chaotic surface for some networks and a much smoother one for others, for example with skip

connections. A non-convex landscape has three characteristic features (Figure 8).

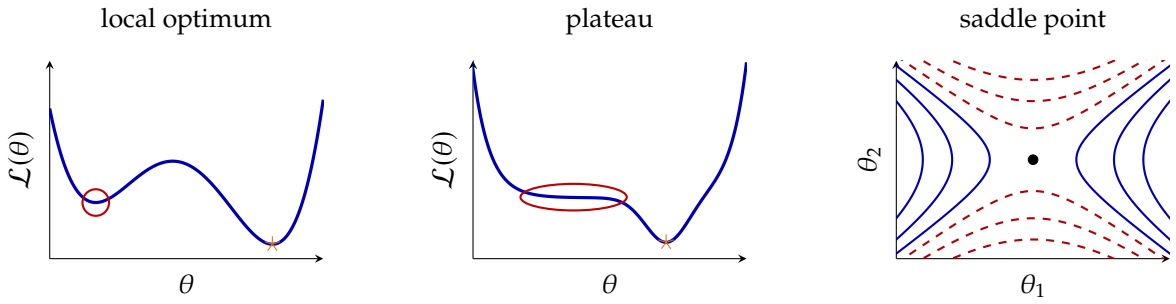


Figure 8: The geography of a loss landscape. Left: a local optimum (circled) and the global optimum (star). Middle: a plateau. Right: contours of the saddle $\theta_1^2 - \theta_2^2$, positive (solid) and negative (dashed) around the critical point at the origin. Schematic.

- *Local optima* are the most obvious issue with non-convex losses, and one of the big reasons people used to worry about neural networks: in principle gradient descent could converge to a solution arbitrarily worse than the global optimum. A bit surprisingly, this becomes less of an issue as the number of parameters grows; for big networks local optima exist but tend to be not much worse than global optima (Choromanska et al.). In histograms of the losses reached over many training runs, networks with more hidden units reach lower and more tightly concentrated losses.
- *Plateaus* are flat regions. A tiny learning rate cannot be used to prevent oscillation, because the learning rate must be large enough not to get stuck on a plateau. Momentum (below) helps.
- *Saddle points* are critical points where the loss curves up in some directions and down in others. The gradient near them is very small, so it takes a long time to get out. This seems like a very special structure, but in fact most critical points in neural network loss landscapes are saddle points.

A CRITICAL POINT is any point where $\nabla_\theta \mathcal{L}(\theta) = 0$. In one dimension it is a (local) minimum if $d^2\mathcal{L}/d\theta^2 > 0$ and a (local) maximum if $d^2\mathcal{L}/d\theta^2 < 0$. In higher dimensions the role of the second derivative is played by the HESSIAN matrix $\nabla^2_\theta \mathcal{L}(\theta)$, with entries $d^2\mathcal{L}/d\theta_i\, d\theta_j$. A critical point is a minimum or maximum only if all eigenvalues of the Hessian are positive, or all negative (for a diagonal Hessian such as $\left[\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right]$, the diagonal entries, here of mixed sign, so this point is a saddle). With many parameters, it is rarely the case that all eigenvalues share a sign.

4.3 Newton's method

A better direction than the gradient points straight at the optimum. A second-order Taylor expansion,

$$\mathcal{L}(\theta) \approx \mathcal{L}(\theta_0) + \nabla_\theta \mathcal{L}(\theta_0)^T(\theta - \theta_0) + \frac{1}{2}(\theta - \theta_0)^T \nabla^2_\theta \mathcal{L}(\theta_0)(\theta - \theta_0),$$

can be optimized analytically: setting its derivative to zero gives Newton's method,

$$\theta^\star \leftarrow \theta_0 - \big(\nabla^2_\theta \mathcal{L}(\theta_0)\big)^{-1} \nabla_\theta \mathcal{L}(\theta_0),$$

which reaches the minimum of a quadratic in one step (Figure 7). Newton's method is an ideal to aspire to, but we cannot afford it. With n parameters, a gradient step costs $O(n)$, while the Hessian has n^2 entries and the naive Newton step costs $O(n^3)$ (fancy methods can be much faster if they avoid forming the Hessian explicitly). We would much prefer methods that do not require second derivatives but somehow "accelerate" gradient descent.

4.4 Momentum

Averaging together successive gradients seems to yield a much better direction. If successive gradient steps point in different directions, we should cancel off the directions that disagree; if they point in similar directions, we should go faster in that direction.

Definition 4.2 [Momentum]. Momentum replaces the gradient $g_k = \nabla_\theta \mathcal{L}(\theta_k)$ in the update $\theta_{k+1} = \theta_k - \alpha g_k$ by

$$g_k = \nabla_\theta \mathcal{L}(\theta_k) + \mu g_{k-1},$$

which "blends in" the previous direction.

This very simple update rule brings in practice some of the benefits of Newton's method at virtually no cost. This kind of momentum has few guarantees; a closely related idea, Nesterov accelerated gradient, carries very appealing guarantees, but in practice we usually just use momentum.

4.5 Gradient scale: RMSProp, AdaGrad, and Adam

The sign of the gradient tells us which way to go along each dimension, but its magnitude is not so informative. For example, with $\mathcal{L}(\theta) = \frac{1}{2}\|f_\theta(x) - y\|^2$ the gradient $\nabla_\theta \mathcal{L}(\theta) = \big(f_\theta(x) - y\big)^T \frac{df}{d\theta}$ is huge when far from the optimum. Even worse, the overall magnitude can change drastically over the course of optimization, making learning rates hard to tune. The idea is to "normalize" out the magnitude of the gradient along each dimension. All operations below act per dimension.

Definition 4.3 [RMSProp and AdaGrad]. RMSProp estimates the per-dimension magnitude by a running average, roughly the squared length of each dimension, and divides each dimension by it:

$$s_k \leftarrow \beta s_{k-1} + (1-\beta)\big(\nabla_\theta \mathcal{L}(\theta_k)\big)^2, \qquad \theta_{k+1} = \theta_k - \alpha \frac{\nabla_\theta \mathcal{L}(\theta_k)}{\sqrt{s_k}}.$$

AdaGrad uses the cumulative magnitude instead, $s_k \leftarrow s_{k-1} + \big(\nabla_\theta \mathcal{L}(\theta_k)\big)^2$, with the same update.

AdaGrad has appealing guarantees for convex problems: its learning rate effectively decreases over time, which is good for convex problems, but only if the optimum is found quickly, before the rate decays too much. RMSProp tends to be much better for deep learning and most non-convex problems.

Adam combines momentum and RMSProp.

Algorithm 4.1: Adam

Input: learning rate α, decay rates β_1, β_2, small $\epsilon > 0$, initial θ_0

$m_0 \leftarrow 0$; $v_0 \leftarrow 0$

for $k = 1, 2, \ldots$ **do**

  $m_k \leftarrow (1-\beta_1)\nabla_\theta \mathcal{L}(\theta_k) + \beta_1 m_{k-1}$ *// first moment estimate (momentum-like)*

  $v_k \leftarrow (1-\beta_2)\big(\nabla_\theta \mathcal{L}(\theta_k)\big)^2 + \beta_2 v_{k-1}$ *// second moment estimate*

  $\hat{m}_k \leftarrow m_k/(1-\beta_1^k)$; $\hat{v}_k \leftarrow v_k/(1-\beta_2^k)$

  $\theta_{k+1} \leftarrow \theta_k - \alpha\, \hat{m}_k/\big(\sqrt{\hat{v}_k} + \epsilon\big)$

Since $m_0 = v_0 = 0$, the moment estimates are small early on, and the correction by $1 - \beta^k$ "blows them up" a bit for small k. The constant ϵ prevents division by zero. Good default settings are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

4.6 Stochastic gradient descent

Gradient descent is expensive because

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid x_i) \approx -E_{p_{\text{data}}(x,y)}\big[\log p_\theta(y \mid x)\big] \approx -\frac{1}{B}\sum_{j=1}^{B} \log p_\theta(y_{i_j} \mid x_{i_j})$$

requires summing over all datapoints in the dataset (ImageNet has 1.5 million images). The loss is itself an estimate of an expectation, so we could simply use far fewer samples, $B \ll N$, and still have a correct (unbiased) estimator.

Algorithm 4.2: Stochastic gradient descent with minibatches

Input: dataset $\mathcal{D}$ of size N, batch size B, learning rate α

for $k = 0, 1, 2, \ldots$ **do**

  sample a minibatch $\mathcal{B} \subset \mathcal{D}$ of B datapoints at random

  $g_k \leftarrow -\nabla_\theta \frac{1}{B}\sum_{i\in\mathcal{B}} \log p(y_i \mid x_i, \theta) \approx \nabla_\theta \mathcal{L}(\theta)$

  $\theta_{k+1} \leftarrow \theta_k - \alpha g_k$

Each iteration samples a different minibatch, and momentum, Adam, and the other methods apply unchanged with g_k in place of the gradient. In practice, sampling randomly is slow because of random memory access, so the dataset is shuffled once in advance (like a deck of cards) and batches are consecutive groups of B datapoints. One pass through the dataset is an epoch.

4.7 Learning rates and tuning

A high learning rate decreases the loss quickly at first but then stalls at a worse value, while a low learning rate makes slow progress (Figure 9). Somewhat counter-intuitively, low learning rates can also result in convergence to worse values, since they get stuck on plateaus that a larger step would cross; high rates instead bounce around near the optimum.

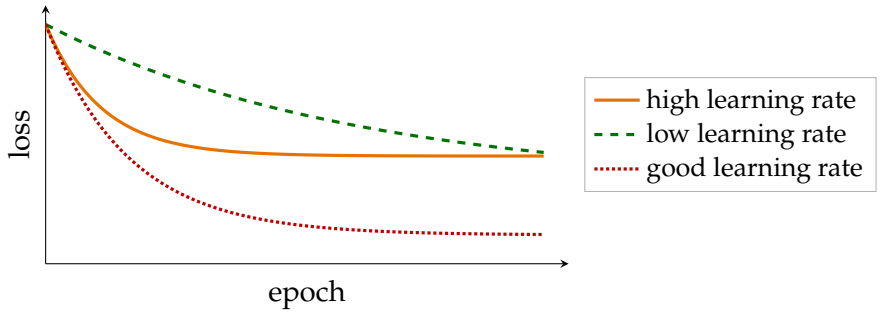


Figure 9: Training loss against epochs for different learning rates. Schematic.

Learning rate decay schedules are usually needed for the best performance with SGD (with momentum). AlexNet on ImageNet, for example, was trained with $\alpha = 0.01$, then 0.001, then 0.0001, and the training and validation accuracies jump up at each decay. Decay is often not needed with Adam. Opinions differ: some think SGD with momentum beats Adam if you want the very best performance, but Adam is easier to tune.

The hyperparameters of (stochastic) gradient descent are:

- the batch size B: larger batches give less noisy gradients and are usually "safer," but more expensive;
- the learning rate α: best to use the biggest rate that still works, and decay it over time;
- the momentum μ: 0.99 is good;
- the Adam parameters β_1, β_2: usually keep the defaults.

Technically these should be tuned on the training loss, since they are parameters of the optimization, but they are often tuned on the validation loss. The relationship between stochastic gradients and regularization is complex; some consider SGD a good regularizer, which suggests using the validation loss.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 4 (Spring 2021).

Lecture 5: Backpropagation

5.1 Computation graphs

A computation graph draws an expression as a program: nodes are variables and operations, and edges carry values. The graph in Figure 10 computes $\|(x_1\theta_1 + x_2\theta_2) - y\|^2$, the MSE loss of a linear regression model; with vectors x and θ it collapses to a dot product, a subtraction, and a square. Neural networks are computation graphs, so generic tools for computation graphs let us train many kinds of neural networks.

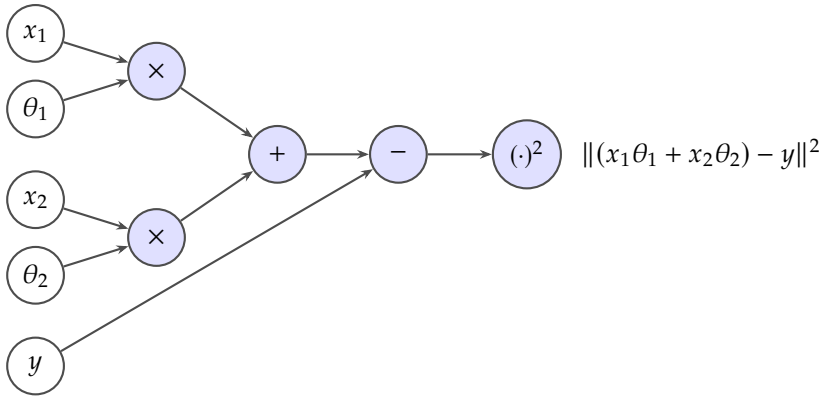


Figure 10: Computation graph of the squared error of a linear model.

For logistic regression with the negative log-likelihood loss,

$$p_\theta(y \mid x) = \frac{\exp(x^T\theta_y)}{\sum_{y'}\exp(x^T\theta_{y'})}, \qquad -\log p_\theta(y \mid x) = -x^T\theta_y + \log\sum_{y'}\exp(x^T\theta_{y'}).$$

In the graph, the label y enters as a one-hot vector (for two classes $(1,0)^T$ or $(0,1)^T$), so its product with the stacked scores $(x^T\theta_{y_1}, x^T\theta_{y_2})$ picks out $x^T\theta_y$. More simply, $f_\theta(x) = \theta x$ with θ a matrix whose rows are $\theta_{y_1}, \ldots, \theta_{y_m}$, followed by the softmax and the log.

The graph has two types of variables: *data* (x, y), which serve as input or target output, and *parameters* (θ). Parameters usually affect one specific operation (though there is often parameter sharing, as in convolutional networks), and y affects only the loss at the end. So we draw the network as a chain

$$x \;\to\; \text{linear layer } (\theta x) \;\to\; \text{softmax} \;\to\; \text{cross-entropy loss.}$$

A linear layer is also called a fully connected layer. Neural network diagrams omit the parameters (every layer has them) and often the loss (cross-entropy always follows softmax), and annotate each

vector with its dimension.

5.2 Learning the features

Logistic regression on hand-designed features, such as $\phi(x) = (x_1, x_2, x_1^2, x_2^2, x_1x_2)$, computes softmax$(\phi(x)^T\theta)$ and can separate data that is not linearly separable in x. To *learn* the features, make each feature a binary logistic regression output:

$$\phi_1(x) = \frac{1}{1+\exp(-x^T w_1^{(1)})}, \qquad \phi(x) = \begin{pmatrix} \sigma(x^T w_1^{(1)}) \\ \sigma(x^T w_2^{(1)}) \\ \sigma(x^T w_3^{(1)}) \end{pmatrix} = \sigma(W^{(1)}x),$$

where $w_i^{(1)}$ (the weights of feature i at layer 1) are the rows of the weight matrix $W^{(1)}$, and from now on θ denotes all parameters of the model. The per-element sigmoid σ is not a softmax: each feature is independent. Then $p(y \mid x) = \text{softmax}(W^{(2)}\sigma(W^{(1)}x))$, with intermediate vectors

$$z^{(1)} = W^{(1)}x, \qquad a^{(1)} = \sigma(z^{(1)}), \qquad z^{(2)} = W^{(2)}a^{(1)},$$

as in Figure 11. A sigmoid layer is a linear layer followed by a sigmoid. Doing this multiple times gives a deeper network, for example three sigmoid layers ($W^{(1)}$ of size 3×2, then $W^{(2)}, W^{(3)}$ of size 3×3) followed by a linear layer and a softmax.

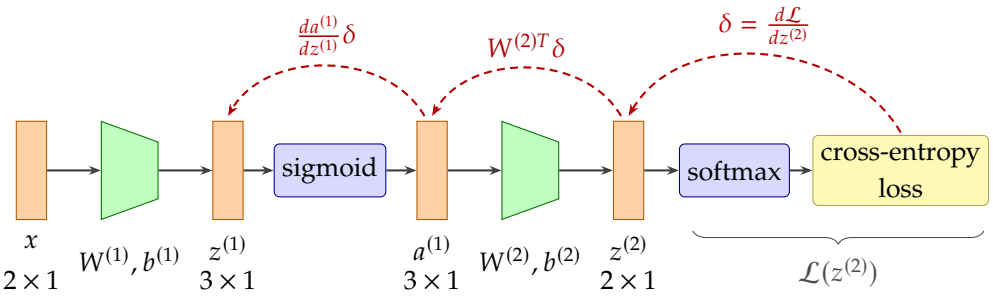


Figure 11: A two-layer network: forward pass (black) and the backward pass of the error signal δ (dashed red). Trapezoids are linear layers.

The sigmoid can be replaced by a wide range of non-linear ACTIVATION FUNCTIONS. Non-linearity is essential: if $\sigma(z) = z$, then $a^{(2)} = W^{(2)}W^{(1)}x = Mx$, and multiple linear layers are just one linear layer. With enough non-linear layers, we can represent anything.

5.3 The chain rule

To train the network we follow the usual recipe: the model class above, the negative log-likelihood loss, and stochastic gradient descent, which needs $\nabla_\theta \mathcal{L}(\theta)$. For $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$, and a scalar z, with

$y = g(x)$ and $z = f(y)$, the high-dimensional chain rule is

$$\frac{d}{dx_i} f(g(x)) = \sum_{j=1}^{m} \frac{dy_j}{dx_i} \frac{dz}{dy_j}, \qquad \text{that is,} \qquad \frac{dz}{dx} = \frac{dy}{dx} \frac{dz}{dy},$$

where $\frac{dz}{dy} \in \mathbb{R}^m$ is a column vector and the Jacobian $\frac{dy}{dx} \in \mathbb{R}^{n \times m}$ has entries $\left(\frac{dy}{dx}\right)_{ij} = \frac{dy_j}{dx_i}$. Some textbooks use the transposed convention, $\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$; these are just two different conventions.

A neural network is a composition of functions, so the chain rule gives its gradients. For the network of Figure 11,

$$\frac{d\mathcal{L}}{dW^{(1)}} = \frac{dz^{(1)}}{dW^{(1)}} \frac{da^{(1)}}{dz^{(1)}} \frac{dz^{(2)}}{da^{(1)}} \frac{d\mathcal{L}}{dz^{(2)}}, \qquad \frac{d\mathcal{L}}{dW^{(2)}} = \frac{dz^{(2)}}{dW^{(2)}} \frac{d\mathcal{L}}{dz^{(2)}},$$

and each Jacobian can be calculated; for example $z^{(2)} = W^{(2)} a^{(1)}$ gives $\frac{dz^{(2)}}{da^{(1)}} = W^{(2)T}$.

Multiplying from the left is a bad idea. If each $z^{(i)}$ or $a^{(i)}$ has about n dimensions, each Jacobian is about $n \times n$, and each matrix product costs $O(n^3)$; AlexNet has layers with 4096 units. Instead, *start on the right*. Because the loss is scalar-valued, $\frac{d\mathcal{L}}{dz^{(2)}}$ is always a vector ($n \times 1$), so every product is a matrix-vector product costing $O(n^2)$: compute $\delta = \frac{dz^{(2)}}{da^{(1)}} \frac{d\mathcal{L}}{dz^{(2)}}$ first, then $\gamma = \frac{da^{(1)}}{dz^{(1)}} \delta$, then $\frac{d\mathcal{L}}{dW^{(1)}} = \frac{dz^{(1)}}{dW^{(1)}} \gamma$.

5.4 The backpropagation algorithm

Write the network as a chain of functions f, each with input x_f and parameters θ_f; for example the last linear layer has $x_f = a^{(n-1)}$, $\theta_f = W^{(n)}$, and output $z^{(n)}$.

Algorithm 5.1: Backpropagation

Input: network $x \to f_1 \to \cdots \to f_L \to \mathcal{L}$, input x, target y
Output: $\frac{d\mathcal{L}}{d\theta_f}$ for every function f

forward pass: calculate each $z^{(i)}$ and $a^{(i)}$
initialize $\delta \leftarrow \frac{d\mathcal{L}}{dz^{(n)}}$
for each f with input x_f and parameters θ_f, from end to start **do**
    $\frac{d\mathcal{L}}{d\theta_f} \leftarrow \frac{df}{d\theta_f} \delta$
    $\delta \leftarrow \frac{df}{dx_f} \delta$

Each step multiplies a Jacobian by the vector δ, so backpropagation needs, for each type of function, only the two products $\frac{df}{d\theta_f} \delta$ and $\frac{df}{dx_f} \delta$, never the full Jacobians.

5.5 Practical implementation

An architecture must specify how many layers there are, how big they are, and what activation function they use, such as the sigmoid $\sigma(x) = 1/(1 + \exp(-x))$ or the RECTIFIED LINEAR UNIT $\mathrm{ReLU}(x) = \max(0, x)$.

A linear layer $z^{(i+1)} = W^{(i)}a^{(i)}$ always outputs 0 when $a^{(i)} = \vec{0}$. The solution is to add a BIAS, additional parameters in each linear layer (unrelated to the bias of the bias-variance tradeoff):

$$z^{(i+1)} = W^{(i)}a^{(i)} + b^{(i)}.$$

The backpropagation recipes are as follows; write $z = Wa + b$ to simplify notation.

Proposition 5.1 [LINEAR LAYER]. *For $z = Wa + b$,*

$$\frac{dz}{da}\delta = W^T\delta, \qquad \frac{dz}{dW}\delta = \delta a^T, \qquad \frac{dz}{db}\delta = \delta.$$

Proof. From $z_i = \sum_k W_{ik}a_k + b_i$: $\frac{dz_i}{da_k} = W_{ik}$, so $\frac{dz}{da} = W^T$ in the convention above. Next, $\frac{dz_i}{dW_{jk}} = a_k$ if $j = i$ and 0 otherwise, so $\frac{dz}{dW}\delta = \sum_i \frac{dz_i}{dW}\delta_i$, where $\frac{dz_i}{dW}$ is the matrix whose ith row is a^T and other rows are zero; the sum stacks the rows $\delta_i a^T$, giving δa^T, a matrix of the same shape as W. Finally $\frac{dz_i}{db_j} = \text{Ind}(i = j)$, so $\frac{dz}{db} = I$. □

Proposition 5.2 [SIGMOID AND RELU]. *For the per-element sigmoid $f_i(z) = \sigma(z_i)$ and ReLU $f_i(z) = \max(0, z_i)$, the Jacobian is diagonal and*

$$\left(\frac{df}{dz}\delta\right)_i = \big(1 - \sigma(z_i)\big)\sigma(z_i)\,\delta_i, \qquad \left(\frac{df}{dz}\delta\right)_i = \text{Ind}(z_i \geq 0)\,\delta_i.$$

Proof. For the sigmoid,

$$\frac{df_i}{dz_i} = \frac{\exp(-z_i)}{1+\exp(-z_i)} \cdot \frac{1}{1+\exp(-z_i)} = \left(\frac{1+\exp(-z_i)}{1+\exp(-z_i)} - \frac{1}{1+\exp(-z_i)}\right)\sigma(z_i) = \big(1 - \sigma(z_i)\big)\sigma(z_i).$$

For ReLU, the derivative is 1 for $z_i > 0$ and 0 for $z_i < 0$. □

The remaining recipe is the softmax followed by the cross-entropy loss. With logits z and label y, $\mathcal{L} = -z_y + \log\sum_j \exp(z_j)$, so $\frac{d\mathcal{L}}{dz_i} = \text{softmax}(z)[i] - \text{Ind}(i = y)$, which initializes δ.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 5 (Spring 2021).

Lecture 6: Convolutional Nets

6.1 Why convolutions

A fully connected network on images is enormous: a $128 \times 128 \times 3$ image has 49,152 values, so a first linear layer with a 64-dimensional output $z^{(1)}$ already has $64 \times 49{,}152 \approx 3{,}000{,}000$ weights. We need a better way.

Many useful image features are *local*: to tell whether a patch of the image contains a feature (an edge in the first layer; perhaps an ear or a nose in the second), it is enough to look at that local patch. So apply a small "mini-layer" to each $3 \times 3 \times 3$ patch: with a 64-dimensional output it has only $64 \times 27 = 1728$ weights, and we get a different output at each image location. This mini-layer is called a FILTER. Sliding a filter bank of depth 4 over a $128 \times 128 \times 3$ image produces a $128 \times 128 \times 4$ output, one channel per filter, to which we apply a non-linearity. Learned first-layer filters look like oriented edges and color blobs.

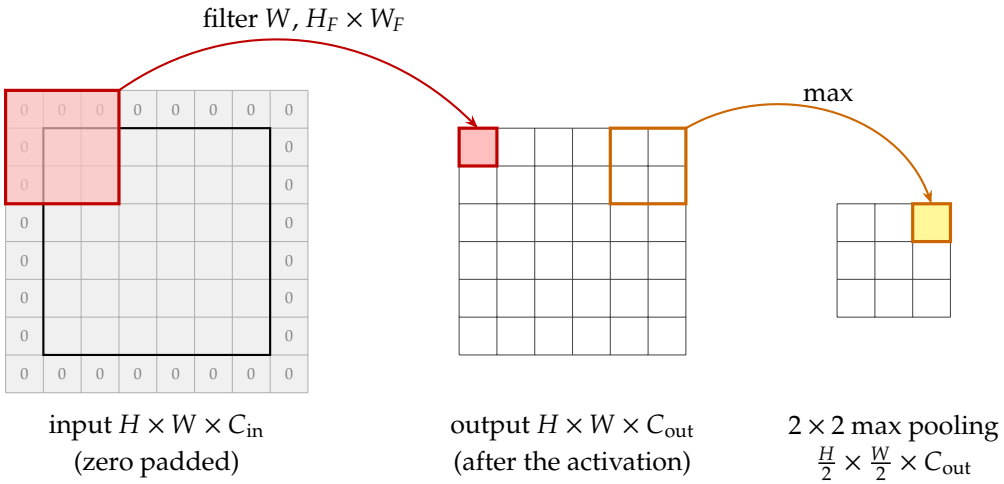


Figure 12: A 3×3 filter window (red) on the zero-padded input produces one entry of the output at each position; 2×2 max pooling (orange) keeps the maximum of each block, per channel. Schematic.

Definition 6.1 [Convolutional layer]. A CONVOLUTIONAL LAYER maps an activation $a^{(1)}$ of size $H_{\text{in}} \times W_{\text{in}} \times C_{\text{in}}$ to $z^{(2)}$ of size $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$ (with $H_{\text{out}}, W_{\text{out}}$ equal or almost equal to $H_{\text{in}}, W_{\text{in}}$), using a filter $W^{(2)}$ of size $H_F \times W_F \times C_{\text{out}} \times C_{\text{in}}$:

$$z^{(2)}[i,j,k] = \sum_{l=0}^{H_F-1} \sum_{m=0}^{W_F-1} \sum_{n=0}^{C_{\text{in}}-1} W^{(2)}[l,m,k,n]\, a^{(1)}\left[i + l - \frac{H_F-1}{2},\ j + m - \frac{W_F-1}{2},\ n\right],$$

followed by an activation function applied per element, $a^{(2)}[i,j,k] = \sigma\left(z^{(2)}[i,j,k]\right)$.

For a single channel the formula reduces to $z^{(2)}[i,j] = \sum_{l,m} W^{(2)}[l,m]\, a^{(1)}[i+l-\frac{H_F-1}{2},\ j+m-\frac{W_F-1}{2}]$: a simple principle, but a bit complicated to write.

All these operations involve N-dimensional arrays, often called TENSORS: an input image is HEIGHT×WIDTH×CHANNELS, a filter is FLT.HEIGHT×FLT.WIDTH×OUTPUT CHAN×INPUT CHAN, and activations are HEIGHT × WIDTH × LAYER.CHANNELS. The inner (rightmost) dimensions work just like vectors and matrices, while matching outer dimensions (height and width) are "broadcast," that is, treated elementwise. A convolution performs a tiny matrix multiply at each position, like a tiny linear layer at each position (Figure 12).

6.2 Pooling

A convolutional layer's output is still as big as the original image. To eventually get down to a single output we must reduce resolution as we go. MAX POOLING takes, for each channel, the maximum over (for example) each 2×2 region, so a $128 \times 128 \times 4$ activation becomes $64 \times 64 \times 4$. This downsamples the "image" and makes the network robust to small translations. Alternating convolutions and pooling, for example $128 \times 128 \times 3 \to 128 \times 128 \times 4 \to 64 \times 64 \times 4 \to 64 \times 64 \times 8 \to 32 \times 32 \times 8$, eventually gives something small enough to *flatten* into a vector and feed into a standard fully connected layer. LeNet, a network for handwritten digit recognition, has exactly this structure: convolutions, subsampling (pooling), convolutions, subsampling, and full connections.

6.3 Padding and stride

At the edges, part of the filter window falls outside the input.

- *Option 1: cut off the edges*, computing outputs only where the window fits. With "radius" $(H_F - 1)/2$ on each side, $H_{\text{out}} = H_{\text{in}} - 2 \cdot \frac{H_F-1}{2}$; for example a $32 \times 32 \times 3$ input with a $5 \times 5 \times 6$ filter gives $28 \times 28 \times 6$. The problem is that activations shrink with every layer, and some people do not like this.
- *Option 2: zero pad* the input. It is simple and preserves the size, at the cost of a weird effect at the boundary, which is usually not a problem; hence its popularity. Remember to subtract the image mean first (fancier contrast normalization is often used in practice).

The standard structure at each layer is: apply the convolution, $H \times W \times C_{\text{in}} \to H \times W \times C_{\text{out}}$; apply the activation σ; and apply pooling of width N, $H \times W \times C_{\text{out}} \to H/N \times W/N \times C_{\text{out}}$. This can be very expensive, a $C_{\text{out}} \times C_{\text{in}}$ matrix multiply at each position of an $H \times W$ image. The idea of a STRIDED CONVOLUTION is to skip over some positions; the amount of skipping is the STRIDE. Some people think strided convolutions are just as good as convolution plus pooling.

6.4 Examples of convolutional networks

AlexNet (Krizhevsky et al., 2012) is a "classic" medium-depth convolutional network, a bit like a modernized LeNet, and the first neural network to attain state-of-the-art results on the ImageNet challenge (ILSVRC: 1.5 million images, 1000 categories). It was trained on two GPUs, which is why its original diagram is split, but that no longer matters. Its layers are:

Layer	Filter / size	Map
CONV1	$11 \times 11 \times 96$, stride 4	$224 \times 224 \times 3 \to 55 \times 55 \times 96$ (no zero padding)
POOL1	3×3, stride 2	$55 \times 55 \times 96 \to 27 \times 27 \times 96$
NORM1	local normalization	
CONV2	$5 \times 5 \times 256$, stride 1	$27 \times 27 \times 96 \to 27 \times 27 \times 256$ (zero padding)
POOL2	3×3, stride 2	$27 \times 27 \times 256 \to 13 \times 13 \times 256$
NORM2	local normalization	
CONV3	$3 \times 3 \times 384$, stride 1	$13 \times 13 \times 256 \to 13 \times 13 \times 384$ (zero padding)
CONV4	$3 \times 3 \times 384$, stride 1	$13 \times 13 \times 384 \to 13 \times 13 \times 384$ (zero padding)
CONV5	$3 \times 3 \times 256$, stride 1	$13 \times 13 \times 384 \to 13 \times 13 \times 256$ (zero padding)
POOL3	3×3, stride 2	$13 \times 13 \times 256 \to 6 \times 6 \times 256$
FC6	4096×9216 matrix	$6 \times 6 \times 256 \to 9216 \to 4096$
FC7		$4096 \to 4096$
FC8		$4096 \to 1000$, then softmax

The 3×3 pooling with stride 2 uses overlapping regions. CONV1 has $11 \cdot 11 \cdot 3 \cdot 96 = 34{,}848$ weights and 96 biases, 34,944 parameters in total. A ReLU follows every convolutional or fully connected layer except the last. AlexNet was trained with data augmentation and dropout; its local normalization is not used much anymore, though other types of normalization are.

VGG is still often used today. It increased depth greatly over the previous best model (19 layers), and it introduced *homogeneous* stacks of convolutions interspersed with resolution reduction: blocks of 3×3 convolutions with 64, 128, 256, 512, 512 channels at resolutions 224, 112, 56, 28, 14 (two, two, three, three, and three convolutions), each block followed by 2×2 pooling, ending at $7 \times 7 \times 512$. Then come fully connected layers $25{,}088 \to 4096 \to 4096 \to 1000$ and a softmax. More layers mean more processing, which is why blocks are repeated. Almost all parameters are in the first fully connected layer ($7 \cdot 7 \cdot 512 \times 4096 \approx 10^8$), while most memory goes to the high-resolution activations of the early layers.

ResNet has 152 layers (with a small 18-layer prototype, ResNet-18). On CIFAR-10, plain networks get *worse* with depth (a 56-layer plain net has higher training and test error than a 20-layer one), while residual networks get better with depth, up to 110 layers. The main idea is the RESIDUAL BLOCK (Figure 13): instead of learning $H(x)$ directly with two stacked weight layers, learn $F(x)$ and output $H(x) = F(x) + x$ through an identity shortcut.

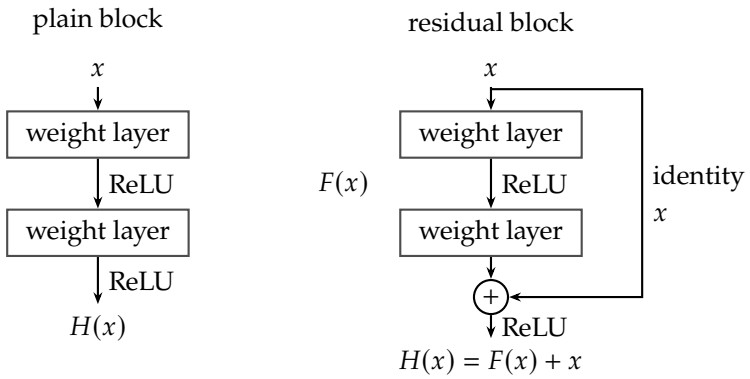


Figure 13: A plain block and a residual block with an identity shortcut.

Why are deep networks hard to train? By backpropagation,

$$\frac{d\mathcal{L}}{dW^{(1)}} = \frac{dz^{(1)}}{dW^{(1)}}\frac{da^{(1)}}{dz^{(1)}}\frac{dz^{(2)}}{da^{(1)}}\frac{d\mathcal{L}}{dz^{(2)}} = J_1 J_2 J_3 \cdots J_n \frac{d\mathcal{L}}{dz^{(n)}}.$$

Multiplying many numbers gives 0 if most of them are below 1 and infinity if most are above 1; only numbers close to 1 give a reasonable answer. For matrices this means we want $J_i \approx I$. In a plain block $\frac{dH}{dx}$ could be big or small, and is not close to I. In a residual block

$$\frac{dH}{dx} = \frac{dF}{dx} + I,$$

and if the weights are not too big, $\frac{dF}{dx}$ is small(ish), so the Jacobian is close to the identity.

ResNet uses generic blocks of many layers interspersed with a few pooling operations; it has no giant fully connected layer at the end, just a mean pool over all x, y positions and a small(ish) fully connected layer into the softmax; and its residual layers provide good gradient flow.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 6 (Spring 2021).

Lecture 7: Getting Neural Nets to Train

Even when everything from the previous lectures is implemented correctly and trained for a long time, there is a good chance a network still will not train. Neural networks are messy and need many "tricks": normalizing inputs, outputs, and activations; initializing weights and biases; gradient clipping; hyperparameter search; and ensembling and dropout.

7.1 Normalizing inputs and outputs

Suppose the first input dimension is much bigger than the second. For the first linear layer,

$$\frac{d\mathcal{L}}{dW^{(1)}} = \frac{dz^{(1)}}{dW^{(1)}}\frac{d\mathcal{L}}{dz^{(1)}} = \delta x^T,$$

so the gradient is scaled by the big x values. The loss landscape becomes an elongated valley instead of round contours, and gradient descent zigzags (Lecture 4). In general we want all entries of x roughly on the same scale. Sometimes this is automatic: image pixels are all roughly in $[0, 1]$ or $\{0, \ldots, 255\}$, and discrete inputs in NLP are one-hot. Sometimes it is a huge problem: to forecast the weather, temperature might range over 40 to 100 and humidity over 0.3 to 0.6.

Standardization transforms the inputs, per dimension, to have $\mu = 0$ and $\sigma = 1$:

$$\bar{x}_i = \frac{x_i - E[x]}{\sqrt{E[(x_i - E[x])^2]}}, \qquad E[x] \approx \frac{1}{N}\sum_{i=1}^{N} x_i.$$

The same applies to outputs when doing regression, and perhaps to activations.

7.2 Batch normalization

Activations inside the network can also develop very different scales per dimension. We can standardize them too, but their mean and standard deviation depend on $W^{(1)}$ and $b^{(1)}$ and change during training. For $z_i^{(1)} = W^{(1)}x_i + b^{(1)}$ and $a_i^{(1)} = \mathrm{ReLU}(z_i^{(1)})$, the basic idea is

$$\mu^{(1)} = \frac{1}{N}\sum_{i=1}^{N} a_i^{(1)}, \quad \sigma^{(1)} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(a_i^{(1)} - \mu^{(1)}\right)^2}, \quad \bar{a}_i^{(1)} = \frac{a_i^{(1)} - \mu^{(1)}}{\sigma^{(1)}}, \quad z_i^{(2)} = W^{(2)}\bar{a}_i^{(1)} + b^{(2)},$$

and so on, recomputing the statistics at every gradient step. Evaluating all points in the dataset at every step is too expensive, so compute the mean and standard deviation over the current batch only.

Definition 7.1 [Batch normalization]. For a minibatch $a^{(1)}_{i_1}, \ldots, a^{(1)}_{i_B}$, a batch normalization layer computes, per dimension,

$$\mu^{(1)} \approx \frac{1}{B}\sum_{j=1}^{B} a^{(1)}_{i_j}, \qquad \sigma^{(1)} \approx \sqrt{\frac{1}{B}\sum_{j=1}^{B}\left(a^{(1)}_{i_j} - \mu^{(1)}\right)^2}, \qquad \bar{a}^{(1)}_i = \frac{a^{(1)}_i - \mu^{(1)}}{\sigma^{(1)}}\gamma + \beta,$$

where the learnable scale γ and bias β have the same dimension as $\bar{a}^{(1)}_i$.

Batch normalization is trained with backpropagation like any other layer, differentiating through $\mu^{(1)}$ and $\sigma^{(1)}$ as well. There are two common placements:

- *ReLU then BN*: the scale and bias seemingly should be subsumed by the next linear layer, and all ReLU outputs are positive;
- *BN then ReLU*: the "classic" version, which just appears to be a transformation of the preceding linear layer.

No one seems to agree on the right way; try a few options and see what works (both often work). With batch normalization we can often use a larger learning rate, models can train much faster, and they generally need less regularization (for example, no dropout). It is a very good idea in many cases.

7.3 Weight initialization

We want the overall scale of activations not to be too big or too small for the initial (random) weights, so that gradients propagate well. Since

$$\frac{d\mathcal{L}}{dW^{(1)}} = J_1 J_2 \cdots J_n \frac{d\mathcal{L}}{dz^{(n)}},$$

a product of many numbers goes to 0 if they are mostly below 1 and to infinity if they are mostly above 1. Basic initialization keeps the activations on a reasonable scale that neither grows nor shrinks with depth; more advanced methods act on the eigenvalues of the Jacobians. Ideally we would initialize at a good solution, but we have no idea where that is; the goal is only well-behaved gradients and activations.

A simple choice, Gaussian random weights $W^{(i)}_{jk} \sim \mathcal{N}(0, 0.0001)$, is bad: in a deep network the activations shrink toward zero layer by layer, and since $\frac{d\mathcal{L}}{dW^{(i)}} = \delta a^{(i-1)T}$, the gradients vanish too.

Proposition 7.2 [Activation scale]. *Let $z_i = \sum_j W_{ij} a_j + b_i$ with $b_i \approx 0$, independent weights $W_{ij} \sim \mathcal{N}(0, \sigma_W^2)$, and zero-mean activations with $E[a_j^2] = \sigma_a^2$ independent of the weights, where a has dimension D_a. Then*

$$E[z_i^2] = \sum_j E[W_{ij}^2] E[a_j^2] = D_a \sigma_W^2 \sigma_a^2.$$

Proof. Expand $z_i^2 = \sum_{j,k} W_{ij} W_{ik} a_j a_k$. For $j \neq k$ the weights are independent with mean zero, so these cross terms have expectation $E[W_{ij}]E[W_{ik}]E[a_j a_k] = 0$, leaving the diagonal terms $E[W_{ij}^2]E[a_j^2]$. □

If $D_a \sigma_W^2 > 1$ the magnitude grows with each layer; if $D_a \sigma_W^2 < 1$ it shrinks. A standardized input $x \sim \mathcal{N}(0, 1)$ is a reasonable starting point, and choosing $\sigma_W^2 = 1/D_a$ preserves the scale. The basic principle, sometimes called Xavier initialization, is to make the standard deviation of W_{ij} about $1/\sqrt{D_a}$.

This ignored the nonlinearities. With $a_j = \mathrm{ReLU}(z_j)$, the negative half of the zero-mean activations is removed, so the variance is cut in half; it might not seem like much, but it adds up over layers. The fix, proposed by He et al. for ResNet, is a standard deviation of $1/\sqrt{\frac{1}{2} D_a}$, which makes a big difference with 150 or more layers. With ReLUs and biases $b_i \approx 0$, on average half of the units are "dead," so biases are often initialized to 0.1 (or another small constant).

For advanced initialization, write each weight matrix by its singular value decomposition, $W^{(i)} = U^{(i)} \Lambda^{(i)} V^{(i)}$, where $U^{(i)}$ and $V^{(i)}$ are scale-preserving transformations (orthonormal bases) and $\Lambda^{(i)}$ is diagonal. Only $\Lambda^{(i)}$ changes the scale, so force it to be the identity: $W^{(i)} \leftarrow U^{(i)} V^{(i)}$. Even simpler, take the SVD of an arbitrary random matrix and keep a factor, which is guaranteed orthonormal:

```
a = get_rng().normal(0.0, 1.0, flat_shape)  # arbitrary random matrix
u, _, v = np.linalg.svd(a, full_matrices=False)
# pick the one with the correct shape (needed if non-square)
q = u if u.shape == flat_shape else v
```

7.4 Gradient clipping

Sometimes a single gradient is enormous: the optimizer took a step that was too big in the wrong place, something got divided by something small (in batch norm, softmax, and so on), or it was just unlucky. One such step can throw the parameters far away. Gradient clipping limits these monster gradients, either per element or by norm:

$$\bar{g}_i \leftarrow \max\big(\min(g_i, c_i), -c_i\big), \qquad \bar{g} \leftarrow g \, \frac{\min(\|g\|, c)}{\|g\|}.$$

To choose c, run a few epochs (assuming training does not explode) and see what "healthy" magnitudes look like.

7.5 Ensembles

Neural networks have many parameters and often high variance, though not nearly as high as basic learning theory would suggest (overfitting is usually not catastrophic). When several high-variance learners are combined, they may agree on the right answer but disagree on the wrong ones: there are many more ways to be wrong than to be right.

Recall Variance $= E_{\mathcal{D}\sim p(\mathcal{D})}[\|f_{\mathcal{D}}(x) - \bar{f}(x)\|^2]$ with $\bar{f}(x) = E_{\mathcal{D}\sim p(\mathcal{D})}[f_{\mathcal{D}}(x)] \approx \frac{1}{M}\sum_{j=1}^{M} f_{\mathcal{D}_j}(x)$. Estimating $\bar{f}$ needs M different datasets. Instead of chopping one big dataset into M parts, the parts may overlap as long as they are independently sampled: for each $\mathcal{D}_j$ pick N indices $i_{j,1}, \ldots, i_{j,N}$ uniformly at random from $\{1, \ldots, N\}$ and set $\mathcal{D}_j = \{(x_{i_{j,1}}, y_{i_{j,1}}), \ldots, (x_{i_{j,N}}, y_{i_{j,N}})\}$. This is RESAMPLING WITH REPLACEMENT: from $\mathcal{D} = \{x_1, x_2, x_3\}$ we might draw $\mathcal{D}_1 = \{x_2, x_3, x_3\}$ and $\mathcal{D}_2 = \{x_3, x_1, x_3\}$. Train separate models $p_{\theta_1}(y \mid x), \ldots, p_{\theta_M}(y \mid x)$, one per $\mathcal{D}_j$, and predict either by the principled approach of averaging the probabilities,

$$p(y \mid x) = \frac{1}{M}\sum_{j=1}^{M} p_{\theta_j}(y \mid x),$$

or by the simple approach of a majority vote.

In practice neural network training already has plenty of randomness: random initialization, random minibatch shuffling, and stochastic gradient descent. So we get much of the same benefit without resampling, by training M models on the same $\mathcal{D}$. Faster variants:

- share the features (the somewhat task-agnostic convolutional layers, often the most expensive part) among all models, with a separate ensemble of classifier "heads" (the task-specific fully connected layers);
- SNAPSHOT ENSEMBLES (Huang et al.) save parameter snapshots over the course of one SGD run, for example with a cyclic learning rate schedule, and use each snapshot as a model. No separate training runs are needed, but things must be set up so that the snapshots are actually different. Predictions can be combined by averaging probabilities, by voting, or simply by averaging the parameter vectors.

One reported comparison (your mileage may vary):

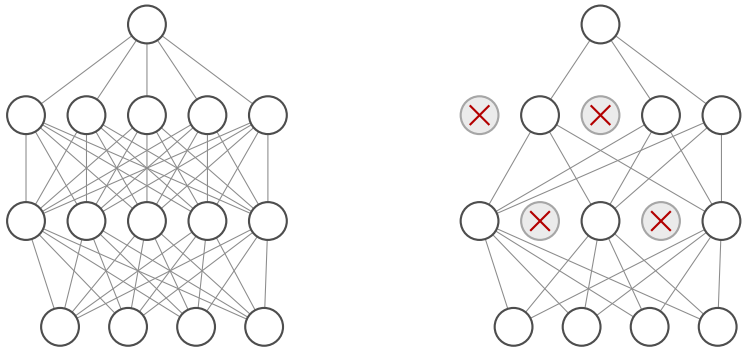
Model	Prediction method	Test accuracy
Baseline (10 epochs)	single model	0.837
True ensemble of 10 models	average predictions	0.855
True ensemble of 10 models	voting	0.851
Snapshots (25) over 10 epochs	average predictions	0.865
Snapshots (25) over 10 epochs	voting	0.861
Snapshots (25) over 10 epochs	parameter averaging	0.864

7.6 Dropout

Bigger ensembles usually work better but are expensive. Can we make many models out of a single network?

Definition 7.3 [Dropout]. Dropout randomly sets some activations to zero in the forward pass: for each $a_j^{(i)}$, set it to $a_j^{(i)} m_{ij}$ with $m_{ij} \sim \text{Bernoulli}(0.5)$, that is, 1 with probability 50% and 0 otherwise, drawing a new mask for every forward pass.

Each dropout mask defines a different "new" network made out of the old one (Figure 14), so training with dropout looks like training a huge ensemble, one model per mask: 2^n of them for n droppable units. Dropout also forces a redundant representation: a "cat score" cannot rely on any single feature such as "has an ear" or "is furry," since any of them may be dropped.



standard network after applying dropout

Figure 14: Dropout samples a thinned network by removing hidden units (crossed) together with their connections.

At test time we want to combine all the models. We could generate many dropout masks and average, but simpler is to stop dropping out. During training, on average half of the dimensions are forced to 0; at test time none are, so $W^{(i)}a^{(i)}$ would be about twice as big. The solution is to divide the weights by 2, $\bar{W}^{(i)} = \frac{1}{2}W^{(i)}$. Equivalently ("inverted dropout"), scale the kept activations by $1/p$ during training, with keep probability p, so that test time is unchanged:

```
p = 0.5  # probability of keeping a unit active
H1 = np.maximum(0, np.dot(W1, X) + b1)
U1 = (np.random.rand(*H1.shape) < p) / p  # dropout mask, scaled by 1/p
H1 *= U1                                  # drop
```

7.7 Hyperparameters

All these tricks add many hyperparameters. Some affect *optimization* (training): the learning rate, momentum, initialization, and batch normalization. Others affect *generalization* (validation): ensembling, dropout, and the architecture (number and size of layers). To pick them:

- recognize which is which, since this really matters: a bad learning rate, momentum, or initialization shows up very early in training, while the effect of the architecture is usually apparent only after training is done;
- go coarse to fine: start with a broad sweep, then zero in, for example a short (5-epoch) sweep of learning rate and weight decay, sampled in log space (`lr = 10**uniform(-3, -6)`), then narrow the range around the best results;
- consider random search instead of grid search: when only some hyperparameters matter, random points try many more distinct values of the important ones (Figure 15).

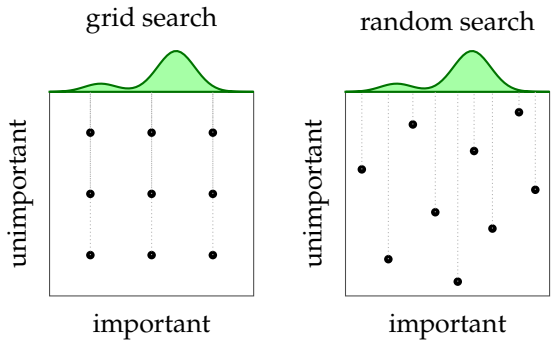


Figure 15: With nine trials, a grid tries only three values of the important hyperparameter, random search nine. The green curve is the validation score. Schematic.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 7 (Spring 2021).

Lecture 8: Computer Vision

So far, convolutional networks map an image to a single output value, such as a semantic category ("bicycle"). The standard computer vision problems go further:

- *object classification*: one label per image;
- *object localization*: a label and a bounding box for the object;
- *object detection*: a label and a box for each of several objects;
- *semantic segmentation* (also called scene understanding): a label for every pixel.

8.1 Object localization

Before, the data was $\mathcal{D} = \{(x_i, y_i)\}$ with x_i an image and y_i a categorical class label. For localization, $y_i = (\ell_i, x_i, y_i, w_i, h_i)$: a class label ℓ_i and a bounding box with position (x_i, y_i), width w_i, and height h_i.

A model predicts a box (x, y, w, h) and a prediction score, for example "cat": 0.64. To decide whether it got it right, the standard measure is the INTERSECTION OVER UNION (Figure 16),

$$\text{IoU} = \frac{I}{U} = \frac{\text{area of intersection of the predicted and true boxes}}{\text{area of their union}}.$$

Datasets have different protocols, but one reasonable one counts a prediction correct if IoU > 0.5, and, when a class label is also output (usually the case), if in addition the class is correct. This is an evaluation standard, not a loss function.

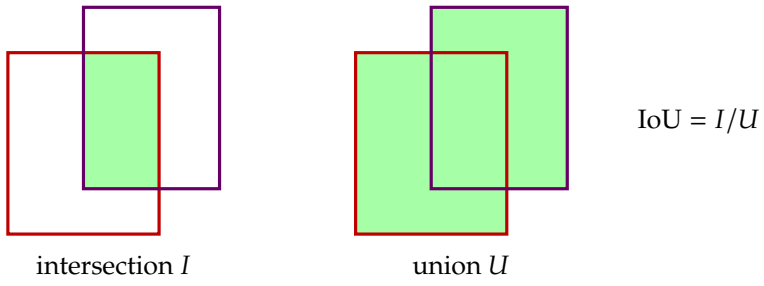


Figure 16: Intersection and union of a true box (red) and a predicted box (purple).

The simplest design treats localization as *regression*: a convolutional network with two heads, one producing the class label (cross-entropy loss) and one producing (x, y, w, h) (a regression loss, such as a Gaussian log-likelihood or MSE). It can be trained jointly (multi-task) or by training

classification first and then the regression head. It is very simple and more or less works, but by itself it is not how localization is usually done.

8.2 Sliding windows and OverFeat

Instead, classify every patch of the image: crop many windows, run the classifier on each to get a class probability ("cat": 0.21 for a patch of grass, 0.85 for the face), and take the box with the highest class probability, or more generally apply non-maximal suppression (below).

A practical approach is OverFeat (Sermanet et al., 2013). Each window also gets a regression head that predicts (x, y, w, h), providing a little "correction" to the sliding window.

- Pretrain on just classification.
- Train a regression head on top of the classification features.
- Pass over different regions at different scales.
- "Average" together the boxes to get a single answer.

Sliding windows are very expensive: 36 windows cost 36 times the compute. But this looks a lot like convolution, and the calculations can be reused across windows. In *convolutional classification*, the fully connected layers at the end are just convolutional layers: instead of two fully connected layers with 4096 units, use two convolutional layers with $1 \times 1 \times 4096$ filters. A network built for 14×14 windows (5×5 convolution to 10×10, 2×2 pooling to 5×5, a 5×5 convolution to 1×1, then two 1×1 convolutions) applied to a 16×16 image produces 12×12, 6×6, and finally a 2×2 output: the predictions of four overlapping windows at once, with all shared computation done once. This kind of calculation reuse is extremely powerful for localization with convolutional networks, and variants of it appear in every method below.

8.3 Object detection

Now the number of objects n_i is different for each image x_i, so a single (x, y, w, h) output no longer suffices. Options:

- *Sliding window*: each window can be a different object. Instead of selecting the window with the highest probability (or merging windows), output an object in each window above some threshold. The big problem is that a high-scoring window probably has other high-scoring windows nearby. Non-maximal suppression (informally) kills off any detection that has other higher-scoring detections of the same class nearby.
- *Actually output multiple things*: the output is a list of bounding boxes, such as {"cat" : 0.21, "dog" : 0.54} with $(x_1, y_1, w_1, h_1, x_2, y_2, w_2, h_2)$. The obvious problem is that the number must be fixed in advance and is usually small.

Neither is good by itself, but they work great combined.

YOLO ("you only look once," Redmon et al., 2015) actually looks 49 times: it produces a different output for each cell of a 7×7 grid, a bit like a sliding window. For each cell it outputs B boxes, each with (x, y, w, h) and a confidence (the IoU, zero if there is no object), plus a class label ℓ. During training, one output must be made "responsible" for each true object: use the best-fit output in that cell (the one with the highest IoU). Using the same trick as before to reuse computation, the cost is not 49 times higher. If there are too many objects, nothing special happens; we just miss them.

CNNs with region proposals. A smarter "sliding window" uses REGION OF INTEREST (RoI) PROPOSALS. R-CNN takes an input image, extracts about 2000 region proposals from a separate proposal method, warps each region and computes CNN features, and classifies each region. This is really slow, but we already know the fix: Fast R-CNN (Girshick, 2015) forwards the whole image through the convolutional network once to get high-resolution features ($C \times H \times W$), then for each proposal max-pools the features within each cell of a grid over the proposed region (*RoI pooling*) to low-resolution features ($C \times h \times w$), which fully connected layers turn into a softmax classifier and bounding-box regressors. This is the same idea as evaluating every location convolutionally. Faster R-CNN (Ren et al., 2015) also learns the proposals: a *region proposal network* slides a small window over the convolutional feature map and, with a design very similar to OverFeat or YOLO, classifies whether *any* object is present around each location and regresses the box locations; the proposals are then RoI pooled for the final classifier.

Follow-ups to YOLO (such as YOLO v5) work better, and SSD (Liu et al., 2015) directly "classifies" locations with a class and a bounding box shape.

8.4 Semantic segmentation

Segmentation labels every single pixel with its class. In some sense this is simpler than detection: there is no longer a variable number of outputs, and every pixel has a label. Instances are ignored for now (two adjacent cows are just one "cow blob," which is acceptable for some purposes). The simple solution is a per-pixel classifier, but we want the output to have the same resolution as the input. That is not hard if we never downsample (zero padding, stride 1, no pooling): a stack of convolutions mapping the $3 \times H \times W$ input to $D \times H \times W$ features, then to $C \times H \times W$ class scores, and an argmax to $H \times W$ predictions. But that is very expensive. The challenge is to design an architecture that makes per-pixel classification computationally tractable.

FULLY CONVOLUTIONAL NETWORKS downsample and then upsample (Figure 17): low-resolution but high-depth processing in the middle integrates context from the entire image, and upsampling at the end turns these low-resolution feature vectors into high-resolution per-pixel predictions. Two operations increase resolution.

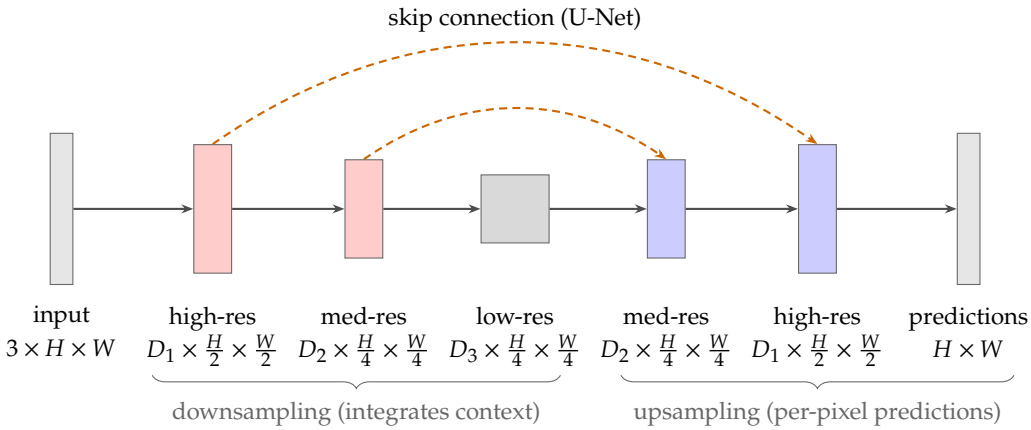


Figure 17: A fully convolutional (bottleneck) segmentation network. U-Net adds connections (dashed) between corresponding downsampling and upsampling layers.

- TRANSPOSE CONVOLUTION. A normal convolution with stride 2 reduces resolution: an $H_f \times W_f \times C_{\text{in}}$ input patch maps to a $1 \times 1 \times C_{\text{out}}$ output, with filter $H_f \times W_f \times C_{\text{in}} \times C_{\text{out}}$. A transpose convolution increases resolution with a fractional "stride" of $\frac{1}{2}$: each $1 \times 1 \times C_{\text{in}}$ input produces an $H_f \times W_f \times C_{\text{out}}$ output patch, with filter $C_{\text{in}} \times H_f \times W_f \times C_{\text{out}}$. Where the output patches of neighboring inputs overlap there are two sets of values, and we just average them.
- MAX UNPOOLING. Max pooling remembers which element was the max; the corresponding unpooling layer places each input value at those remembered positions and fills the rest with zeros (Figure 18). It pairs corresponding downsampling and upsampling layers.

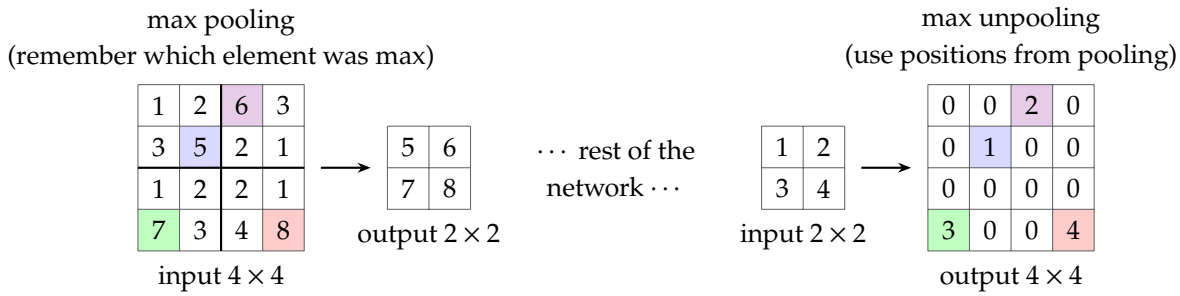


Figure 18: Max pooling records the positions of the maxima (colored), and max unpooling later uses them.

The bottleneck architecture of Long et al. (2014) also fuses predictions from several resolutions. U-NET (Ronneberger et al., 2015) concatenates activations from the convolutional layers on the downsampling path to the corresponding upsampling layers ("copy and crop"), using 3×3 convolutions with ReLU, 2×2 max pooling, 2×2 up-convolutions, and a final 1×1 convolution to the output segmentation map.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 8 (Spring 2021).

Lecture 9: Generating Images from CNNs

9.1 What do convolutional networks "see"?

Neuroscientists probed what the brain sees by recording from the visual cortex of a cat while showing it light bars at different orientations; individual neurons fire for particular orientations. Learned first-layer filters of convolutional networks look similar, but what do the deeper layers respond to? Knowing this helps us interpret what networks pay attention to, understand when they will work and when they will not, compare models and architectures, and manipulate images based on network responses.

There are two options: visualize the *filter* itself, or visualize the *stimuli* that activate a "neuron." Visualizing filters works for the first layer, whose filters act directly on pixels, but the filters of higher layers cannot be visualized in a way that makes much sense.

9.2 Visualizing neuron responses

Idea 1: look for images that maximally excite specific units. Run many image patches through the network and, for a chosen unit, keep the patches with the largest activations. For units in the fifth pooling layer of a detection network (Girshick et al.), the top regions reveal units aligned to concepts such as people or text, while others capture textures and materials such as dot arrays or specular reflections.

Idea 2: see which pixels maximally influence the value at some unit. "Influence" means: given a pixel x_{ij} and a unit a^ℓ_{mnp}, how much does changing x_{ij} change a^ℓ_{mnp}? The answer is the derivative da^ℓ_{mnp}/dx_{ij}, which backpropagation computes. Run the forward pass to calculate each $a^{(i)}$ and $z^{(i)}$, then backpropagate from layer ℓ instead of from the loss:

1. set δ to be the same size as a^ℓ;
2. set $\delta_{mnp} = 1$ and all other entries to 0;
3. backpropagate from layer ℓ to the image, using $\delta \leftarrow \frac{df}{dx_f}\delta$ at each layer;
4. the last δ is $\frac{da^\ell_{mnp}}{dx}$.

Plain backpropagation is not very interpretable, because many units in the network contribute positive or negative gradients. Guided backpropagation (Springenberg et al., 2014) keeps only the positive gradients, avoiding some of the complicated negative contributions and giving a cleaner signal: it heuristically changes the backward step of each ReLU so that every entry is set to $\max(0, \delta_{ijk})$, "zeroing out negative gradients at each layer." For a ReLU layer $f^{\ell+1} = \max(f^\ell, 0)$

with incoming signal $R^{\ell+1}$, the three backward rules are

$$\underbrace{R^\ell = \mathbf{1}(f^\ell > 0)\, R^{\ell+1}}_{\text{backpropagation}}, \qquad \underbrace{R^\ell = \mathbf{1}(R^{\ell+1} > 0)\, R^{\ell+1}}_{\text{“deconvnet”}}, \qquad \underbrace{R^\ell = \mathbf{1}(f^\ell > 0)\, \mathbf{1}(R^{\ell+1} > 0)\, R^{\ell+1}}_{\text{guided backpropagation}},$$

all elementwise. The deconvnet and guided rules do not compute a true gradient but an imputed version. On a cat image, the raw gradient is noisy while guided backpropagation highlights the eyes. For units in deeper layers (CONV6, CONV9) the guided-backpropagation patterns match the corresponding image crops.

9.3 Optimizing the image

Idea 3: optimize the image to maximally activate a particular unit. Before, we just computed da^ℓ_{mnp}/dx_{ij}. Now we compute $g = da^\ell_{mnp}/dx$ and update $x \leftarrow x + \alpha g$, repeatedly. This is gradient ascent on

$$x \leftarrow \arg\max_x a^\ell_{mnp}(x), \qquad \text{more generally} \qquad x \leftarrow \arg\max_x S(x),$$

where $S(x)$ is some activation or class label. The problem is that it is too easy to produce “crazy” images: if making the eyes more blue increases the activation, why not set the blue channel to 10^9? So add an IMAGE REGULARIZER to prevent crazy images,

$$x \leftarrow \arg\max_x S(x) + R(x), \qquad \text{for example} \quad R(x) = -\lambda \|x\|_2^2.$$

To visualize classes (Simonyan et al., 2013), maximize the class label. An important detail: maximize not the class probability but the activation right before the softmax: with $p(y \mid x) = \text{softmax}(W^\ell a^\ell(x) + b^\ell)$, take $S(x)$ to be an entry of $W^\ell a^\ell(x) + b^\ell$. The resulting images show recognizable fragments of dumbbells, dalmatians, geese, and so on.

A slightly more nuanced regularizer (Yosinski et al., 2015) gives cleaner images:

1. update the image with the gradient;
2. blur the image a little;
3. zero out any pixel with a small value;
4. repeat.

This produces much more natural-looking images for classes such as flamingo or pelican, and also for individual units in each layer: simple textures in early layers and object parts or whole objects (pirate ships, teddy bears) in layers 7 and 8.

9.4 DeepDream

Before, we asked what a unit, say channel 17 in layer conv5, is looking at. Now we ask how an image would change: what would it look like if it were a Monet painting? The questions are related: Monet paintings have particular feature distributions, and we can "transport" these distributions to other images.

Looking at clouds, a network might think a cloud looks kind of like a dog. Asking "where is the dog?" and answering "let me show you" means exaggerating whatever features are already present, just as making a cat's eyes more blue makes the image more cat-like.

Algorithm 9.1: DeepDream

Input: image x, layer ℓ

for $t = 1, 2, \ldots$ **do**
- run the forward pass to compute the activations a^ℓ at layer ℓ
- set $\delta \leftarrow a^\ell$ *// instead of a one-hot δ*
- backpropagate δ to the image and apply the gradient to x

Setting $\delta = a^\ell$ is gradient ascent on $S(x) = \frac{1}{2}\|a^\ell(x)\|^2$, which amplifies every feature the layer already detects. The reference implementation also uses a *jitter* regularizer, randomly shifting the image before each step and shifting it back afterwards, normalizes the step by the mean absolute gradient, and clips the pixels to the valid range. Applied at a middle layer of an Inception network (`inception_4c/output`), clouds sprout animal faces and fantastical buildings; this is again $x \leftarrow \arg\max_x S(x) + R(x)$.

9.5 Style transfer

Instead of exaggerating the features of a single image, make the features of one image look more like those of another (Gatys et al., 2015). The *style* of an image is the relationship between its features, for example lots of angles and straight lines; its *content* is the spatial positions of features, such as where curves, edges, and corners are located. Combining the style of a painting with the content of a photo really works.

Style asks which features tend to co-occur: "each time we see a straight line, we also see this texture," "each time we see a curve, we also see flat shading." With feature maps f_k at layer ℓ, the feature covariance is

$$\mathrm{Cov}_{km} = E[f_k f_m],$$

estimated by averaging over all positions in the image, and it forms the Gram matrix $G_{km} = \mathrm{Cov}_{km}$. Surprisingly, if the features have this covariance, the image has the right style. Let G^ℓ be the Gram matrix of the style source image at layer ℓ and $A^\ell(x)$ that of the new image. Then

$$\mathcal{L}_{\text{style}}(x) = \sum_\ell \sum_{k,m} \left(G^\ell_{km} - A^\ell_{km}(x)\right)^2 w_\ell,$$

with a different weight w_ℓ on each layer to prioritize the contribution of different levels of abstraction to the desired style. Content is matched directly, at one chosen layer ℓ:

$$\mathcal{L}_{\text{content}}(x) = \sum_{i,j} \sum_{k} \left(f^{\ell}_{ijk}(x_{\text{content}}) - f^{\ell}_{ijk}(x)\right)^2.$$

The new image is

$$x \leftarrow \arg\min_{x} \mathcal{L}_{\text{style}}(x) + \mathcal{L}_{\text{content}}(x),$$

optimized by backpropagating to the pixels as before.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 9 (Spring 2021).

Lecture 10: Recurrent Neural Networks

10.1 Variable-size inputs

Before, each input x_i had a fixed size. Now inputs are sequences of different lengths, for example $x_1 = (x_{1,1}, x_{1,2}, x_{1,3}, x_{1,4})$, $x_2 = (x_{2,1}, x_{2,2}, x_{2,3})$, and $x_3 = (x_{3,1}, \ldots, x_{3,5})$: classifying the sentiment of a phrase (a sequence of words), recognizing a phoneme from sound (a sequence of sounds), or classifying the activity in a video (a sequence of images).

The simple idea is to zero-pad up to the length of the longest sequence, $(x_{i,1}, x_{i,2}, x_{i,3}, 0, 0, 0)$. It is very simple and can work in a pinch, but it does not scale well to very long sequences.

Another idea is to give the network one input per layer. Each layer concatenates the previous activation with the current input element,

$$\bar{a}^{\ell-1} = \begin{bmatrix} a^{\ell-1} \\ x_{i,t} \end{bmatrix}, \qquad z^\ell = W^\ell \bar{a}^{\ell-1} + b^\ell, \qquad a^\ell = \sigma(z^\ell),$$

and the missing layers of shorter sequences are handled by starting later with $a^\ell = 0$: anything preceding the first input does not matter. This is more efficient than zero-padding up to the maximum length, each layer is much smaller than the giant first layer needed to take the whole sequence at once, and shorter sequences evaluate fewer layers. But the total number of weight matrices grows with the maximum sequence length.

Definition 10.1 [Recurrent neural network]. A recurrent neural network (RNN) shares the weights of these layers, $W^{\ell_i} = W^{\ell_j}$ and $b^{\ell_i} = b^{\ell_j}$ for all i, j, so it can have as many "layers" as we want.

An RNN could also be called a "variable-depth" network. It is often described as a network that "extends a standard neural network along the time dimension," and drawn as a single layer with a loop that feeds back its own previous value. This is technically true but somewhat unhelpful, and makes RNNs seem more mystical than they are: RNNs are just neural networks that share weights across multiple layers, take an input at each layer, and have a variable number of layers.

10.2 Training by backpropagation

The forward pass computes each $a^{(i)}$ and $z^{(i)}$ as usual. In the backward pass, taken literally, the update $\frac{d\mathcal{L}}{d\theta_f} \leftarrow \frac{df}{d\theta_f}\delta$ at layer $\ell - 1$ would overwrite the gradient computed at layer ℓ, since W, b are

the same at all layers. Instead, *accumulate* the gradient during the backward pass:

$$\frac{d\mathcal{L}}{d\theta_f} \mathrel{+}= \frac{df}{d\theta_f}\delta, \qquad \delta \leftarrow \frac{df}{dx_f}\delta.$$

(Most libraries do this automatically.) To see why, take $f(x) = g(x, h(x))$, which resembles the role of W in an RNN; by the chain rule

$$\frac{d}{dx}f(x) = \frac{dg}{dx} + \frac{dh}{dx}\frac{dg}{dh},$$

the derivative through the first argument plus the derivative through the second.

10.3 Variable-size outputs

Outputs can also be sequences: generating a text caption for an image, predicting a sequence of future video frames, or generating audio. Now there is an output at every layer:

$$z^\ell = W^\ell a^{\ell-1} + b^\ell, \qquad a^\ell = \sigma(z^\ell), \qquad \hat{y}_\ell = f(a^\ell),$$

where the readout function or "decoder" f can be as simple as a linear layer followed by a softmax. Each output has its own loss, for example a cross-entropy, and the total is

$$\mathcal{L}(\hat{y}_{1:T}) = \sum_\ell \mathcal{L}_\ell(\hat{y}_\ell).$$

In the computation graph, each σ_ℓ now feeds both the next linear layer and its own readout f_ℓ and loss $\mathcal{L}_\ell$, so it is not completely obvious how to backpropagate. Graph-structured backpropagation, also called reverse-mode automatic differentiation, handles this with one rule: process each function $f(x_f) = y_f$ starting from the last, where $\delta = 1$, and for each node with multiple descendants in the graph, simply add up the delta vectors coming from all descendants,

$$\delta_{y_f} = \delta^1_{y_f} + \delta^2_{y_f}, \qquad \delta_{x_f} = \frac{df}{dx_f}\delta_{y_f}, \qquad \frac{d\mathcal{L}}{d\theta_f} = \frac{df}{d\theta_f}\delta_{y_f}.$$

With both an input and an output at each step (Figure 19), the equations are as before with $\bar{a}^{\ell-1} = (a^{\ell-1}, x_{i,t})$. Examples are generating a caption for an image (it is a bit subtle why there are inputs at each step; see below) and translating text into another language (though there are much better ways to do it, Lecture 11).

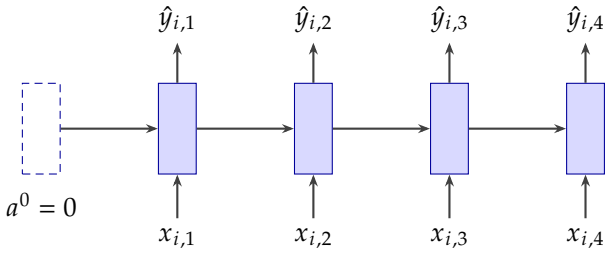


Figure 19: An RNN with an input and an output at each step, all sharing the same weights.

10.4 Vanishing and exploding gradients

RNNs are extremely deep networks: a sequence of length 1000 or more is like backpropagating through 1000 or more layers, and $\frac{d\mathcal{L}}{dW^{(1)}} = J_1 J_2 \cdots J_n \frac{d\mathcal{L}}{dz^{(n)}}$ goes to 0 if the factors are mostly below 1 and to infinity if they are mostly above 1. Exploding gradients can be fixed with gradient clipping. Vanishing gradients are a big problem: the gradient signal from later steps never reaches the earlier steps, which prevents the RNN from "remembering" things from the beginning.

As before, we want gradients close to 1, but which gradients? Each layer computes $\bar{a}_{t-1} = (a_{t-1}, x_t)$, $z_t = W\bar{a}_{t-1} + b$, $a_t = \sigma(z_t)$, which together define the *RNN dynamics* $a_t = q(a_{t-1}, x_t)$. The best gradient flow has dynamics Jacobian $\frac{dq}{da_{t-1}} \approx I$. That is not always good: it is good only when we want to *remember*, and sometimes we want to *forget*. So we want $\frac{dq_i}{da_{t-1,i}} \approx 1$ exactly when we choose to remember $a_{t-1,i}$.

The intuition is that each unit gets a little "neural circuit" that decides whether to remember or overwrite. If remembering, it copies the previous activation as it is; if forgetting, it overwrites it with something based on the current input. With a forget gate $f_t \in [0, 1]$,

$$a_t = a_{t-1} f_t + g_t, \qquad \frac{dq_i}{da_{t-1,i}} = f_t \in [0, 1].$$

The vector a_t is called the cell state.

10.5 LSTM cells

Definition 10.2 [LSTM]. A long short-term memory (LSTM) cell keeps a cell state a_t and an output h_t. One linear map of the previous output and the input produces four vectors, each with the same dimension as h_{t-1} (so the output is 4 times larger than in a plain RNN cell):

$$W \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b = \begin{bmatrix} \bar{f}_t \\ \bar{i}_t \\ \bar{g}_t \\ \bar{o}_t \end{bmatrix}, \qquad f_t = \sigma(\bar{f}_t), \quad i_t = \sigma(\bar{i}_t), \quad g_t = \tanh(\bar{g}_t), \quad o_t = \sigma(\bar{o}_t),$$

so $f_t, i_t, o_t \in [0,1]$ and $g_t \in [-1,1]$. Then, elementwise,

$$a_t = a_{t-1} f_t + i_t g_t, \qquad h_t = o_t \tanh(a_t).$$

The output h_t goes to the next step and to the decoder that produces $\hat{y}_t$.

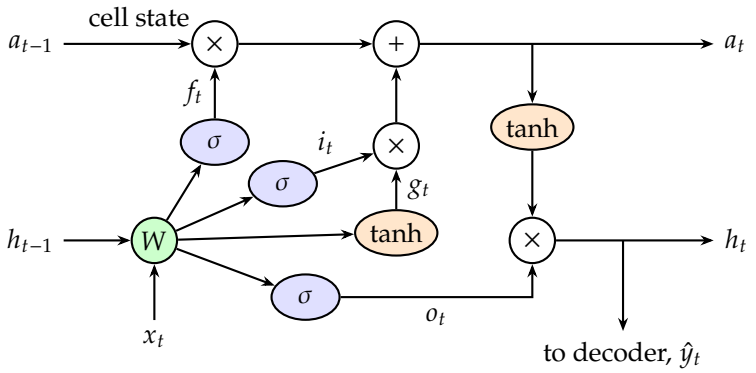


Figure 20: An LSTM cell: forget gate f_t, input gate i_t, candidate g_t, and output gate o_t.

Is this all a little arbitrary? Yes, but it works quite well in practice, much better than a naive RNN. LSTMs train better because the cell state $a_t = a_{t-1} f_t + i_t g_t$ changes very little from step to step, a "long-term" memory, while h_t changes all the time (multiplicatively), a "short-term" memory.

Some practical notes.

- In practice RNNs almost always have both an input and an output at each step (see below).
- Naive RNNs almost never work in practice. LSTM units work fine in many cases and dramatically improve over naive RNNs, but still require much more hyperparameter tuning than fully connected or convolutional networks.
- Some alternatives can work better for sequences: temporal convolutions and transformers (temporal attention).
- LSTM cells are annoyingly complicated, but once implemented they can be used like any other layer.
- Some simpler variants work just as well, such as the GATED RECURRENT UNIT (GRU).

10.6 Autoregressive models and structured prediction

Most RNNs used in practice have an input and an output at every step, because most problems that require multiple outputs have strong dependencies between the outputs. This is called STRUCTURED PREDICTION.

Example: *Text generation.* Train on "I think therefore I am," "I like machine learning," and "I am not just a neural network." After "I" the first output might be think 0.3, like 0.3, am 0.4; the second therefore 0.3, machine 0.3, not 0.4; the third I 0.3, learning 0.3, just 0.4. If the outputs are sampled independently, we may get "I think machine just," nonsense, even though the network had exactly the right probabilities. The key idea is that past outputs should influence future outputs: feeding the sampled word "think" back as the next input makes the next distribution therefore 0.8, machine 0.1, not 0.1, and then I 0.8.

Such AUTOREGRESSIVE models are trained, in the basic version, by setting the inputs to entire training sequences and the ground-truth outputs to the same sequences offset by one step:

$$x_{1:5} = (\text{“I”}, \text{“think”}, \text{“therefore”}, \text{“I”}, \text{“am”}),$$
$$y_{1:5} = (\text{“think”}, \text{“therefore”}, \text{“I”}, \text{“am”}, \text{stop_token}).$$

This teaches the network to output "am" after seeing "I think therefore I."

Distributional shift. Suppose that after "I" the network outputs think 0.6, like 0.3, drive 0.1, and we sample the unlikely but possible "drive." We got unlucky, and now the model is completely confused: it never saw "I drive" before, so its next prediction (hippo 0.6, paintbrush 0.3, California 0.1) is complete nonsense. The problem is a training/test discrepancy: during training the network always saw true sequences as inputs, but at test time it gets its own, potentially incorrect, predictions. This is called DISTRIBUTIONAL SHIFT, because the input distribution shifts from true strings (at training) to synthetic strings (at test time). Even one random mistake can completely scramble the output.

SCHEDULED SAMPLING (Bengio et al., 2015), an old trick from reinforcement learning adapted to RNNs, randomly decides during training whether to give the network the ground-truth token or its own previous prediction as input. The probability of using the ground truth follows a schedule (for example exponential, inverse sigmoid, or linear decay over training): at the beginning, mostly feed ground-truth tokens, since the model's predictions are mostly nonsense; at the end, mostly feed the model's own predictions, to mitigate distributional shift.

10.7 Ways to use RNNs

RNNs can be one to one, one to many (for example image captioning), many to one (activity recognition), many to many with the outputs after the inputs (machine translation), or many to many with an output at each input (frame-level video annotation); in reality we almost always use autoregressive generation. Further variants:

- the input can be processed by a (non-recurrent) *encoder* before entering the RNN, very common with image inputs, and the output by a (non-recurrent) *decoder*, which is less common since we could just use more RNN layers;
- it is easy to stack as many RNN layers as necessary;

- we can even make recurrent (LSTM) convolutional layers: replace the matrix multiplication $W[h_{t-1}; x_t] + b$ by a convolution, so that each "filter" becomes a little LSTM cell;
- *bidirectional* models run one RNN forward and another backward over the sequence and combine them. In speech recognition the word at a time step can be hard to guess without the rest of the utterance (we cannot tell whether a word is finished until we hear its ending). This is an even bigger problem in machine translation, which uses slightly different models.

Character-level RNNs trained on Shakespeare produce gibberish at first and, with more training, increasingly plausible text: plays with speaker names and verse, and, trained on the LaTeX source of an algebraic geometry textbook, fake lemmas and proofs. GPT-2 generates even more coherent text on the same autoregressive principle, with a different type of model, the transformer (Lecture 12).

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 10 (Spring 2021).

Lecture 11: Sequence-to-Sequence Models

11.1 Neural language models

A basic neural language model is an RNN trained on natural sentences, such as "I think therefore I am," "I like machine learning," and "I am not just a neural network" (in reality, several million of them), to predict the next word at each step. Sentences are *tokenized*, each word being a token. The simplest representation of a token is a one-hot vector, whose dimension is the number of possible words and whose single 1 is at the index of the word; more complex are word embeddings (Lecture 13).

Two special tokens complete the setup. The model is trained to output an <EOS> token when the sequence ends. To complete a sequence (give the model "I think. . . " and have it finish), feed in the given words and then sample; to generate an entirely new sequence, start with a special <START> token.

11.2 Conditional language models and sequence-to-sequence models

How do we tell the RNN *what* to generate? A conditional language model sets the initial state to a vector encoding of the desired content, $a_0 = f(x)$, instead of $a_0 = 0$. For image captioning, a CNN encoder computes $f(x)$ from the image and an RNN decoder generates "A cute puppy <EOS>" from <START>; the training data are images paired with captions.

We can also condition on *another sequence*: an RNN encoder reads "Un chiot mignon" and its final state initializes an RNN decoder that writes "A cute puppy" (Figure 21). In reality the input sequence is often read in reverse, which puts the first source words closest to the first target words; and the decoder's <START> token doubles as the <EOS> token of the input, so the encoder needs no <START> token.

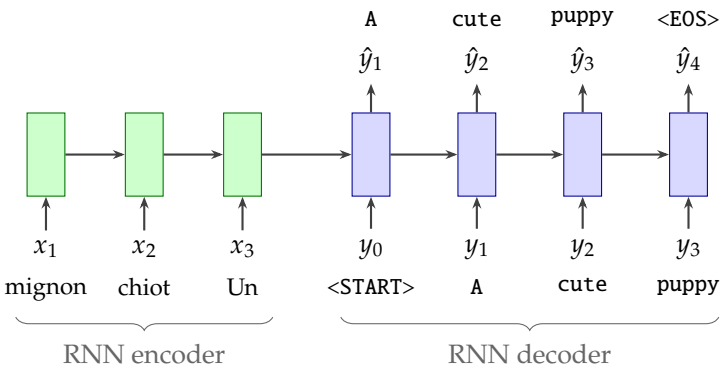


Figure 21: A sequence-to-sequence model with the source read in reverse.

Definition 11.1 [Sequence-to-sequence model]. A sequence-to-sequence (seq2seq) model consists of an encoder RNN and a decoder RNN, typically two separate RNNs with different weights, trained end to end on paired data (for example pairs of French and English sentences).

A more realistic model has multiple RNN layers, each using LSTM (or GRU) cells, and is trained end to end on pairs of sequences that can have different lengths (Sutskever, Vinyals, and Le, 2014). Besides translating one language into another, seq2seq models can summarize a long sentence into a short one, respond to a question with an answer, or perhaps generate Python code from text.

11.3 Decoding the most likely sequence

Each decoder output represents $p(y_t \mid x_{1:T}, y_{0:t-1})$: it depends on the whole input sequence and on the output sequence so far. The probability of an output sequence is

$$p(y_{1:T_y} \mid x_{1:T}) = \prod_{t=1}^{T_y} p(y_t \mid x_{1:T}, y_{0:t-1}).$$

Greedy decoding picks the most likely token at every step and can go wrong. Translating "Un chiot mignon," the first step might give One 0.51, A 0.48; after "One" the next gives Puppy 0.53, Cute 0.47; and after "One puppy" there is no good continuation (Is 0.70, <EOS> 0.25). Choosing "A" instead has slightly lower probability at the first step, but then Cute 0.53 and Puppy 0.95 give a much higher product. To maximize the product of *all* probabilities we should not greedily select the most likely word at the first step.

With M words there are in general M^T sequences of length T, and any of them might be optimal. Decoding is a *search* problem over a tree whose branches are the words at each step. Any tree search algorithm could be used, but exact search is very expensive. Fortunately the structure of the problem makes simple approximate search work very well: although choosing the most likely word at the first step may not be optimal, choosing a very low-probability word is very unlikely to lead to a good result. We cannot be greedy, but we can be *somewhat* greedy. This is not true in general; it is a guess based on what we know about sequence decoding.

11.4 Beam search

Beam search stores the k best sequences so far and updates each of them. The special case $k = 1$ is greedy decoding; k is often around 5 to 10. In practice we sum log probabilities as we go, to avoid underflow:

$$\log p(y_{1:T_y} \mid x_{1:T}) = \sum_{t=1}^{T_y} \log p(y_t \mid x_{1:T}, y_{0:t-1}).$$

Example: *Beam search with $k = 2$* (from CS224n, Christopher Manning). Translate the French "il a

m'entarté" ("he hit me with a pie"), which has no perfectly equivalent English word. Figure 22 shows the cumulative log probabilities, for example $\log p(\text{he} \mid \text{START}) + \log p(\text{hit} \mid \text{START}, \text{he}) = -1.7$. After two words the best are "I was" (−1.6) and "he hit" (−1.7); after three, "he hit me" (−2.5) and "he hit a" (−2.8); after four, "he hit me with" (−3.3) and "he hit a pie" (−3.4).

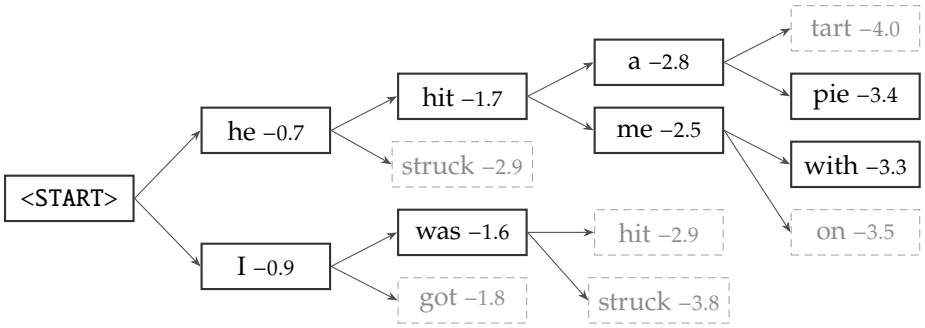


Figure 22: The first steps of beam search with $k = 2$, with cumulative log probabilities. Dashed hypotheses fall out of the beam.

When one of the highest-scoring hypotheses ends in <EOS>, save it with its score but do not pick it for further expansion (there is nothing to expand), and keep expanding the k remaining best hypotheses; continuing the example, "he hit me with a pie <EOS>" is saved. Continue until some cutoff length H or until N hypotheses end in <EOS>.

Which saved sequence do we pick? We might end with "he hit me with a pie" ($\log p = -4.5$), "he threw a pie" (−3.2), and "I was hit with a pie that he threw" (−7.2). Extending a fixed prefix adds a nonpositive log probability, so its unnormalized score cannot increase. This can favor short outputs; it does not mean that every longer sequence scores below every shorter one. A common length correction divides by the sequence length:

$$\text{score}(y_{1:T} \mid x_{1:T}) = \frac{1}{T} \sum_{t=1}^{T} \log p(y_t \mid x_{1:T}, y_{0:t-1}).$$

Algorithm 11.1: Beam search

Input: beam width k, maximum length H
for each time step t **do**
  for each hypothesis $y_{1:t-1,i}$ that we are tracking (there are k) **do**
    find the top k tokens $y_{t,i,1}, \ldots, y_{t,i,k}$ from the softmax log probabilities
  sort the resulting k^2 length-t sequences by their total log probability
  save any sequences that end in <EOS>
  keep the top k, and advance each hypothesis to time $t + 1$ if $t < H$
return the saved sequence with the highest score

11.5 Attention

In a seq2seq model, all information about the source sequence must pass through the encoder's final activations. This forms a BOTTLENECK. What if the decoder could "peek" at the source sentence while decoding?

The idea is to compare a QUERY to a set of KEYS. Every encoder step t produces a key vector $k_t = k(h_t)$, representing what type of information is present at that step, computed by some learned function such as $k_t = \sigma(W_k h_t + b_k)$. Every decoder step l produces a query vector $q_l = q(s_l)$, representing what we are looking for at that step. As a crude intuition, a key might encode "the subject of the sentence" and a query might ask for "the subject of the sentence"; in reality what keys and queries mean is learned, not chosen manually. The ATTENTION SCORE for encoder step t and decoder step l is

$$e_{t,l} = k_t \cdot q_l.$$

Intuitively, we would send h_t for $\arg\max_t e_{t,l}$ to decoder step l, but the argmax is not differentiable. Instead we use a softmax and send a weighted average, which approximates h_t for $\arg\max_t e_{t,l}$:

$$\alpha_{t,l} = \frac{\exp(e_{t,l})}{\sum_{t'} \exp(e_{t',l})}, \qquad a_l = \sum_t \alpha_{t,l} h_t.$$

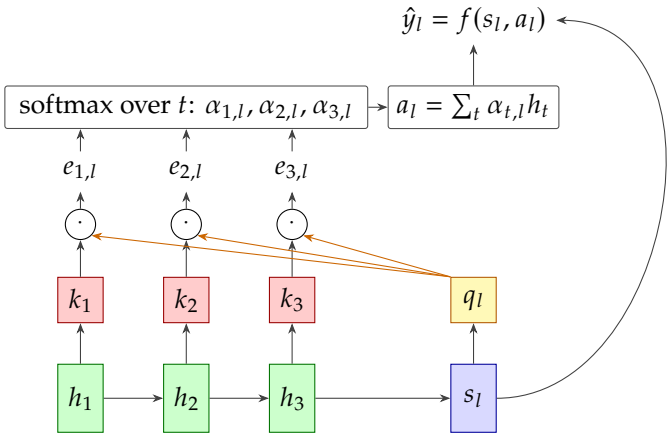


Figure 23: Attention at decoder step l: the query q_l is compared with every key, and the softmax weights combine the encoder activations into a_l.

The attended vector a_l can be used in various ways: for the readout, for example $\hat{y}_l = f(s_l, a_l)$; concatenated as input to the next RNN layer in a multi-layer (stacked) RNN; or concatenated to the hidden state of the next decoder step, $\bar{s}_l = (s_{l-1}, a_{l-1}, x_l)$, which is like appending a to the input. In the running example, the decoder attends mostly to "Un" when producing "A," to "mignon" for "cute," and to "chiot" for "puppy."

Variants.

- The simplest choice takes k and q to be identity functions, $k_t = h_t$ and $q_l = s_l$, so $e_{t,l} = h_t \cdot s_l$.
- *Linear multiplicative attention* uses $k_t = W_k h_t$ and $q_l = W_q s_l$, so $e_{t,l} = h_t^T W_k^T W_q s_l = h_t^T W_e s_l$, and we just learn the matrix W_e.
- A *learned value encoding* averages a learned function of the encoder activations, $a_l = \sum_t \alpha_{t,l} v(h_t)$.

Attention is very powerful because every decoder step is now connected to every encoder step. Connections go from $O(T)$ steps to $O(1)$, so gradients are much better behaved ($O(1)$ propagation length), which becomes very important for very long sequences; the bottleneck matters much less; and it works much better in practice.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 11 (Spring 2021).

Lecture 12: Transformers

12.1 Is attention all we need?

If we have attention, do we even need recurrent connections? Attention can access every time step, so in principle it can do everything that recurrence can, and more. To transform an RNN into a purely attention-based model we must first fix one problem: without recurrence, decoder step $l = 2$ cannot access s_1 or s_0, and the encoder has no temporal dependencies at all.

Definition 12.1 [Self-attention]. In self-attention, every step t of the sequence computes, from $h_t = \sigma(Wx_t + b)$ (or any other non-linear function, with weights shared across time steps), a key, a query, and a value,

$$k_t = k(h_t), \quad q_t = q(h_t), \quad v_t = v(h_t), \qquad \text{for example} \quad k_t = W_k h_t,\ q_t = W_q h_t,\ v_t = W_v h_t,$$

and every step l attends to all steps:

$$e_{l,t} = q_l \cdot k_t, \qquad \alpha_{l,t} = \frac{\exp(e_{l,t})}{\sum_{t'} \exp(e_{l,t'})}, \qquad a_l = \sum_t \alpha_{l,t} v_t.$$

Previously the value was just $v(h_t) = h_t$; now it is, for example, $W_v h_t$. This is not a recurrent model, but it still shares weights. A *self-attention layer* maps $h_1, \ldots, h_T$ to $a_1, \ldots, a_T$, and such layers can be stacked, repeating until the sequence has been processed enough, and at the end decoded into an answer.

The basic concept of self-attention yields a very powerful sequence model, the transformer, but four components are needed to address its fundamental limitations:

1. *positional encoding*, which addresses the lack of sequence information;
2. *multi-headed attention*, which allows querying multiple positions at each layer;
3. *adding nonlinearities*, since so far each successive layer is linear in the previous one;
4. *masked decoding*, which prevents attention lookups into the future.

12.2 Positional encoding

We see "he hit me with a pie," but naive self-attention sees only a bag of words: "a pie hit me with he," "a hit with me he pie," and "he pie me with a hit" all look the same. Most alternative orderings are nonsense, but some change the meaning, and in general the position of words in a

sentence carries information. The idea is to add information to the representation at the beginning that indicates where it is in the sequence, $h_t = f(x_t, t)$ for some function f.

The naive positional encoding just appends t to the input, $\bar{x}_t = (x_t, t)$. This is not a great idea, because absolute position is less important than relative position: in "I walk my dog every day" and "every single day I walk my dog," what matters is that "my dog" comes right after "I walk," not its absolute position. We want tokens with similar relative positions to have similar positional encodings.

Definition 12.2 [Sinusoidal positional encoding]. With dimensionality d for the encoding, the sinusoidal positional encoding $p_t \in \mathbb{R}^d$ has coordinates

$$p_{t,2i-1} = \sin \frac{t}{10000^{2i/d}}, \qquad p_{t,2i} = \cos \frac{t}{10000^{2i/d}}, \qquad i = 1, \ldots, \tfrac{d}{2}.$$

This is a frequency-based representation: the fastest dimensions act as an "even-odd" indicator and the slowest ones as a "first half versus second half" indicator (Figure 24).

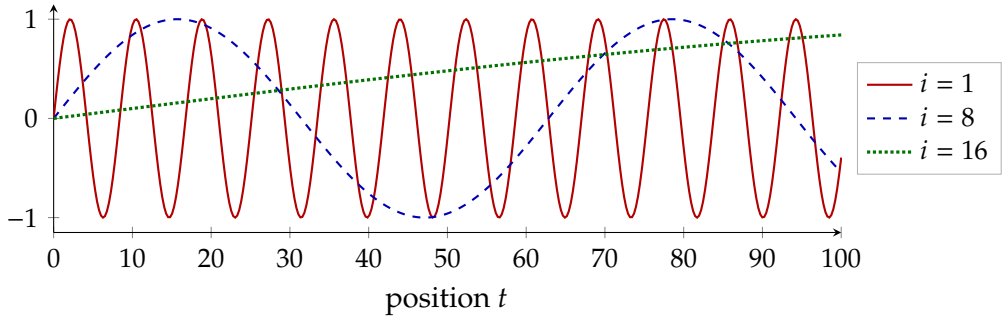


Figure 24: Coordinates $\sin(t/10000^{2i/d})$ of the sinusoidal positional encoding for $d = 64$, plotted from the formula: low i oscillates quickly, high i slowly.

Another idea is to just *learn* a positional encoding: the same learned values p_t for every sequence, but different for different time steps, while the inputs x_t differ for every sequence. That is $P = [p_1, \ldots, p_T] \in \mathbb{R}^{d \times T}$, with d the dimensionality and T the maximum sequence length. It is more flexible (and perhaps more optimal) than the sin/cos encoding, but a bit more complex: we must pick a maximum sequence length and cannot generalize beyond it.

Given x_t and p_t at each step, the simple choice is to concatenate them, $\bar{x}_t = (x_t, p_t)$. More often, we just add them after embedding the input: the input to self-attention is $\text{emb}(x_t) + p_t$, where emb is some learned function (for example some fully connected layers with linear layers and nonlinearities).

12.3 Multi-head attention

Since we now rely entirely on attention, we may want to incorporate more than one time step at once. But $a_l = \sum_t \alpha_{l,t} v_t$ tends to be dominated by one value because of the softmax, and with $e_{l,t} = q_l \cdot k_t$ it is hard to specify that we want two different things, such as the subject and the object of a sentence.

The idea of MULTI-HEAD ATTENTION is to have multiple keys, queries, and values for every time step, $k_{t,i}, q_{t,i}, v_{t,i}$ for heads $i = 1, \ldots, m$, and to compute the weights independently for each head:

$$e_{l,t,i} = q_{l,i} \cdot k_{t,i}, \qquad \alpha_{l,t,i} = \frac{\exp(e_{l,t,i})}{\sum_{t'} \exp(e_{l,t',i})}, \qquad a_{l,i} = \sum_t \alpha_{l,t,i} v_{t,i}.$$

The full attention vector is formed by concatenation, $a_l = (a_{l,1}, \ldots, a_{l,m})$. Around 8 heads seems to work pretty well for big models.

12.4 Adding nonlinearities

Self-attention is linear: with $v_t = W_v h_t$,

$$a_l = \sum_t \alpha_{l,t} v_t = \sum_t \alpha_{l,t} W_v h_t = W_v \sum_t \alpha_{l,t} h_t,$$

a linear transformation of the previous layer, with non-linear weights $\alpha_{l,t}$. This is not very expressive. The fix is to alternate self-attention with a non-linear (learned) function applied at every position after every self-attention layer, for example

$$h_t^\ell = \sigma(W^\ell a_t^\ell + b^\ell),$$

just a neural network applied at each position, sometimes called a POSITION-WISE FEEDFORWARD NETWORK.

12.5 Masked decoding

Consider a crude self-attention "language model" that reads y_1, y_2, y_3 and predicts $\hat{y}_2, \hat{y}_3, \hat{y}_4$ (in reality there would be many alternating self-attention layers and position-wise networks). The big problem is that self-attention at step 1 can look at the values at steps 2 and 3, which are based on the inputs at steps 2 and 3. At test time, when decoding, the inputs at steps 2 and 3 will be based on the output at step 1, which would require knowing the inputs at steps 2 and 3. We must allow self-attention into the past but not into the future. The easy solution is MASKED ATTENTION

(Figure 25):

$$e_{l,t} = \begin{cases} q_l \cdot k_t & \text{if } l \geq t, \\ -\infty & \text{otherwise;} \end{cases}$$

in practice, just replace $\exp(e_{l,t})$ by 0 inside the softmax when $l < t$.

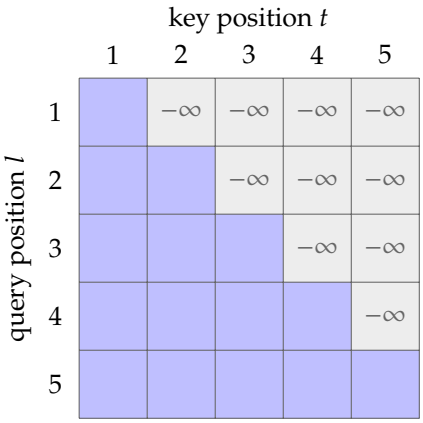


Figure 25: The causal mask: query position l may attend only to key positions $t \leq l$ (blue), where $e_{l,t} = q_l \cdot k_t$.

12.6 The transformer

Models built from successive self-attention and position-wise non-linear layers are generally called TRANSFORMERS, because they transform one sequence into another at each layer. The "classic" transformer (Vaswani et al., *Attention Is All You Need*, 2017) is a sequence-to-sequence model; well-known follow-up works such as BERT and GPT use transformers for language modeling.

Compared with a seq2seq RNN, the classic transformer has an *encoder*, which applies a position-wise encoder to the inputs plus positional encodings p_1, p_2, p_3 and then N repetitions of self-attention and a position-wise non-linear network, and a *decoder*, which does the same for the shifted outputs y_0, y_1, y_2 with *masked* self-attention and an extra CROSS-ATTENTION step, followed by a position-wise softmax that outputs $\hat{y}_1, \ldots, \hat{y}_4$ (Figure 26).

Cross-attention combines encoder and decoder values and is much more like the standard attention of Lecture 11. At decoder layer ℓ, the query comes from the output s_l^ℓ of the decoder's position-wise network at step l, and the keys and values from the encoder outputs h_t:

$$q_l^\ell = W_q^\ell s_l^\ell, \quad k_t^\ell = W_k^\ell h_t, \quad v_t^\ell = W_v^\ell h_t, \qquad e_{l,t}^\ell = q_l^\ell \cdot k_t^\ell, \quad \alpha_{l,t}^\ell = \frac{\exp(e_{l,t}^\ell)}{\sum_{t'} \exp(e_{l,t'}^\ell)}, \quad c_l^\ell = \sum_t \alpha_{l,t}^\ell v_t^\ell.$$

In reality, cross-attention is also multi-headed.

Layer normalization. Batch normalization is very helpful but hard to use with sequence models: sequences have different lengths, which makes normalizing across the batch hard, and they can be very long, so batches are sometimes small. The simple solution is LAYER NORMALIZATION, like batch norm but not across the batch. Batch norm normalizes each of the d dimensions over the B samples $a_1, \dots, a_B$ of the batch,

$$\mu = \frac{1}{B}\sum_{i=1}^{B} a_i, \quad \sigma = \sqrt{\frac{1}{B}\sum_{i=1}^{B}(a_i - \mu)^2}, \quad \bar{a}_i = \frac{a_i - \mu}{\sigma}\gamma + \beta,$$

with d-dimensional μ, σ. Layer norm normalizes a single vector a over its d dimensions, with scalar μ, σ:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} a_j, \quad \sigma = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(a_j - \mu)^2}, \quad \bar{a} = \frac{a - \mu}{\sigma}\gamma + \beta.$$

Putting it all together. The transformer of Vaswani et al. has 6 layers, each with $d = 512$. Each encoder layer takes the input $\bar{h}_t^{\ell-1}$, computes multi-head attention a_t^ℓ (concatenating the attention from all heads), and then

$$\begin{aligned}
\bar{a}_t^\ell &= \text{LayerNorm}\big(\bar{h}_t^{\ell-1} + a_t^\ell\big) && \text{(essentially a residual connection with LN),}\\
h_t^\ell &= W_2^\ell\,\text{ReLU}\big(W_1^\ell \bar{a}_t^\ell + b_1^\ell\big) + b_2^\ell && \text{(a two-layer network at each position),}\\
\bar{h}_t^\ell &= \text{LayerNorm}\big(\bar{a}_t^\ell + h_t^\ell\big) && \text{(passed to the next layer } \ell+1\text{).}
\end{aligned}$$

Each decoder layer is the same as an encoder layer but with masked self-attention, followed by multi-head cross-attention and the position-wise network, each with a residual connection and LN. The decoder decodes one position at a time with masked attention.

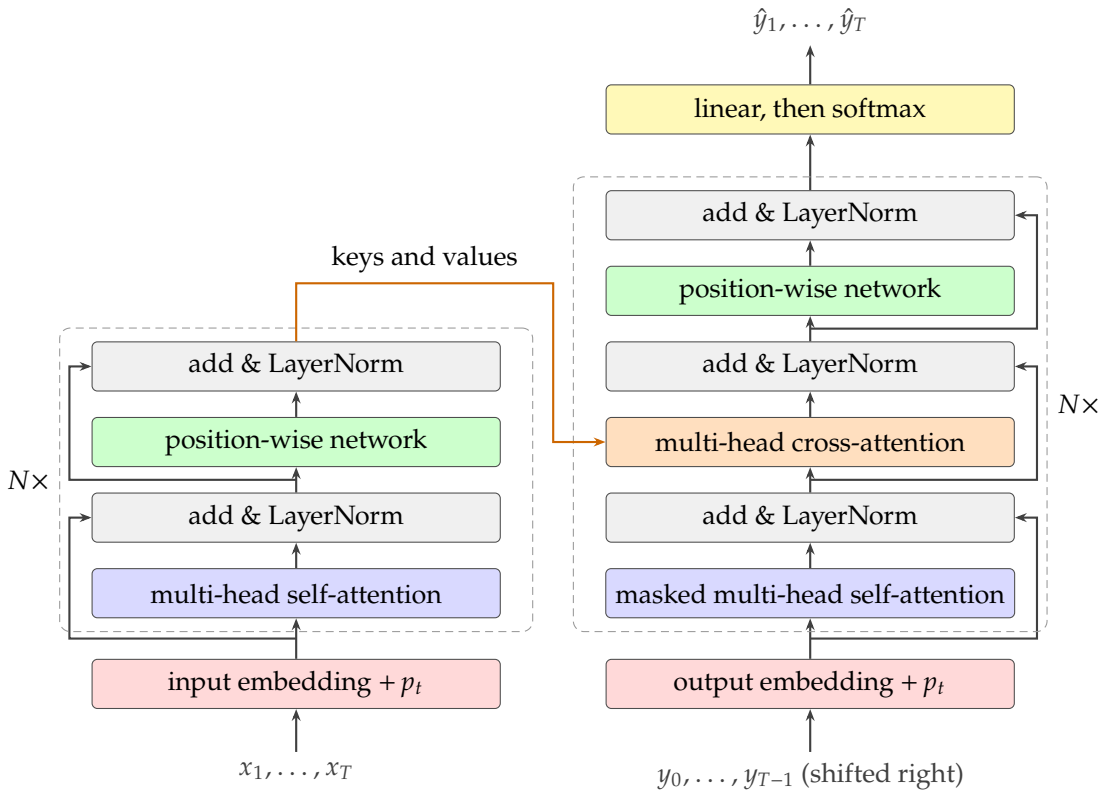


Figure 26: The classic transformer: encoder (left) and decoder (right), each repeated N times, with residual connections around every sub-layer.

12.7 Why transformers?

The downsides are that attention computations are technically $O(n^2)$ in the sequence length, and transformers are somewhat more complex to implement (positional encodings and so on). The benefits are much better long-range connections, much easier parallelization, and, in practice, much greater depth than an RNN. The benefits seem to vastly outweigh the downsides: transformers work much better than RNNs (and LSTMs) in many cases, arguably one of the most important sequence modeling improvements of the past decade.

In practice this means larger models for the same cost, and larger models perform better. For translation, the transformer reaches better BLEU scores at a fraction of the training cost:

	BLEU		Training cost (FLOPs)	
Model	EN-DE	EN-FR	EN-DE	EN-FR
GNMT + RL ensemble	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S ensemble	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

For text summarization (generating Wikipedia articles, Liu et al., 2018), the previous state-of-the-art seq2seq model with attention ($L = 500$) reaches test perplexity 5.04952 (lower is better; the metric is similar to 1/likelihood) and ROUGE-L 12.7, while transformer variants reach perplexity 2.46645 ($L = 500$) down to 1.90325 ($L = 7500$), with ROUGE-L up to 38.8.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 12 (Spring 2021).

Lecture 13: Applications: NLP

13.1 Unsupervised pretraining

Deep learning works best with a lot of data. There is plenty of text (news, books, the web), but most of it is unlabeled: there is thousands of times more valid English text than labeled or paired data such as English/French translations. The big challenge is how to use freely available, unlabeled text to help apply deep learning to NLP. The idea is to learn a representation of language from unlabeled text and use it for tasks such as translation. Such representations range from local, non-contextual ones (*word embeddings*), through *sentence embeddings*, to global, context-dependent ones (*pretrained language models*).

13.2 Word embeddings

A one-hot vector, whose dimension is the number of possible words, means basically nothing by itself. (The pixels of an image at least mean something: not a great metric space, but something.) Learning downstream tasks might be much easier with a more meaningful representation, in which vectors of similar words are close together.

The basic idea is that the meaning of a word is determined by the words that occur in close proximity to it; put another way, the more interchangeable words are, the more similar they are ("Seattle hotel" versus "Seattle motel"). In "... government debt problems turning into *banking* crises..." and "... saying that Europe needs unified *banking* regulation..." the context words represent *banking*. The basic principle is to pick a representation for each word such that its neighbors are "close" under it.

More formally, can we predict the neighbors of a word from its embedding? For a center word c at position t and a context ("outside") word o within a window around it (Figure 27), use learned vectors u_o for o and v_c for c and set

$$p(o \mid c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)},$$

where V is all possible words (the vocabulary). This looks a bit like a logistic regression model.

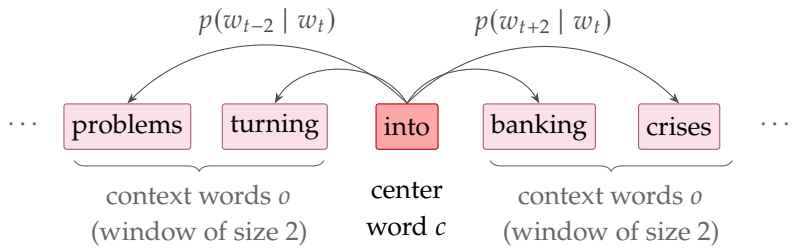


Figure 27: Context words within a window around the center word c.

Definition 13.1 [WORD2VEC]. WORD2VEC (Mikolov et al., 2013) trains the vectors $u_1, \ldots, u_n, v_1, \ldots, v_n$ of all possible words by

$$\arg\max_{u_1,\ldots,u_n,v_1,\ldots,v_n} \sum_{c,o} \log p(o \mid c),$$

summing over all combinations of c and o, for example, for each word c, all words o within 5 steps. The parameters are $\theta = (v_{\text{aardvark}}, \ldots, v_{\text{zebra}}, u_{\text{aardvark}}, \ldots, u_{\text{zebra}})$.

Why two vectors per word? It makes optimization easier. At the end we average them, which gives a representation of each word.

The vocabulary might be huge, so the denominator might be really costly to compute. Another idea is a *binary* classification problem, "is this the right word or not":

$$p(o \text{ is the right word} \mid c) = \sigma(u_o^T v_c) = \frac{1}{1 + \exp(-u_o^T v_c)}.$$

This is not enough by itself: maximizing it alone is solved by making all the vectors huge and aligned. So also include randomly chosen "negatives" w,

$$p(o \text{ is the wrong word} \mid c) = \sigma(-u_o^T v_c) = \frac{1}{1 + \exp(u_o^T v_c)},$$

and optimize

$$\arg\max_{u_1,\ldots,u_n,v_1,\ldots,v_n} \sum_{c,o} \Big(\log \sigma(u_o^T v_c) + \sum_{w} \log \sigma(-u_w^T v_c) \Big).$$

The intuition is to push v_c toward u_o and away from the other vectors u_w.

The learned vectors show algebraic relations (Figure 28),

$$\text{vec("woman")} - \text{vec("man")} \simeq \text{vec("aunt")} - \text{vec("uncle")} \simeq \text{vec("queen")} - \text{vec("king")},$$

although this is a little idealized and most relationships are not nearly this nice. With a model trained on a 6-billion-word corpus of news articles, such offsets answer analogy questions about capital cities (Athens : Greece :: Oslo : Norway), currencies, cities and states, gender (brother : sister :: grandson : granddaughter), adjectives and adverbs, opposites, comparatives, superlatives,

participles, nationality adjectives, tenses, and plurals; and relationships such as France–Paris give Italy: Rome and Japan: Tokyo, Einstein–scientist gives Messi: midfielder and Picasso: painter, and Japan–sushi gives Germany: bratwurst.

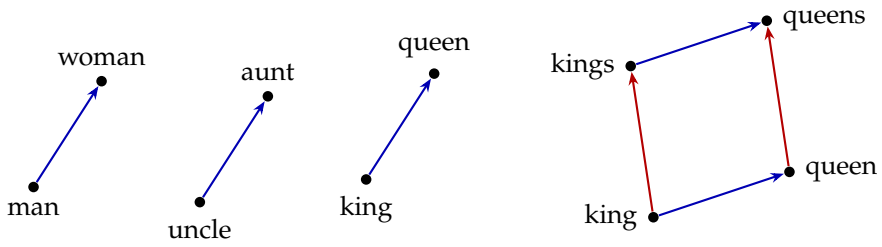


Figure 28: Relationships as consistent vector offsets (gender in blue, number in red). Schematic.

Word embeddings are used in place of one-hot vectors and are much more meaningful for downstream applications: train them on a large unlabeled corpus, then use them as input to a supervised model trained on a much smaller corpus. This is a simple type of unsupervised pretraining.

13.3 Contextual representations and ELMo

A word embedding does not change when the word is used in different ways: "play" has the same word2vec representation in "Let's play baseball" and "I saw a play yesterday," even though the meanings differ. To learn representations that depend on context: train a language model, run it on a sentence, and use its hidden state as the representation for downstream tasks. Two questions follow: how to train the best language model for this, and how to use it downstream.

ELMo uses a bidirectional LSTM for context-dependent embeddings. A forward LM predicts the next word and a backward LM predicts the previous word; both are trained as language models. Together, all the hidden states at position t, $h^{\text{fwd}}_{t,i}$ and $h^{\text{bwd}}_{t,i}$ for layers $i = 1, \ldots, L$, form a representation of the word at t (for example "cute" in "A cute puppy"):

$$\text{simple version: } \text{ELMO}_t = \left[h^{\text{fwd}}_{t,L}, h^{\text{bwd}}_{t,L}\right], \qquad \text{complex version: } \text{ELMO}_t = \gamma \sum_{i=1}^{L} w_i \left[h^{\text{fwd}}_{t,i}, h^{\text{bwd}}_{t,i}\right],$$

using the top-layer hidden states, or a combination with task-specific weights γ, w_i learned as part of the downstream task. The ELMo representation is concatenated to the word embedding (or one-hot vector) as the input to a downstream task-specific sequence model, for example the encoder and decoder of a translation model. This provides a context-specific and semantically meaningful representation of each token. The ELMo paper does not test on translation, but on question answering, textual entailment, semantic role labeling, coreference resolution, named entity extraction, and sentiment analysis, and the language model helps with all of them.

13.4 BERT

Can we use a transformer instead? The decoder part of a transformer, with masked self-attention, is a proper language model, but it has a direction (forward, depending on the masking), while ELMo is bidirectional. We could train two transformers and make a "transformer ELMo." Could we instead remove the mask and have one transformer? Then predicting the next word is trivially easy: self-attention at time t can access the "right answer" from the input at time $t + 1$.

Definition 13.2 [BERT**].** BERT is essentially the encoder part of a transformer, with no masked self-attention and no shift of the outputs by one. It is trained by randomly replacing 15% of the input tokens with a `[MASK]` token and predicting the original tokens: from "I `[MASK]` therefore I `[MASK]`" output "I think therefore I am."

Needing to predict the missing words forces the model to "work hard" to learn a good representation, without masked self-attention, which makes it bidirectional.

The input consists of *two* sentences, since many downstream tasks (question answering, natural language inference) process two sentences. Pairs of sentences in the data are transformed in two ways: randomly replace 15% of the tokens with `[MASK]`, and randomly swap the order of the sentences 50% of the time. Training has two outputs:

- a binary classifier on the first output: does the first sentence follow the second or precede it? This forces learning sentence-level representations;
- reconstruction of all tokens at each time step (predicting the actual token in place of each `[MASK]`), which forces learning context-dependent word-level representations.

To use BERT, put a cross-entropy loss on only the first output, replacing the sentence-order classifier with a task classifier (entailment classification, semantic equivalence such as Quora question pairs, or sentiment classification), and fine-tune the whole model end to end on the new task. The per-position outputs can also be fine-tuned: to highlight the span of a paragraph that contains the answer to a question, or to label each position with a named-entity class (person name, location, other categories). Features can also be pulled out, just like with ELMo: the output of each encoder layer along a token's path is a feature for that token. For named-entity recognition on CoNLL-2003, the dev F1 score is 91.0 using the first layer (the embedding), 94.9 using the last hidden layer, 95.5 summing all 12 layers, 95.6 using the second-to-last layer, 95.9 summing the last four, and 96.1 concatenating the last four.

BERT results are extremely good. On GLUE, a battery of varied natural language understanding tasks, the average score is 74.0 for the pre-OpenAI state of the art, 71.0 for BiLSTM with ELMo and attention, 75.1 for OpenAI GPT, 79.6 for $\text{BERT}_{\text{BASE}}$ (12 layers), and 82.1 for $\text{BERT}_{\text{LARGE}}$ (24 layers). Since then BERT has been applied to nearly every NLP task imaginable, often making a huge difference in performance.

13.5 GPT and summary

One-directional (forward) transformers do have one big advantage over BERT: generation is not really possible with BERT, but a forward model with masked attention can do it. GPT (GPT-2, GPT-3, and so on) is the classic example; given a short prompt about unicorns discovered in the Andes, GPT-2 continues with several coherent paragraphs.

Model	Architecture	Trade-off
BERT	bidirectional transformer	great representations; cannot generate text
OpenAI GPT	one-directional transformer	can generate text; OK representations
ELMo	bidirectional LSTM	OK representations (largely supplanted by BERT)

Pretrained language models are trained on very large unlabeled text datasets, such as Wikipedia, often hundreds or thousands of millions of sentences. Their representations depend on context (the meaning of a word is informed by the whole sentence) and can even represent entire sentences (for example BERT's first output token). Fine-tuning modifies all the weights of the language model, rather than just using its hidden states as features. This works extremely well and is very important in modern NLP.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 13 (Spring 2021).

Lecture 14: Learning-Based Control and Imitation

14.1 From prediction to control

So far we have learned to *predict*: labels from images, text from speech, translations. What about learning to *control*, for example a robot arm or a car? Three things change.

- In prediction the data is i.i.d., $p(\mathcal{D}) = \prod_i p(y_i \mid x_i)p(x_i)$: the output y_1 does not change the input x_2. This is very important, because it lets us focus on the highest *average* accuracy over the whole dataset. In control, each decision can change future inputs, and mistakes are not equally costly: a wrong choice on a cliffside road is a disaster, while one on a straight road may be OK.
- Prediction has ground-truth labels ("puppy"); control may have only high-level supervision, such as the goal "drive to the grocery store." What steering command is that?
- The objective is not to predict the right label but to accomplish the task.

These are not just issues for control: real-world deployments of machine learning often have the same feedback issues. For example, the decisions of a traffic prediction system might affect the routes people take, which changes the traffic. Reinforcement learning will address all of these issues, one piece at a time.

14.2 Terminology

The input is now an observation $\mathbf{o}_t$ (it used to be x), the output an action $\mathbf{a}_t$ (it used to be y), and the model a policy $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ (it used to be $p_\theta(y \mid x)$): seeing a tiger, choose among running away, ignoring it, and petting it. Underneath is a state $\mathbf{s}_t$, and $\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)$ is a fully observed policy. The observation is what the agent perceives (an image of a cheetah behind a car); the state is the underlying configuration of the world (the positions and velocities). This distinction will be very important later.

The states evolve by the dynamics $p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$ (Figure 29), which satisfy the Markov property: $\mathbf{s}_{t+1}$ is independent of $\mathbf{s}_{t-1}$ given $\mathbf{s}_t$ and $\mathbf{a}_t$.

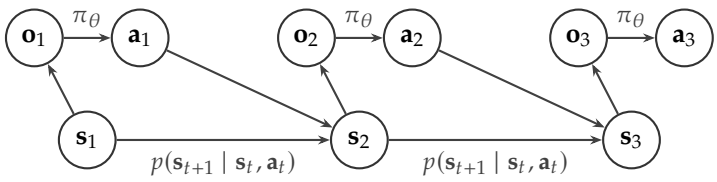


Figure 29: States $\mathbf{s}_t$ produce observations $\mathbf{o}_t$; the policy chooses actions $\mathbf{a}_t$, which drive the state transitions.

In the control literature (Pontryagin), states and actions are often written $\mathbf{x}_t$ and $\mathbf{u}_t$ (from the Russian *upravlenie*, "control"), while the notation here follows Bellman.

14.3 Behavioral cloning

The simplest approach to IMITATION LEARNING is BEHAVIORAL CLONING: record observations $\mathbf{o}_t$ and actions $\mathbf{a}_t$ from a human expert (for example camera images and steering commands from a driver), store them as training data, and train $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ by supervised learning.

Does it work? In principle, no. A small mistake puts the policy in a slightly unfamiliar state, where it makes a bigger mistake, and its trajectory drifts away from the training trajectory (Figure 30). Yet in practice it can: NVIDIA's end-to-end driving system (Bojarski et al., 2016) learned to drive this way.

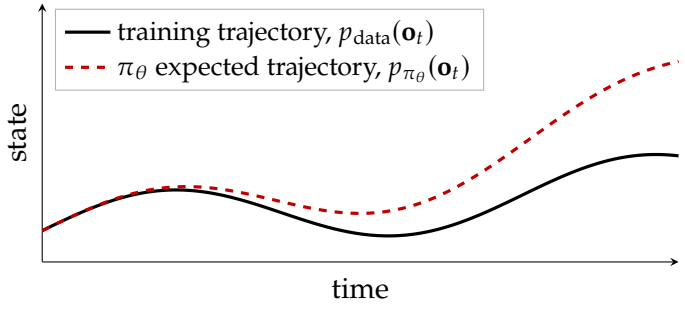


Figure 30: Compounding errors under behavioral cloning: the observations visited by the policy drift away from the training data. Schematic.

The problem is that $p_{\text{data}}(\mathbf{o}_t) \neq p_{\pi_\theta}(\mathbf{o}_t)$. This is exactly the distributional shift of autoregressive sequence models (Lecture 10): after one unlikely but possible mistake ("I drive"), the network sees inputs it never saw during training and outputs nonsense. Scheduled sampling fixed that by feeding the model's own predictions during training. In control we could likewise take the predicted action $\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ and observe the resulting $\mathbf{o}_{t+1}$, but that requires interacting with the world, because we do not know $p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$.

14.4 Getting behavioral cloning to work

Why did the driving system work? Besides the center camera, it recorded left and right cameras. Their images are labeled with the recorded steering angle, adjusted for the shift and rotation, so that they teach the policy to steer back toward the center. Random shifts and rotations augment the data further, and the CNN is trained by backpropagating the error between the desired and the network-computed steering command.

More generally, if $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ is *very* accurate, then maybe $p_{\text{data}}(\mathbf{o}_t) \approx p_{\pi_\theta}(\mathbf{o}_t)$. Why might we fail to fit the expert?

1. *Non-Markovian behavior.* A policy $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ depends only on the current observation: if it sees the same thing twice, it does the same thing twice, regardless of what happened before. This is often very unnatural for human demonstrators, whose behavior depends on all past observations, $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_1, \ldots, \mathbf{o}_t)$. Feeding a variable number of frames into one network needs too many weights; instead, process each frame with a CNN with shared weights and feed the results into an RNN, whose state summarizes the history. LSTM cells typically work better here.

2. *Multimodal behavior.* To go around a tree, an expert may pass on the left or on the right. With discrete actions a softmax over $p(a_1), p(a_2), p(a_3)$ represents both modes, but a single Gaussian over a continuous action averages them and drives into the tree. Remedies:

   - output a *mixture of Gaussians*, $\pi(\mathbf{a} \mid \mathbf{o}) = \sum_i w_i \mathcal{N}(\mu_i, \Sigma_i)$, with the network producing $w_1, \mu_1, \Sigma_1, \ldots, w_N, \mu_N, \Sigma_N$;
   - a *latent variable model*: feed a random $\xi \sim \mathcal{N}(0, \mathbf{I})$ into the network along with the observation, so that a unimodal output can represent a multimodal distribution (examples: conditional variational autoencoders, normalizing flows/realNVP, Stein variational gradient descent);
   - *autoregressive discretization*: output a discretized distribution over dimension 1 only, sample its value, feed it back into the network to get a discretized distribution over dimension 2, and so on.

   Better ways to model multimodal distributions come with generative models (Lectures 17–19).

Thus behavioral cloning, which should not work in principle, sometimes works well with hacks such as the left and right images, models with memory (RNNs), better distribution modeling, and generally care to get high accuracy.

14.5 DAgger

Can we make $p_{\text{data}}(\mathbf{o}_t) = p_{\pi_\theta}(\mathbf{o}_t)$? The idea is to be clever about $p_{\text{data}}(\mathbf{o}_t)$ instead of $p_{\pi_\theta}(\mathbf{o}_t)$: collect training data from $p_{\pi_\theta}(\mathbf{o}_t)$, simply by running the policy. But then the data needs action labels.

Algorithm 14.1: DAgger (Dataset Aggregation; Ross et al., 2011)

train $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \ldots, \mathbf{o}_N, \mathbf{a}_N\}$
repeat
  run $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ to get a dataset $\mathcal{D}_\pi = \{\mathbf{o}_1, \ldots, \mathbf{o}_M\}$
  ask a human to label $\mathcal{D}_\pi$ with actions $\mathbf{a}_t$
  aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$
  train $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ on $\mathcal{D}$
until done

DAgger was used, for example, to train a drone to fly through a forest from camera images. Its

problem is the labeling step: a human must look at the observations $\mathbf{o}_t$ that the policy visits and provide the actions $\mathbf{a}_t$ they would have taken, which is costly.

DAgger addresses the distribution mismatch but requires costly data collection and labeling, and behavioral cloning needs some (heuristic) hacks and very accurate models. The recommendation is to try behavioral cloning first, but prepare to be disappointed. The other two differences from prediction, high-level supervision and an objective of accomplishing the task, are addressed by reinforcement learning (Lecture 15).

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 14 (Spring 2021).

Lecture 15: Reinforcement Learning

15.1 Rewards and Markov decision processes

Prediction assumes i.i.d. data, ground-truth supervision, and the objective of predicting the right label. In control, each decision can change future inputs, supervision may be high-level (a goal), and the objective is to accomplish the task (Lecture 14). Reinforcement learning addresses the last two.

How do we specify what we want? A reward function $r(\mathbf{s}, \mathbf{a})$ tells us which states and actions are better (a car driving smoothly gets high reward, a crash low reward). The policy is now written $\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)$ in terms of states; observations come back later. We want to maximize reward over all time, not just greedily.

Definition 15.1 [Markov decision process]. A Markov decision process (MDP) is $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$: a state space $\mathcal{S}$ of states $s \in \mathcal{S}$ and an action space $\mathcal{A}$ of actions $a \in \mathcal{A}$ (each discrete or continuous), a transition operator $\mathcal{T}(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t) = p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$ (also called the transition probability or the dynamics), and a reward function $r \colon \mathcal{S} \times \mathcal{A} \to \mathbb{R}$.

A partially observed MDP adds an observation space $\mathcal{O}$ and an emission probability $\mathcal{E} = p(\mathbf{o}_t \mid \mathbf{s}_t)$, giving $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{E}, r\}$. For now we stick to regular (fully observed) MDPs.

15.2 The goal of reinforcement learning

The policy and the dynamics define a distribution over trajectories $\tau = (\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)$, a Markov chain on $(\mathbf{s}, \mathbf{a})$:

$$p_\theta(\tau) = p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)\, p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t).$$

The goal is

$$\theta^\star = \arg\max_\theta \underbrace{E_{\tau \sim p_\theta(\tau)}\left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right]}_{J(\theta)}.$$

The objective can be evaluated by sampling: run the policy N times and average,

$$J(\theta) = E_{\tau \sim p_\theta(\tau)}\left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}),$$

a sum over samples from π_θ.

15.3 Policy gradients

Write $\pi_\theta(\tau)$ for $p_\theta(\tau)$ and $r(\tau) = \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t)$, so that $J(\theta) = E_{\tau\sim\pi_\theta(\tau)}[r(\tau)] = \int \pi_\theta(\tau) r(\tau)\, d\tau$. A convenient identity is

$$\pi_\theta(\tau)\nabla_\theta \log \pi_\theta(\tau) = \pi_\theta(\tau)\frac{\nabla_\theta \pi_\theta(\tau)}{\pi_\theta(\tau)} = \nabla_\theta \pi_\theta(\tau).$$

Proposition 15.2 [Policy gradient].

$$\nabla_\theta J(\theta) = E_{\tau\sim\pi_\theta(\tau)}\big[\nabla_\theta \log \pi_\theta(\tau)\, r(\tau)\big] = E_{\tau\sim\pi_\theta(\tau)}\Big[\Big(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)\Big)\Big(\sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t)\Big)\Big].$$

Proof. By the identity, $\nabla_\theta J(\theta) = \int \nabla_\theta \pi_\theta(\tau) r(\tau)\, d\tau = \int \pi_\theta(\tau)\nabla_\theta \log \pi_\theta(\tau) r(\tau)\, d\tau$, which is the first expectation. Taking the log of both sides of the trajectory distribution,

$$\log \pi_\theta(\tau) = \log p(\mathbf{s}_1) + \sum_{t=1}^{T}\big[\log \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t) + \log p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)\big],$$

and the initial-state and dynamics terms do not depend on θ, so their gradients vanish. □

The dynamics drop out, so the gradient can be estimated from samples without knowing them:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\Big(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})\Big)\Big(\sum_{t=1}^{T} r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})\Big), \qquad \theta \leftarrow \theta + \alpha\nabla_\theta J(\theta).$$

Here $\nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})$ is the gradient of the log probability that the policy network assigns to the action it took.

REINFORCE, below, is an instance of the generic loop of reinforcement learning (Figure 31): generate samples, fit a model to estimate the return, and improve the policy.

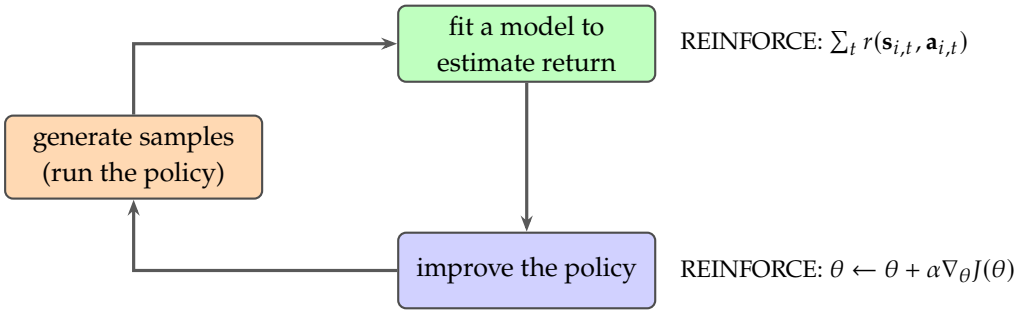


Figure 31: The anatomy of a reinforcement learning algorithm.

Algorithm 15.1: REINFORCE

repeat
  sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)$ (run the policy)
  $\nabla_\theta J(\theta) \approx \sum_i \big(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i \mid \mathbf{s}_t^i)\big)\big(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i)\big)$
  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
until done

Comparison to maximum likelihood. Behavioral cloning trains the same policy by maximum likelihood on expert data,

$$\nabla_\theta J_{\mathrm{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \Big(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) \Big) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log \pi_\theta(\tau_i),$$

while the policy gradient is $\frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\tau_i) r(\tau_i)$: a maximum-likelihood gradient on the policy's own samples, weighted by their reward. Good stuff is made more likely and bad stuff less likely; this simply formalizes "trial and error."

Partial observability. With observations, $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \big(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{o}_{i,t})\big)\big(\sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})\big)$. The Markov property was never used, so the policy gradient applies to partially observed MDPs without modification.

15.4 Making policy gradients work

Reward to go. By causality, the policy at time t' cannot affect the reward at time t when $t < t'$. So each log-probability term needs only the rewards from its own time onward:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})\, \hat{Q}_{i,t}, \qquad \hat{Q}_{i,t} = \sum_{t'=t}^{T} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}),$$

where $\hat{Q}_{i,t}$ is the REWARD TO GO. Dropping the past rewards removes terms with zero mean, which gives a better (lower-variance) estimator.

Baselines. Subtract a BASELINE b, for example the average reward $b = \frac{1}{N} \sum_{i=1}^{N} r(\tau_i)$:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log \pi_\theta(\tau_i) \big[r(\tau_i) - b \big].$$

Are we allowed to do that? Yes, since by the identity

$$E\big[\nabla_\theta \log \pi_\theta(\tau) b\big] = \int \pi_\theta(\tau) \nabla_\theta \log \pi_\theta(\tau) b \, d\tau = \int \nabla_\theta \pi_\theta(\tau) b \, d\tau = b \nabla_\theta \int \pi_\theta(\tau) \, d\tau = b \nabla_\theta 1 = 0.$$

Subtracting a baseline is unbiased in expectation. The average reward is not the best baseline, but it is pretty good.

15.5 Off-policy policy gradients

The policy gradient $\nabla_\theta J(\theta) = E_{\tau \sim \pi_\theta(\tau)}[\nabla_\theta \log \pi_\theta(\tau) r(\tau)]$ is ON-POLICY: the expectation is under the current policy, so the sampling step of REINFORCE cannot be skipped and must be redone after every update. Since neural networks change only a little with each gradient step, on-policy learning can be extremely inefficient.

What if we have samples from some other $\bar{\pi}(\tau)$ instead? IMPORTANCE SAMPLING rewrites an expectation under p as one under q:

$$E_{x \sim p(x)}[f(x)] = \int p(x) f(x) \, dx = \int q(x) \frac{p(x)}{q(x)} f(x) \, dx = E_{x \sim q(x)}\Big[\frac{p(x)}{q(x)} f(x)\Big].$$

Hence $J(\theta) = E_{\tau \sim \bar{\pi}(\tau)}\big[\frac{\pi_\theta(\tau)}{\bar{\pi}(\tau)} r(\tau)\big]$, and the initial-state and dynamics factors cancel in the ratio:

$$\frac{\pi_\theta(\tau)}{\bar{\pi}(\tau)} = \frac{p(\mathbf{s}_1) \prod_{t=1}^T \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t) p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)}{p(\mathbf{s}_1) \prod_{t=1}^T \bar{\pi}(\mathbf{a}_t \mid \mathbf{s}_t) p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)} = \frac{\prod_{t=1}^T \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)}{\prod_{t=1}^T \bar{\pi}(\mathbf{a}_t \mid \mathbf{s}_t)}.$$

To estimate the value of some *new* parameters θ' from samples of π_θ,

$$\begin{aligned} J(\theta') &= E_{\tau \sim \pi_\theta(\tau)}\Big[\frac{\pi_{\theta'}(\tau)}{\pi_\theta(\tau)} r(\tau)\Big], \\ \nabla_{\theta'} J(\theta') &= E_{\tau \sim \pi_\theta(\tau)}\Big[\frac{\nabla_{\theta'} \pi_{\theta'}(\tau)}{\pi_\theta(\tau)} r(\tau)\Big] = E_{\tau \sim \pi_\theta(\tau)}\Big[\frac{\pi_{\theta'}(\tau)}{\pi_\theta(\tau)} \nabla_{\theta'} \log \pi_{\theta'}(\tau) r(\tau)\Big], \end{aligned}$$

since only $\pi_{\theta'}(\tau)$ depends on θ'. Estimated locally at $\theta = \theta'$, the ratio is 1 and we recover the on-policy gradient. When $\theta \neq \theta'$,

$$\begin{aligned} \nabla_{\theta'} J(\theta') &= E_{\tau \sim \pi_\theta(\tau)}\Big[\Big(\prod_{t=1}^T \frac{\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)}\Big)\Big(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)\Big)\Big(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)\Big)\Big] \\ &= E_{\tau \sim \pi_\theta(\tau)}\Big[\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)\Big(\prod_{t'=1}^t \frac{\pi_{\theta'}(\mathbf{a}_{t'} \mid \mathbf{s}_{t'})}{\pi_\theta(\mathbf{a}_{t'} \mid \mathbf{s}_{t'})}\Big)\Big(\sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'})\Big(\prod_{t''=t}^{t'} \frac{\pi_{\theta'}(\mathbf{a}_{t''} \mid \mathbf{s}_{t''})}{\pi_\theta(\mathbf{a}_{t''} \mid \mathbf{s}_{t''})}\Big)\Big)\Big], \end{aligned}$$

using causality: future actions do not affect the current weight. If we ignore the last product, we get a policy iteration algorithm (Lecture 16).

The remaining product $\prod_{t'=1}^{t}$ is exponential in T. So write the objective a bit differently. The on-policy gradient is $\frac{1}{N}\sum_i \sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})\hat{Q}_{i,t}$ with $(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \sim \pi_\theta(\mathbf{s}_t, \mathbf{a}_t)$, a sum over state-action marginals. The off-policy version weights each term by the ratio of marginals,

$$\begin{aligned}\nabla_{\theta'} J(\theta') &\approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\frac{\pi_{\theta'}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})}{\pi_\theta(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})}\nabla_{\theta'}\log \pi_{\theta'}(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})\hat{Q}_{i,t} \\ &= \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\frac{\pi_{\theta'}(\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{s}_{i,t})}\frac{\pi_{\theta'}(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})}\nabla_{\theta'}\log \pi_{\theta'}(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})\hat{Q}_{i,t},\end{aligned}$$

and the first-order approximation ignores the state ratio $\pi_{\theta'}(\mathbf{s}_{i,t})/\pi_\theta(\mathbf{s}_{i,t})$. This simplification is very commonly used in practice.

15.6 Implementation with automatic differentiation

Computing $\nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})$ explicitly for every sample is pretty inefficient. Instead, build a graph whose gradient is the policy gradient. Maximum likelihood has $J_{\mathrm{ML}}(\theta) \approx \frac{1}{N}\sum_i \sum_t \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})$, so implement the policy gradient as a weighted maximum likelihood with the "pseudo-loss"

$$\tilde{J}(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t})\hat{Q}_{i,t},$$

where the log-likelihood is a cross-entropy (discrete actions) or a squared error (Gaussian actions). With discrete actions, in TensorFlow-style pseudocode, the maximum-likelihood loss is the mean of the softmax cross entropies, and the policy gradient multiplies them by the $\hat{Q}_{i,t}$ values first:

```
# Given:
# actions - (N*T) x Da tensor of actions
# states - (N*T) x Ds tensor of states
# q_values - (N*T) x 1 tensor of estimated state-action values
# Build the graph:
logits = policy.predictions(states)  # (N*T) x Da tensor of action logits
negative_likelihoods = tf.nn.softmax_cross_entropy_with_logits(
    labels=actions, logits=logits)
weighted_negative_likelihoods = tf.multiply(negative_likelihoods, q_values)
loss = tf.reduce_mean(weighted_negative_likelihoods)
gradients = loss.gradients(loss, variables)
```

In practice, remember that the gradient has high variance. This is not the same as supervised learning: the gradients will be really noisy, so consider much larger batches. Tweaking learning rates is very hard; adaptive step size rules such as Adam can be OK-ish, and there are policy-gradient-specific step size methods. For example, TRUST REGION POLICY OPTIMIZATION (Schulman et al.,

2015) uses a natural gradient with automatic step adjustment, for discrete and continuous actions.

Policy gradients are used for learning control, for "hard" attention, where a network selects where to look (Mnih et al., *Recurrent Models of Visual Attention*), and for discrete latent variable models. In general, the policy gradient (REINFORCE) can be used in any setting where we must differentiate through a stochastic but non-differentiable operation.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 15 (Spring 2021).

Lecture 16: Q-Learning

16.1 Value functions and the policy gradient

REINFORCE with the reward to go uses

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) \hat{Q}_{i,t}, \qquad \hat{Q}_{i,t} = \sum_{t'=t}^{T} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}),$$

where $\hat{Q}_{i,t}$ estimates the expected reward if we take action $\mathbf{a}_{i,t}$ in state $\mathbf{s}_{i,t}$. It sums the rewards of a single trajectory, while many trajectories could have followed from the same point. Can we get a better estimate?

Definition 16.1 [Value functions]. For a policy π,

- the Q-function $Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{T} E_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t, \mathbf{a}_t\right]$ is the total expected reward from taking $\mathbf{a}_t$ in $\mathbf{s}_t$ (the true expected reward to go);
- the value function $V^\pi(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\left[Q^\pi(\mathbf{s}_t, \mathbf{a}_t)\right]$ is the total reward from $\mathbf{s}_t$;
- the advantage $A^\pi(\mathbf{s}_t, \mathbf{a}_t) = Q^\pi(\mathbf{s}_t, \mathbf{a}_t) - V^\pi(\mathbf{s}_t)$ is how much better $\mathbf{a}_t$ is than the average action.

With the true Q, the gradient is $\frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) Q(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$. For the baseline, instead of averaging rewards over all samples, average the Q-values of the actions in the same state; that average is exactly $V(\mathbf{s}_t)$. So

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) A^\pi(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}).$$

The better the estimate of A^π, the lower the variance. The REINFORCE estimate $\sum_{t'} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) - b$ is unbiased but a high-variance single-sample estimate.

16.2 Fitting value functions

Should we fit Q^π, V^π, or A^π? Since $Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \sum_{t'=t+1}^{T} E_{\pi_\theta}[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t, \mathbf{a}_t] = r(\mathbf{s}_t, \mathbf{a}_t) + E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)}[V^\pi(\mathbf{s}_{t+1})]$, we can approximate with a single sample

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^\pi(\mathbf{s}_{t+1}), \qquad A^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^\pi(\mathbf{s}_{t+1}) - V^\pi(\mathbf{s}_t),$$

so it suffices to fit $V^\pi(\mathbf{s})$, with a neural network $\hat{V}^\pi_\phi(\mathbf{s})$ with parameters ϕ.

Fitting V^π is POLICY EVALUATION: $V^\pi(\mathbf{s}_t) = \sum_{t'=t}^{T} E_{\pi_\theta}[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t]$, and $J(\theta) = E_{\mathbf{s}_1 \sim p(\mathbf{s}_1)}[V^\pi(\mathbf{s}_1)]$. *Monte Carlo policy evaluation*, which is what the policy gradient does, uses $V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^{T} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$, or better $\frac{1}{N} \sum_{i=1}^{N} \sum_{t'=t}^{T} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$, which requires resetting the simulator to $\mathbf{s}_t$. With function approximation, the single-sample version is not as good as averaging but still pretty good, because the same function must fit many samples from similar states. The training data are $\left\{(\mathbf{s}_{i,t}, y_{i,t})\right\}$ with $y_{i,t} = \sum_{t'=t}^{T} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$, and the fit is supervised regression, $\mathcal{L}(\phi) = \frac{1}{2} \sum_i \|\hat{V}_\phi^\pi(\mathbf{s}_i) - y_i\|^2$.

Can we do better? The ideal target is

$$y_{i,t} = \sum_{t'=t}^{T} E_{\pi_\theta}[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_{i,t}] \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + V^\pi(\mathbf{s}_{i,t+1}) \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1}),$$

directly using the previously fitted value function. The training data become $\left\{(\mathbf{s}_{i,t}, r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1}))\right\}$, with the same regression loss. This is called a BOOTSTRAPPED estimate. Examples of policy evaluation are TD-Gammon (Tesauro, 1992) and AlphaGo (Silver et al., 2016): the reward is the game outcome and $\hat{V}_\phi^\pi(\mathbf{s}_t)$ the expected outcome given the board state.

16.3 Actor-critic algorithms

What if the episode length T is infinite? Then $\hat{V}_\phi^\pi$ can grow infinitely large in many cases. The simple trick is that it is better to get rewards sooner than later: use the target

$$y_{i,t} \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1}),$$

with a DISCOUNT FACTOR $\gamma \in [0, 1]$ (0.99 works well). The discount changes the MDP: every step leads to an absorbing "dead" state with probability $1 - \gamma$ (Figure 32), which suits both episodic and continuous (cyclical) tasks.

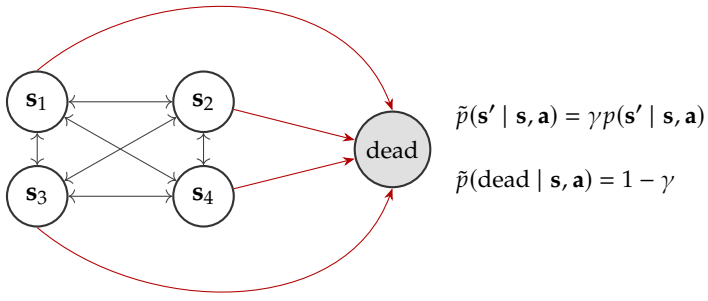


Figure 32: Discounting as a modified MDP that dies with probability $1 - \gamma$ at every step.

These ACTOR-CRITIC algorithms, below, train the policy (the ACTOR) with advantages computed from a fitted value function (the CRITIC). A two-network design, one network for $\hat{V}_\phi^\pi(\mathbf{s})$ and one for $\pi_\theta(\mathbf{a} \mid \mathbf{s})$, is simple and stable but shares no features between actor and critic; a shared network with two

output heads shares them.

Algorithm 16.1: Batch actor-critic

repeat
  sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ (run it on the robot)
  fit $\hat{V}^\pi_\phi(\mathbf{s})$ to sampled reward sums
  evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}^\pi_\phi(\mathbf{s}'_i) - \hat{V}^\pi_\phi(\mathbf{s}_i)$
  $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i \mid \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
until done

Algorithm 16.2: Online actor-critic

repeat
  take action $\mathbf{a} \sim \pi_\theta(\mathbf{a} \mid \mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
  update $\hat{V}^\pi_\phi$ using the target $r + \gamma \hat{V}^\pi_\phi(\mathbf{s}')$
  evaluate $\hat{A}^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}^\pi_\phi(\mathbf{s}') - \hat{V}^\pi_\phi(\mathbf{s})$
  $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a} \mid \mathbf{s}) \hat{A}^\pi(\mathbf{s}, \mathbf{a})$
  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
until done

16.4 From policy iteration to value iteration

Can we use just a value function? $A^\pi(\mathbf{s}_t, \mathbf{a}_t)$ says how much better $\mathbf{a}_t$ is than the average action according to π, so $\arg\max_{\mathbf{a}_t} A^\pi(\mathbf{s}_t, \mathbf{a}_t)$ is the best action from $\mathbf{s}_t$ if we then follow π: at least as good as any $\mathbf{a}_t \sim \pi(\mathbf{a}_t \mid \mathbf{s}_t)$, regardless of what $\pi(\mathbf{a}_t \mid \mathbf{s}_t)$ is. So forget policy gradients and use

$$\pi'(\mathbf{a}_t \mid \mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} A^\pi(\mathbf{s}_t, \mathbf{a}_t), \\ 0 & \text{otherwise,} \end{cases}$$

which is at least as good as π, and probably better. Policy iteration repeats: evaluate $A^\pi(\mathbf{s}, \mathbf{a})$, then set $\pi \leftarrow \pi'$. As before, $A^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma E[V^\pi(\mathbf{s}')] - V^\pi(\mathbf{s})$, so we evaluate $V^\pi(\mathbf{s})$.

Dynamic programming. Assume we know $p(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})$ and that $\mathbf{s}$ and $\mathbf{a}$ are both discrete and small, for example 16 states with 4 actions each. Then the full $V^\pi(\mathbf{s})$ fits in a table, and $\mathcal{T}$ is a $16 \times 16 \times 4$ tensor. The bootstrapped update

$$V^\pi(\mathbf{s}) \leftarrow E_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})}\left[r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s},\mathbf{a})}[V^\pi(\mathbf{s}')]\right]$$

just uses the current estimate on the right. Since π' is a deterministic policy $\pi(\mathbf{s}) = \mathbf{a}$, this simplifies to $V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s},\pi(\mathbf{s}))}[V^\pi(\mathbf{s}')]$, which is policy evaluation within policy iteration.

Even simpler: $\arg\max_{\mathbf{a}_t} A^\pi(\mathbf{s}_t, \mathbf{a}_t) = \arg\max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$, and $Q^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma E[V^\pi(\mathbf{s}')]$ is a bit simpler. Taking the max over a table of $Q(\mathbf{s}, \mathbf{a})$ gives the policy, and also approximates the new value. So skip the policy and compute values directly.

Algorithm 16.3: Value iteration

repeat
  set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')]$
  set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$
until converged

Fitted value iteration. A big table with one entry per discrete $\mathbf{s}$ suffers the CURSE OF DIMENSIONALITY: for 200×200 color images, $|\mathcal{S}| = (255^3)^{200\times200}$, more than the number of atoms in the universe. Instead use a neural network $V_\phi \colon \mathcal{S} \to \mathbb{R}$ with the loss $\mathcal{L}(\phi) = \frac{1}{2}\left\|V_\phi(\mathbf{s}) - \max_{\mathbf{a}} Q^\pi(\mathbf{s}, \mathbf{a})\right\|^2$:

1. set $\mathbf{y}_i \leftarrow \max_{\mathbf{a}_i}\left(r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\phi(\mathbf{s}_i')]\right)$;
2. set $\phi \leftarrow \arg\min_\phi \frac{1}{2}\sum_i \|V_\phi(\mathbf{s}_i) - \mathbf{y}_i\|^2$.

16.5 Fitted Q-iteration

The max in step 1 needs the outcomes of *different* actions from the same state, so it requires knowing the transition dynamics. Back to policy iteration: instead of evaluating $V^\pi(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma E_{\mathbf{s}'\sim p(\mathbf{s}'|\mathbf{s},\pi(\mathbf{s}))}[V^\pi(\mathbf{s}')]$, evaluate

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E_{\mathbf{s}'\sim p(\mathbf{s}'|\mathbf{s},\mathbf{a})}[Q^\pi(\mathbf{s}', \pi(\mathbf{s}'))],$$

which can be fit using samples. Doing the "max" trick again with Q-values, the fitted Q-iteration algorithm sets $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma E[V_\phi(\mathbf{s}_i')]$, approximating $E[V(\mathbf{s}_i')] \approx \max_{\mathbf{a}'} Q_\phi(\mathbf{s}_i', \mathbf{a}_i')$, and then fits $\phi \leftarrow \arg\min_\phi \frac{1}{2}\sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$. This does not require simulating different actions. It works even for off-policy samples (unlike actor-critic) and needs only one network, with no high-variance policy gradient; but it has no convergence guarantees with non-linear function approximation.

Algorithm 16.4: Full fitted Q-iteration

Input: dataset size N, collection policy, iterations K, gradient steps S
repeat
  collect a dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i', r_i)\}$ using some policy
  for $k = 1, \ldots, K$ **do**
    set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}_i'} Q_\phi(\mathbf{s}_i', \mathbf{a}_i')$
    set $\phi \leftarrow \arg\min_\phi \frac{1}{2}\sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$ (with S gradient steps)
until done

The network $Q_\phi(\mathbf{s}, \mathbf{a})$ either takes both $\mathbf{s}$ and $\mathbf{a}$ as input and outputs one value, which is more common with continuous actions, or takes $\mathbf{s}$ and outputs $Q_\phi(\mathbf{s}, \mathbf{a}_1), Q_\phi(\mathbf{s}, \mathbf{a}_2), \ldots$ for every action,

which is more common with discrete actions.

16.6 Q-learning

The policy is $\mathbf{a} = \arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})$. Doing one step at a time gives ONLINE Q-LEARNING:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$;
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$;
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)\big(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\big)$.

It is off-policy, so there are many choices for step 1.

Exploration. The final policy is greedy, $\pi(\mathbf{a}_t \mid \mathbf{s}_t) = 1$ if $\mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q_\phi(\mathbf{s}_t, \mathbf{a}_t)$ and 0 otherwise. This is a bad idea for step 1, because it never tries actions it currently believes are worse. Alternatives are EPSILON-GREEDY exploration,

$$\pi(\mathbf{a}_t \mid \mathbf{s}_t) = \begin{cases} 1 - \epsilon & \text{if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q_\phi(\mathbf{s}_t, \mathbf{a}_t), \\ \epsilon / (|\mathcal{A}| - 1) & \text{otherwise,} \end{cases}$$

and BOLTZMANN EXPLORATION, $\pi(\mathbf{a}_t \mid \mathbf{s}_t) \propto \exp(Q_\phi(\mathbf{s}_t, \mathbf{a}_t))$.

What is wrong? Step 3 looks like gradient descent, which converges, right? But Q-learning is *not* gradient descent: in

$$\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)\Big(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \big[r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)\big]\Big)$$

there is no gradient through the target value. In addition, sequential states are strongly correlated, and the target value is always changing.

Replay buffers. Online Q-learning is the special case of fitted Q-iteration with $K = 1$ and one gradient step. Any collection policy works (with broad support), so instead of collecting a fresh dataset, load data from a REPLAY BUFFER $\mathcal{B}$ of transitions, still with one gradient step: sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$ and update $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)\big(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)]\big)$. The samples are no longer correlated, and multiple samples in the batch give a low-variance gradient. The buffer must be fed periodically by running a policy such as ϵ-greedy (Figure 33). Full Q-learning with a replay buffer repeats: collect a dataset using some policy and add it to $\mathcal{B}$; then K times, sample a batch from $\mathcal{B}$ and take the gradient step. $K = 1$ is common, though larger K is more efficient.

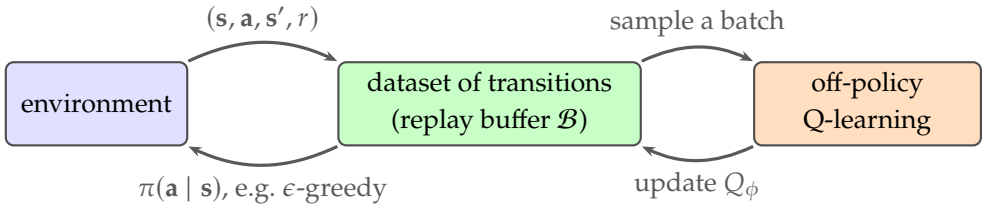


Figure 33: Q-learning with a replay buffer.

Target networks. The replay buffer fixes the correlation, but Q-learning is still not gradient descent: it takes one gradient step toward a moving target, while fitted Q-iteration solves a perfectly well-defined, stable regression. A TARGET NETWORK $Q_{\phi'}$ keeps the targets fixed in the inner loop.

Algorithm 16.5: Q-learning with replay buffer and target network

repeat
  save target network parameters: $\phi' \leftarrow \phi$
  for N times **do**
    collect a dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy, add it to $\mathcal{B}$
    for K times **do**
      sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$
      $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)\big(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}'_i, \mathbf{a}'_i)]\big)$
until done

The targets do not change in the inner loop, so it is supervised regression. The "classic" deep Q-learning algorithm, DQN (Mnih et al., 2013), is the case $K = 1$:

Algorithm 16.6: Deep Q-learning (DQN)

repeat
  take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
  sample a mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
  compute $y_j = r_j + \gamma \max_{\mathbf{a}'_j} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)$ using the target network $Q_{\phi'}$
  $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)\big(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j\big)$
  update ϕ': copy ϕ every N steps
until done

16.7 Continuous actions and practice

With continuous actions, the max in online Q-iteration is very inconvenient (but not impossible). The idea is to use actor-critic, but with Q-functions, to train off-policy:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$;
2. sample a mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly;

3. compute $y_j = r_j + \gamma E_{\mathbf{a}'_j \sim \pi_{\theta'}(\mathbf{a}'_j|\mathbf{s}'_j)}[Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)]$ using targets ϕ' and θ';
4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)\big(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j\big)$;
5. $\theta \leftarrow \theta + \beta \sum_j \nabla_\theta E_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_j)}[Q_\phi(\mathbf{s}_j, \mathbf{a})]$, a policy gradient;
6. update ϕ' and θ' every N steps.

Simple practical tips for Q-learning:

- Q-learning takes some care to stabilize: test on easy, reliable tasks first (such as Pong or Breakout, where learning curves are smooth, unlike harder games) to make sure the implementation is correct;
- large replay buffers help improve stability, since the method then looks more like fitted Q-iteration;
- it takes time, so be patient: it might be no better than random for a while;
- start with high exploration (ϵ) and gradually reduce it.

DQN ("Human-level control through deep reinforcement learning") applied Q-learning with convolutional networks to Atari games, using a replay buffer and a target network, a one-step backup, and one gradient step; it can be improved a lot with double Q-learning and other tricks. QT-Opt (Kalashnikov et al.) scales Q-learning with continuous actions to robotic grasping: stored data from all past experiments and live data collection feed training buffers, Bellman updaters compute $Q_T(\mathbf{s}, \mathbf{a}) = r + \max_{\mathbf{a}'} Q_\theta(\mathbf{s}', \mathbf{a}')$, and training threads minimize $\|Q_\theta(\mathbf{s}, \mathbf{a}) - Q_T(\mathbf{s}, \mathbf{a})\|^2$.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 16 (Spring 2021).

Lecture 17: Autoencoders and Latent Variable Models

17.1 Generative models

So far we have modeled $p(y \mid x)$, for example a label given an image. A generative model represents $p(x)$ itself, for example a Gaussian $\mathcal{N}(\mu, \sigma)$ fit to data points, or a language model

$$p(x) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2})p(x_4 \mid x_{1:3})p(x_5 \mid x_{1:4})p(x_6 \mid x_{1:5}),$$

where an RNN reads <START> (not actually a variable) and the previous words and outputs $\hat{x}_1, \ldots, \hat{x}_6$. These are just different ways to solve the same problem, called unsupervised learning: can we go from "language models" to "everything models"? The reasons to do it are the same as for language modeling: unsupervised pretraining on lots of data, representation learning, pretraining for later fine-tuning, and actually generating things.

17.2 Autoregressive generative models

Can we "language model" images? Before, a time step was a word; now a time step is a pixel, with some (arbitrary) ordering on pixels:

$$p(x) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_{1:2})p(x_4 \mid x_{1:3})p(x_5 \mid x_{1:4})p(x_6 \mid x_{1:5}).$$

Given the top of an image, we can ask the "image language model" to fill in the blanks. This is the main idea; the details are how to order the pixels and what kind of model to use.

Definition 17.1 [Autoregressive generative model]. To train an autoregressive generative model:

1. divide x into dimensions $x_1, \ldots, x_n$;
2. discretize each x_i into k values;
3. model $p(x)$ by the chain rule, $p(x) = \prod_i p(x_i \mid x_1, \ldots, x_{i-1})$, each factor being just a softmax;
4. use your favorite sequence model to model $p(x)$.

Such a model is used for *sampling*, by ancestral sampling in sequence (x_1, then x_2, and so on); for *completion*, by feeding in the actual values of the known x_i; and for *representations*, with the same idea as ELMo or BERT.

PixelRNN (van den Oord et al., 2016) generates pixels one at a time, left to right and top to bottom, and one color channel at a time:

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p(x_i \mid x_1, \ldots, x_{i-1}), \qquad p(x_i \mid \mathbf{x}_{<i}) = p(x_{i,R} \mid \mathbf{x}_{<i})\, p(x_{i,G} \mid \mathbf{x}_{<i}, x_{i,R})\, p(x_{i,B} \mid \mathbf{x}_{<i}, x_{i,R}, x_{i,G}),$$

each factor a 256-way softmax. In practice it is very slow; row-by-row LSTMs might struggle to capture spatial context (the pixels right above are "far away" in the sequence); and many improvements are possible, such as a diagonal BiLSTM.

PixelCNN makes this much faster by not building a full RNN over all pixels, but using a convolution to determine the value of a pixel from its neighborhood. The entries of the filter at pixels that have not been generated yet are masked out; stacking masked convolutions still lets pixels outside the immediate neighborhood influence x_i through earlier layers. Training can be parallelized, since all pixels are known; generation cannot, since each pixel needs the previous ones.

Pixel transformers (Parmar et al., *Image Transformer*, 2018) apply masked self-attention and position-wise networks, repeated N times, to the pixel sequence. For an RNN a pixel far back in the ordering, such as the one right above, is far away; for self-attention all pixels are equally "close." The problem is that the number of pixels can be huge, making attention prohibitively expensive, so attention is computed only for pixels that are not too far away (local 1D or 2D attention), which looks a little like PixelCNN. Trained on CIFAR-10, its samples are noticeably more coherent than PixelRNN's.

Conditional autoregressive models generate something conditioned on another piece of information, just like conditional language models: an encoder maps the context to the initial state. The encoder can be extremely simple (a class label, to generate images of a category such as African elephant, coral reef, or sorrel horse, as with a conditional PixelCNN) or extremely complex (a multimodal policy conditioned on observations in imitation learning).

Autoregressive generative models are "language models" for other types of data (more accurately, language models are a special type of autoregressive generative model), and can be represented by RNNs (such as LSTMs), local context models (PixelCNN), or transformers. Compared with the models that follow, they provide a full distribution with probabilities and are conceptually very simple, but they are very slow for large datapoints such as images and generally limited in image resolution.

17.3 Autoencoders

From a high level, an autoregressive model is trained to produce the *same* image that goes in as input; some structure in the model makes this "reconstruction" nontrivial, that is, prevents learning an identity function. In an RNN or PixelCNN, each pixel must be predicted from "previous" pixels or a masked neighborhood; in a pixel transformer, the structure comes from the masking in self-attention (as in BERT); without it, the task would be trivial. All of this is *spatial* structure. Can we use more abstract structure instead?

Definition 17.2 [Autoencoder]. An autoencoder is a network that encodes an input into a hidden state and then decodes the input as accurately as possible from that hidden state (Figure 34). The hidden state is the part used for downstream tasks.

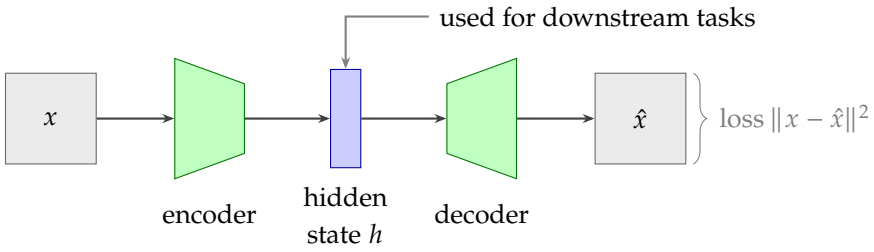


Figure 34: An autoencoder.

Forcing structure: something about the design of the model, the data processing, or the regularization must force the autoencoder to learn a structured representation.

- *Dimensionality*: make the hidden state smaller than the input and output, so the network must compress. Very simple to implement, but simply reducing dimensionality often does not provide the structure we want.
- *Sparsity*: force the hidden state to be sparse (most entries zero). A principled approach that can provide a "disentangled" representation, but harder in practice: it requires choosing the regularizer and adjusting hyperparameters.
- *Denoising*: corrupt the input with noise, forcing the autoencoder to distinguish noise from signal. Very simple to implement, but it is not clear which layer to use as the bottleneck, and there are many ad hoc choices (how much noise to add).
- *Probabilistic modeling*: force the hidden state to agree with a prior distribution (Lecture 18).

Bottleneck autoencoder. For example, a 100×100 image (10,000 dimensions) is encoded into 128 dimensions. If the encoder and decoder are both linear (usually not very interesting), this exactly recovers PCA. In general it is "non-linear dimensionality reduction," useful simply because algorithms that are only tractable in low dimensions (such as discretization) become applicable. For

example, Lange and Riedmiller (2012) learned to drive a slot car from camera images by running reinforcement learning on the bottleneck features of a deep autoencoder. Today this design is rather antiquated and rarely used.

Sparse autoencoder. Can we describe the input by a small set of "attributes"? A list of pixel colors (#FE057D, #FD0263, ...) is *dense*, with most values non-zero, and not structured; attributes such as has_ears = 1, has_wings = 0, has_wheels = 0 are very structured and *sparse*, since there are many possible attributes and most images have few of them. (The idea originated in neuroscience, where researchers believe the brain uses sparse representations; see "sparse coding.") A sparse autoencoder adds a *sparsity loss* on the hidden state h, for example the ℓ_1 norm $\sum_{j=1}^{D} |h_j|$, whose gradient is always $\text{sgn}(h_j)$ and which, like L1 regularization, prefers exact zeros. The hidden dimension may be very large, even larger than the input (OVERCOMPLETE), since sparsity rather than size provides the compression. Other sparsity losses and models include lifetime sparsity and spike-and-slab models.

Denoising autoencoder. A good model that has learned meaningful structure should "fill in the blanks": corrupt the input image with noise and train the autoencoder to reconstruct the clean image. There are many variants of this basic idea, and it is one of the most widely used simple autoencoder designs.

Layerwise pretraining. In the early days of deep learning (roughly 2006 to 2009), a dominant way to train deep networks was to train an autoencoder on the input, then another on its hidden states, and so on, and finally stack the encoders and fine-tune on the task (for example classifying `[DOG]`). Then we got much better at training deep networks end to end (ReLU, batch norm, better hyperparameter tuning) and largely stopped doing this, and autoencoders became less important.

Today autoencoders are much less used because there are better alternatives: VAEs and contrastive learning for representation learning, and GANs, VAEs, and autoregressive models for generation. They remain a viable option for "quick and dirty" representation learning that is very fast and can work OK. Their big problem is that sampling (generation) from an autoencoder is hard, which limits their uses; the variational autoencoder (Lecture 18) addresses this and is the most widely used autoencoder today.

17.4 Latent variable models

A mixture model $p(x) = \sum_z p(x \mid z)p(z)$, with z the mixture element, is a LATENT VARIABLE MODEL; a conditional version is $p(y \mid x) = \sum_z p(y \mid x, z)p(z)$, as in the multimodal policies of Lecture 14, where a network outputs $w_1, \mu_1, \Sigma_1, \ldots, w_N, \mu_N, \Sigma_N$. In general,

$$p(x) = \int p(x \mid z)p(z)\, dz,$$

where both $p(z)$ (for example a Gaussian) and $p(x \mid z)$ (for example a conditional Gaussian $p(x \mid z) = \mathcal{N}(\mu_{\text{nn}}(z), \sigma_{\text{nn}}(z))$ with a neural network computing its parameters) are "easy" distributions, yet $p(x)$ can be very complex (Figure 35).

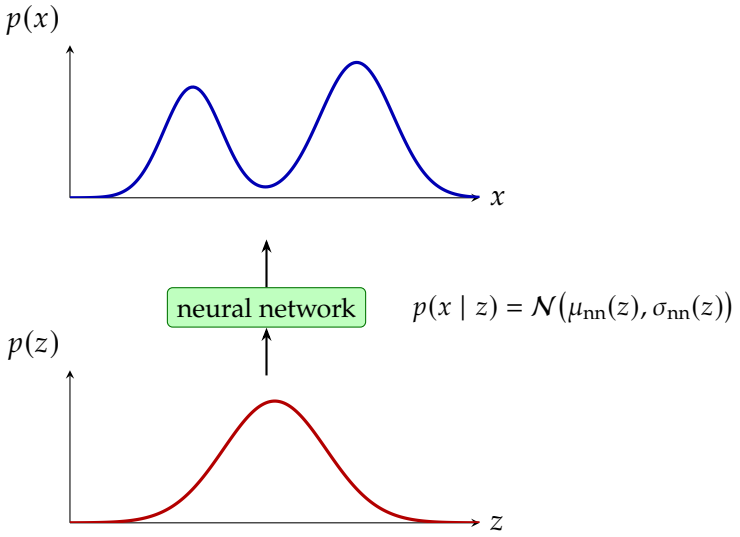


Figure 35: A latent variable model turns an easy distribution $p(z)$ into a complex $p(x)$. Schematic.

How do we train latent variable models? The model is $p_\theta(x)$, the data $\mathcal{D} = \{x_1, \ldots, x_N\}$, and the maximum likelihood fit is

$$\theta \leftarrow \arg\max_\theta \frac{1}{N} \sum_i \log p_\theta(x_i) = \arg\max_\theta \frac{1}{N} \sum_i \log\Big(\int p_\theta(x_i \mid z) p(z)\, dz \Big),$$

which is completely intractable. An alternative is the *expected* log-likelihood,

$$\theta \leftarrow \arg\max_\theta \frac{1}{N} \sum_i E_{z \sim p(z \mid x_i)} \big[\log p_\theta(x_i, z) \big].$$

The intuition is to "guess" the most likely z given x_i and pretend it is the right one; since there are many possible values of z, use the distribution $p(z \mid x_i)$. But how do we calculate $p(z \mid x_i)$? This is called PROBABILISTIC INFERENCE.

17.5 Latent variable models in deep learning

A latent variable deep generative model is (usually) just a model that turns random numbers into valid samples, such as images. There are many such models: VAEs, GANs, normalizing flows, and others. Generation has two steps:

1. sample $z \sim p(z)$, for example $\mathcal{N}(0, \mathbf{I})$: "generate a vector of random numbers";

2. sample $x \sim p(x \mid z)$: "turn that vector of random numbers into an image."

The prior $p(z)$ is easy, just Gaussian random numbers. The decoder $p_\theta(x \mid z)$ is a little more complex. If pixels are continuous-valued, $p_\theta(x \mid z) = \mathcal{N}(\mu_\theta(z); \sigma_\theta(z))$, where the mean is a neural network function and the variance is optionally one too. The easy choice is a constant σ, either learned (independent of z) or chosen manually (for example 1), which reduces to an MSE loss. If pixels are discrete-valued, we could use a 256-way softmax as in PixelRNN or PixelCNN, which works very well but is a little slow; other choices, especially common for the best performing models, are the discretized logistic and binary cross-entropy. For the architecture of $p_\theta(x \mid z)$, the easy choice is a big fully connected network (linear layers and ReLU), which works well for tiny images (such as MNIST) or non-image data; the better choice uses transpose convolutions, like the upsampling half of a segmentation network.

There are three basic ways to train such a model:

1. perform inference to figure out $p(z \mid x_i)$ for each training image x_i, then minimize the *expected* negative log-likelihood $E_{p(z|x_i)}[-\log p(x_i \mid z)]$: sample $z \sim p(z \mid x_i)$ and reduce $-\log p(x_i \mid z)$ with SGD. This leads to *variational autoencoders*;
2. use an *invertible* mapping from z to x: *normalizing flows*;
3. match the distribution $E_{z \sim p(z)}[p_\theta(x \mid z)]$ to the data distribution: *generative adversarial networks*.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 17 (Spring 2021).

Lecture 18: Variational Autoencoders and Invertible Models

18.1 The variational approximation

A latent variable model $p(x) = \int p(x \mid z)p(z)\,dz$ combines easy distributions $p(z)$ and $p(x \mid z)$ into a complex $p(x)$ (Lecture 17). Training it by the expected log-likelihood,

$$\theta \leftarrow \arg\max_\theta \frac{1}{N}\sum_i E_{z\sim p(z|x_i)}[\log p_\theta(x_i, z)],$$

requires probabilistic inference: how do we calculate $p(z \mid x_i)$? What if we approximate it with a simple distribution $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$? Then we can bound $\log p(x_i)$ using Jensen's inequality, $\log E[y] \geq E[\log y]$:

$$\begin{aligned}\log p(x_i) &= \log \int_z p(x_i \mid z)p(z) = \log \int_z p(x_i \mid z)p(z)\frac{q_i(z)}{q_i(z)} = \log E_{z\sim q_i(z)}\Big[\frac{p(x_i \mid z)p(z)}{q_i(z)}\Big] \\ &\geq E_{z\sim q_i(z)}\Big[\log \frac{p(x_i \mid z)p(z)}{q_i(z)}\Big] = E_{z\sim q_i(z)}\big[\log p(x_i \mid z) + \log p(z)\big] + \mathcal{H}(q_i).\end{aligned}$$

Maximizing the right-hand side pushes up $\log p(x_i)$.

Definition 18.1 [Entropy and KL divergence]. The entropy of p is

$$\mathcal{H}(p) = -E_{x\sim p(x)}[\log p(x)] = -\int_x p(x)\log p(x)\,dx.$$

The KL divergence between q and p is

$$D_{\mathrm{KL}}(q\|p) = E_{x\sim q(x)}\Big[\log \frac{q(x)}{p(x)}\Big] = E_{x\sim q(x)}[\log q(x)] - E_{x\sim q(x)}[\log p(x)] = -E_{x\sim q(x)}[\log p(x)] - \mathcal{H}(q).$$

Entropy measures how random a random variable is, or equivalently how large its log probability is in expectation under itself: a wide distribution has high entropy and a narrow one low entropy. (For a Bernoulli variable, $\mathcal{H}$ is maximal at probability $\frac{1}{2}$.) The KL divergence measures how different two distributions are, or how small the expected log probability of one distribution is under another, minus entropy. The entropy term matters: in the bound $E_{z\sim q_i(z)}[\log p(x_i \mid z) + \log p(z)] + \mathcal{H}(q_i)$, the first part is maximized by putting q_i at the peak of $p(x_i, z)$, and the entropy then makes q_i as wide as possible, so that it covers $p(x_i, z)$.

Proposition 18.2 [Evidence lower bound]. *Let* $\mathcal{L}_i(p, q_i) = E_{z\sim q_i(z)}[\log p(x_i \mid z) + \log p(z)] + \mathcal{H}(q_i)$. *Then*

$$\log p(x_i) = D_{\mathrm{KL}}\big(q_i(z)\,\|\,p(z \mid x_i)\big) + \mathcal{L}_i(p, q_i), \qquad \textit{so} \qquad \log p(x_i) \geq \mathcal{L}_i(p, q_i).$$

Proof. Using $p(z \mid x_i) = p(x_i, z)/p(x_i)$,

$$\begin{aligned} D_{\mathrm{KL}}\big(q_i(z) \| p(z \mid x_i)\big) &= E_{z\sim q_i(z)}\left[\log \frac{q_i(z)}{p(z \mid x_i)}\right] = E_{z\sim q_i(z)}\left[\log \frac{q_i(z)p(x_i)}{p(x_i, z)}\right] \\ &= -E_{z\sim q_i(z)}[\log p(x_i \mid z) + \log p(z)] + E_{z\sim q_i(z)}[\log q_i(z)] + \log p(x_i) \\ &= -E_{z\sim q_i(z)}[\log p(x_i \mid z) + \log p(z)] - \mathcal{H}(q_i) + \log p(x_i) \\ &= -\mathcal{L}_i(p, q_i) + \log p(x_i). \end{aligned}$$

The bound follows since the KL divergence is non-negative. □

What makes a good $q_i(z)$? Intuitively, $q_i(z)$ should approximate $p(z \mid x_i)$, in the sense of KL divergence. Since $\log p(x_i)$ is independent of q_i, maximizing $\mathcal{L}_i(p, q_i)$ with respect to q_i minimizes the KL divergence. So instead of $\theta \leftarrow \arg\max_\theta \frac{1}{N}\sum_i \log p_\theta(x_i)$, we optimize

$$\theta \leftarrow \arg\max_\theta \frac{1}{N}\sum_i \mathcal{L}_i(p, q_i), \qquad \mathcal{L}_i(p, q_i) = E_{z\sim q_i(z)}[\log p_\theta(x_i \mid z) + \log p(z)] + \mathcal{H}(q_i).$$

For each x_i (or mini-batch): sample $z \sim q_i(z)$ and compute $\nabla_\theta \mathcal{L}_i(p, q_i) \approx \nabla_\theta \log p_\theta(x_i \mid z)$; set $\theta \leftarrow \theta + \alpha\nabla_\theta \mathcal{L}_i(p, q_i)$; and update q_i to maximize $\mathcal{L}_i(p, q_i)$, for $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$ by gradient ascent on μ_i, σ_i.

18.2 Amortized variational inference

The problem is the number of parameters, $|\theta| + (|\mu_i| + |\sigma_i|) \times N$: one q_i per datapoint. Since $q_i(z)$ should approximate $p(z \mid x_i)$, learn a *network* $q_i(z) = q(z \mid x_i) \approx p(z \mid x_i)$, namely $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$, alongside the decoder $p_\theta(x \mid z)$. This is AMORTIZED VARIATIONAL INFERENCE, with the bound

$$\log p(x_i) \geq \mathcal{L}\big(p_\theta(x_i \mid z), q_\phi(z \mid x_i)\big) = E_{z\sim q_\phi(z|x_i)}[\log p_\theta(x_i \mid z) + \log p(z)] + \mathcal{H}(q_\phi(z \mid x_i)).$$

For each x_i (or mini-batch): sample $z \sim q_\phi(z \mid x_i)$, set $\nabla_\theta \mathcal{L} \approx \nabla_\theta \log p_\theta(x_i \mid z)$, and update $\theta \leftarrow \theta + \alpha\nabla_\theta \mathcal{L}$ and $\phi \leftarrow \phi + \alpha\nabla_\phi \mathcal{L}$.

How do we calculate $\nabla_\phi \mathcal{L}$? The entropy of a Gaussian has a closed form. The remaining term is

$$J(\phi) = E_{z\sim q_\phi(z|x_i)}[r(x_i, z)], \qquad r(x_i, z) = \log p_\theta(x_i \mid z) + \log p(z),$$

for which we could just use the policy gradient, $\nabla J(\phi) \approx \frac{1}{M}\sum_j \nabla_\phi \log q_\phi(z_j \mid x_i) r(x_i, z_j)$. What is wrong with this gradient? It has high variance.

18.3 The reparameterization trick

Is there a better way? Write the sample as $z = \mu_\phi(x) + \epsilon\sigma_\phi(x)$ with $\epsilon \sim \mathcal{N}(0,1)$, which is independent of ϕ. Then

$$J(\phi) = E_{z\sim q_\phi(z|x_i)}[r(x_i, z)] = E_{\epsilon\sim\mathcal{N}(0,1)}\big[r(x_i, \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i))\big],$$

and to estimate $\nabla_\phi J(\phi)$, sample $\epsilon_1, \dots, \epsilon_M$ from $\mathcal{N}(0,1)$ (a single sample works well) and use

$$\nabla_\phi J(\phi) \approx \frac{1}{M}\sum_j \nabla_\phi r\big(x_i, \mu_\phi(x_i) + \epsilon_j\sigma_\phi(x_i)\big).$$

This is the REPARAMETERIZATION TRICK; most autodiff software (for example TensorFlow) computes this gradient for us.

Another way to look at it:

$$\begin{aligned}
\mathcal{L}_i &= E_{z\sim q_\phi(z|x_i)}[\log p_\theta(x_i \mid z) + \log p(z)] + \mathcal{H}(q_\phi(z \mid x_i)) \\
&= E_{z\sim q_\phi(z|x_i)}[\log p_\theta(x_i \mid z)] + \underbrace{E_{z\sim q_\phi(z|x_i)}[\log p(z)] + \mathcal{H}(q_\phi(z \mid x_i))}_{-D_{\mathrm{KL}}(q_\phi(z|x_i)\|p(z))} \\
&= E_{\epsilon\sim\mathcal{N}(0,1)}\big[\log p_\theta(x_i \mid \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i))\big] - D_{\mathrm{KL}}\big(q_\phi(z \mid x_i)\|p(z)\big) \\
&\approx \log p_\theta\big(x_i \mid \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i)\big) - D_{\mathrm{KL}}\big(q_\phi(z \mid x_i)\|p(z)\big).
\end{aligned}$$

The KL term often has a convenient analytical form, for example between Gaussians.

The policy gradient can handle both discrete and continuous latent variables, but has high variance and requires multiple samples and small learning rates. The reparameterization trick works only for continuous latent variables, but is very simple to implement and has low variance.

18.4 The variational autoencoder

Definition 18.3 [VARIATIONAL AUTOENCODER]. A VARIATIONAL AUTOENCODER (VAE) has an encoder $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$ and a decoder $p_\theta(x \mid z) = \mathcal{N}(\mu_\theta(z), \sigma_\theta(z))$, trained by

$$\max_{\theta,\phi} \frac{1}{N}\sum_i \log p_\theta\big(x_i \mid \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i)\big) - D_{\mathrm{KL}}\big(q_\phi(z \mid x_i)\|p(z)\big)$$

(Figure 36).

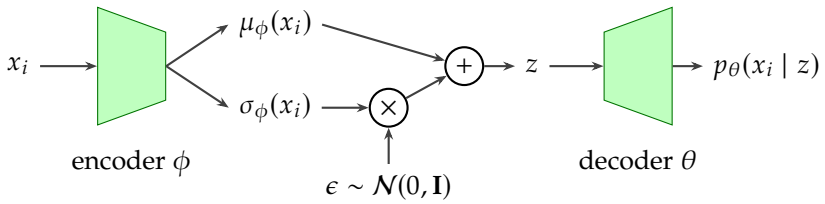


Figure 36: The variational autoencoder with the reparameterization trick.

To sample, draw $z \sim p(z)$ and then $x \sim p(x \mid z)$, since $p(x) = \int p(x \mid z)p(z)\,dz$. Why does this work? In $\mathcal{L}_i = E_{z\sim q_\phi(z|x_i)}[\log p_\theta(x_i \mid z)] - D_{\mathrm{KL}}(q_\phi(z \mid x_i)\|p(z))$, the first term trains the decoder to reconstruct x_i from codes drawn from the encoder, and the KL term keeps those codes close to the prior, so codes drawn from $p(z)$ at sampling time look like codes the decoder was trained on. Trained on faces, sampled codes decode to plausible faces.

Conditional models. To generate y_i conditioned on x_i, use

$$\mathcal{L}_i = E_{z\sim q_\phi(z|x_i,y_i)}\big[\log p_\theta(y_i \mid x_i, z) + \log p(z \mid x_i)\big] + \mathcal{H}\big(q_\phi(z \mid x_i, y_i)\big),$$

just like before, but now generating y_i, with everything conditioned on x_i; the prior $p(z)$ can optionally depend on x. The encoder takes (x_i, y_i) and outputs $\mu_\phi(x_i, y_i)$, $\sigma_\phi(x_i, y_i)$; the decoder takes x_i and z and outputs $p_\theta(y_i \mid x_i, z)$. At test time, sample $z \sim p(z \mid x_i)$ (for example $\mathcal{N}(0, \mathbf{I})$) and $y \sim p(y \mid x_i, z)$. For example, $p(y \mid x, z)$ can be a multimodal driving policy given a camera image.

VAEs with convolutions. For example, a $64 \times 64 \times 3$ image passes through $5 \times 5 \times 32$, $3 \times 3 \times 64$, and $3 \times 3 \times 128$ convolutions with stride 2 ($30 \times 30 \times 32$, $14 \times 14 \times 64$, $6 \times 6 \times 128$), is flattened to 1024 units, and produces μ_z and σ_z of dimension 256; then $z = \mu_z + \epsilon\sigma_z$ with $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ is decoded by a fully connected layer to 1024 units and transpose convolutions into an (independent) mean and variance for each pixel. A fully convolutional VAE is possible, but be careful with the latent codes: the prior $p(z) = \mathcal{N}(0, \mathbf{I})$ implies that all dimensions of z are independent.

18.5 VAEs in practice

A common issue is that VAEs, especially conditional ones, are very tempted to ignore the latent codes or to generate poor samples. The quantity to control carefully is $D_{\mathrm{KL}}(q_\phi(z \mid x)\|p(z))$.

- *The latent code is ignored* ($p_\theta(x \mid z) \to p(x)$): the KL is too low and there is no information in z. Reconstructions ($z \sim q_\phi(z \mid x)$, $x \sim p_\theta(x \mid z)$) look like a blurry "average" image.
- *The latent code is not compressed* ($q_\phi(z \mid x)$ very far from $p(z)$): the KL is too high and there is too much information in z. Samples ($z \sim p(z)$, $x \sim p_\theta(x \mid z)$) are garbage.

The fix multiplies the regularizer by β to adjust its strength,

$$\max_{\theta,\phi} \frac{1}{N} \sum_i \log p_\theta\big(x_i \mid \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i)\big) - \beta D_{\mathrm{KL}}\big(q_\phi(z \mid x_i) \| p(z)\big).$$

Adjust β manually to get good reconstructions *and* good samples, or schedule it: start low (to get the VAE to use z to reconstruct) and end high (to get the samples to be good).

18.6 Invertible models and normalizing flows

A simpler kind of model replaces $p(x \mid z) = \mathcal{N}(\mu_{\mathrm{nn}}(z), \sigma_{\mathrm{nn}}(z))$ by a *deterministic* function $x = f(z)$. Why is this such a big deal? By the CHANGE OF VARIABLES FORMULA,

$$p(x) = p(z)\left|\det\Big(\frac{df(z)}{dz}\Big)\right|^{-1}, \qquad z = f^{-1}(x),$$

where the determinant corrects for the change in local density due to f. The basic idea is to learn an *invertible* mapping from z to x whose determinant is easy to compute. There is no more need for lower bounds: we get exact probabilities and likelihoods. The training objective $\max_\theta \frac{1}{N}\sum_{i=1}^N \log p(x_i)$ becomes

$$\max_\theta \frac{1}{N} \sum_{i=1}^N \log p\big(f^{-1}(x_i)\big) - \log\left|\det\Big(\frac{df(z)}{dz}\Big)\right|,$$

and we choose a special architecture that makes f^{-1} and the determinant easy to compute.

Definition 18.4 [NORMALIZING FLOW]. A NORMALIZING FLOW model consists of multiple layers of invertible transformations, $f(z) = f_4(f_3(f_2(f_1(z))))$.

If each layer is invertible, the whole model is invertible; invertible layers often have very convenient determinants; and the log-determinant of the whole model is just the sum of the log-determinants of the layers. The goal is to design an invertible layer and compose many of them into a fully invertible network.

NICE (Dinh et al., *Non-linear Independent Components Estimation*, 2014). An ordinary layer $x = \mathrm{ReLU}(Wz + b)$ is not invertible. Instead, force part of the layer to keep all the information, so that anything changed by the non-linear transformation can be recovered: split z into $z_{1:d}$ and $z_{d+1:n}$ and set

$$x_{1:d} = z_{1:d}, \qquad x_{d+1:n} = z_{d+1:n} + g_\theta(z_{1:d}),$$

where g_θ is a neural network, for example $g_\theta(a) = \mathrm{ReLU}(Wa + b)$. (This describes one layer; in reality there are many.) Given x, we can recover z: first $z_{1:d} = x_{1:d}$, then $g_\theta(z_{1:d})$, then

$z_{d+1:n} = x_{d+1:n} - g_\theta(z_{1:d})$. So the layer is invertible. Its Jacobian is block triangular,

$$\frac{df(z)}{dz} = \begin{bmatrix} \frac{dx_{1:d}}{dz_{1:d}} & \frac{dx_{1:d}}{dz_{d+1:n}} \\ \frac{dx_{d+1:n}}{dz_{1:d}} & \frac{dx_{d+1:n}}{dz_{d+1:n}} \end{bmatrix} = \begin{bmatrix} \mathbf{I} & 0 \\ \frac{dg_\theta}{dz_{1:d}} & \mathbf{I} \end{bmatrix}, \qquad \left|\det\left(\frac{df(z)}{dz}\right)\right| = 1.$$

This is very simple and convenient, but representationally a bit limiting. NICE samples on MNIST, faces (TFD), SVHN, and CIFAR-10 are recognizable but blurry.

Real-NVP (Dinh et al., *Density estimation using Real-NVP*, 2016), a non-volume-preserving transformation, adds an elementwise scaling (Figure 37):

$$x_{1:d} = z_{1:d}, \qquad x_{d+1:n} = z_{d+1:n} \odot \exp\big(h_\theta(z_{1:d})\big) + g_\theta(z_{1:d}).$$

The inverse is derived the same way: recover $z_{1:d} = x_{1:d}$, then $g_\theta(z_{1:d})$ and $h_\theta(z_{1:d})$, then $z_{d+1:n} = \big(x_{d+1:n} - g_\theta(z_{1:d})\big)/\exp\big(h_\theta(z_{1:d})\big)$. The Jacobian is

$$\frac{df(z)}{dz} = \begin{bmatrix} \mathbf{I} & 0 \\ \frac{dx_{d+1:n}}{dz_{1:d}} & \operatorname{diag}\big(\exp(h_\theta(z_{1:d}))\big) \end{bmatrix}, \qquad \left|\det\left(\frac{df(z)}{dz}\right)\right| = \prod_{i=d+1}^{n} \exp\big(h_\theta(z_{1:d})_i\big).$$

This is significantly more expressive, and Real-NVP samples of scenes, faces, and bedrooms are much sharper.

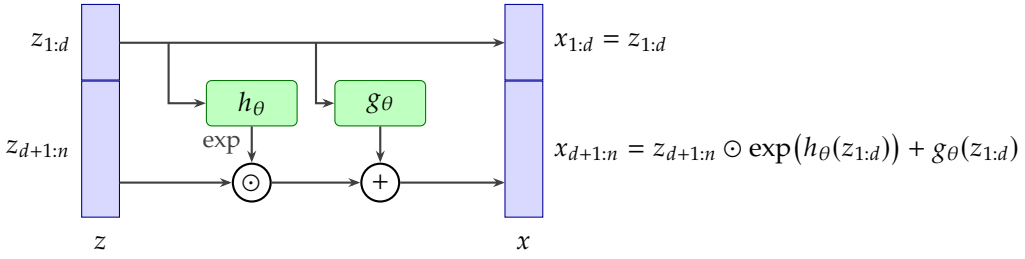


Figure 37: A Real-NVP coupling layer; NICE is the case $h_\theta = 0$.

Invertible models give exact probabilities and likelihoods with no need for lower bounds, and are (perhaps) conceptually simpler; but they require a special architecture, and z must have the same dimensionality as x.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 18 (Spring 2021).

Lecture 19: Generative Adversarial Networks

19.1 Matching distributions at the population level

A latent variable model generates by sampling $z \sim p(z)$, "a vector of random numbers," and then $x \sim p_\theta(x \mid z)$, "turning that vector into an image." Instead of training an encoder, can we train the whole model to generate images that look similar to real images *at the population level*? No two faces are the same, yet a set of real faces and a set of generated faces can look alike as populations, so that it is impossible to tell which set is real.

The idea is to train a network to guess which images are real and which are fake: a classifier $D(x)$, "is this a real image?", trained on a "True" set of real photographs and a "False" set of generated images. This model can then serve as a loss function for the generator:

1. get a "True" dataset $\mathcal{D}_T = \{(x_i)\}$;
2. get a generator $G_\theta(z)$;
3. sample a "False" dataset $\mathcal{D}_F$: $z \sim p(z)$, $x = G(z)$;
4. train a discriminator $D_\phi(x) = p_\phi(y \mid x)$ using $\mathcal{D}_T$ and $\mathcal{D}_F$;
5. use $-\log D(x)$ as the "loss" to train $G(z)$.

This almost works, but has two major problems: how to get a generator in step 2, and the fact that if step 5 is done only once, it is too easy for $G(z)$ to "fool" $D(x)$. The fix is to start the generator from a random initialization and alternate: repeat steps 3 to 5, updating D_ϕ with one SGD step and then G with one SGD step. (In reality there are various losses, but the idea is similar.) This is a generative adversarial network (GAN).

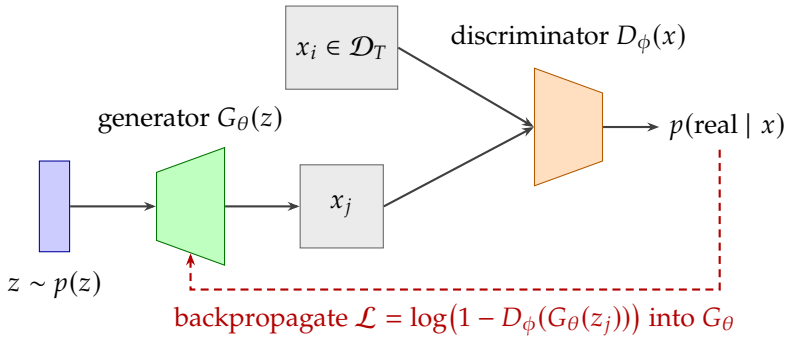


Figure 38: A GAN: the discriminator classifies real and generated images, and its loss is backpropagated into the generator.

Why do GANs learn distributions? The generator wants to make $D(x) = 0.5$ for all generated x: the discriminator "can't tell if real or fake." To do this it must generate images that look realistic (obviously), and also generate *all* possible realistic images. If the training data contains cats and dogs but the generator produces very realistic dogs only, the discriminator learns that dogs are often fake: a real cat gets True = 1.0, while a real-looking dog gets only True = 0.25. The generator does better if it generates not only realistic pictures but all realistic pictures.

GANs scale from small models on MNIST, faces, and CIFAR-10 (Goodfellow et al., 2014) to 1024×1024 faces with progressive growing (Karras et al., 2017) and class-conditional 512×512 ImageNet samples (BigGAN, Brock et al., 2018); conditional GANs also translate images, for example labels to street scenes, day to night, or edges to photos (Isola et al., 2017).

19.2 The GAN game

The "classic" GAN is a two-player game:

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)}[\log D(x)] + E_{z \sim p(z)}[\log(1 - D(G(z)))],$$

estimated by $\frac{1}{N} \sum_{i=1}^{N} \log D(x_i)$ over $x_i \in \mathcal{D}_T$ and $\frac{1}{N} \sum_{j=1}^{N} \log(1 - D(x_j))$ over $x_j = G(z_j)$ with random numbers z_j. With parameters, $\min_\theta \max_\phi V(\theta, \phi) = E_{x \sim p_{\text{data}}(x)}[\log D_\phi(x)] + E_{z \sim p(z)}[\log(1 - D_\phi(G_\theta(z)))]$. Two details are needed, both pretty simple: how to make this work with stochastic gradient descent and ascent, and how to compute the gradients. Alternate

$$\phi \leftarrow \phi + \alpha \nabla_\phi V(\theta, \phi), \qquad \theta \leftarrow \theta - \alpha \nabla_\theta V(\theta, \phi),$$

where

$$\nabla_\phi V(\theta, \phi) \approx \nabla_\phi \Big(\frac{1}{N} \sum_{i=1}^{N} \log D_\phi(x_i) + \frac{1}{N} \sum_{j=1}^{N} \log(1 - D_\phi(x_j)) \Big)$$

is just a cross-entropy loss, and $\nabla_\theta V(\theta, \phi) \approx \nabla_\theta \big(\frac{1}{N} \sum_{j=1}^{N} \log(1 - D_\phi(G_\theta(z_j))) \big)$. For the latter, just backpropagate from the discriminator into the generator: with $x = G_\theta(z)$ and $\mathcal{L} = \log(1 - D_\phi(x))$, the chain rule gives $\frac{d\mathcal{L}}{d\theta} = \frac{dx}{d\theta} \frac{d\mathcal{L}}{dx}$.

19.3 What does the GAN optimize?

What can we say about $G(z)$ at convergence? The idea is to express $D(x)$ in closed form as a function of G.

Proposition 19.1 [Optimal discriminator]. *For a fixed generator with distribution p_G,*

$$D^\star_G(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

Proof. Write $p = p_{\text{data}}$ and $q = p_G$, so $\mathcal{D}_T = \{x_i \sim p(x)\}$ and $\mathcal{D}_F = \{x_j \sim q(x)\}$, and $D^\star = \arg\max_D E_p[\log D(x)] + E_q[\log(1 - D(x))]$. Setting the functional derivative to zero, $E_p\left[\frac{1}{D(x)}\right] - E_q\left[\frac{1}{1-D(x)}\right] = 0$, that is, $\sum_x \frac{p(x)}{D(x)} - \sum_x \frac{q(x)}{1-D(x)} = 0$. Plugging in $D^\star(x) = p(x)/(p(x)+q(x))$ gives $\sum_x \big(p(x) + q(x)\big) - \sum_x \big(p(x) + q(x)\big) = 0$, as required. □

Now the objective for G is entirely a function of G:

$$V(D_G^\star, G) = E_{p_{\text{data}}(x)}\big[\log p_{\text{data}}(x) - \log(p_{\text{data}}(x) + p_G(x))\big] + E_{p_G(x)}\big[\log p_G(x) - \log(p_{\text{data}}(x) + p_G(x))\big].$$

To make sense of these funny expressions, let $q(x) = \frac{p_{\text{data}}(x)+p_G(x)}{2}$; the $-\log 4$ accounts for the $\frac{1}{2}$ factors:

$$\begin{aligned} V(D_G^\star, G) &= \underbrace{E_{p_{\text{data}}(x)}[\log p_{\text{data}}(x) - \log q(x)]}_{D_{\text{KL}}(p_{\text{data}} \| q)} + \underbrace{E_{p_G(x)}[\log p_G(x) - \log q(x)]}_{D_{\text{KL}}(p_G \| q)} - \log 4 \\ &= 2D_{\text{JS}}(p_{\text{data}} \| p_G) - \log 4, \end{aligned}$$

where $D_{\text{JS}}(p\|p') = \frac{1}{2}D_{\text{KL}}(p\|q) + \frac{1}{2}D_{\text{KL}}(p'\|q)$, with q their average, is the Jensen-Shannon divergence. It goes to zero if the distributions match, and it is symmetric, unlike the KL divergence. So the GAN really is trying to match the data distribution. This describes the objective at the Nash equilibrium; it does not guarantee that training actually finds the equilibrium.

A practical aside. The generator loss should be $E_{z\sim p(z)}[\log(1 - D_\phi(G_\theta(z)))]$, "minimize the probability that the image is fake." In practice we often use $E_{z\sim p(z)}[-\log D_\phi(G_\theta(z))]$, "maximize the probability that the image is real" (though there are other variants). The reason is the shape of the two curves: $\log(1 - D)$ has a small gradient when the generator is bad (the probability of fake is high, $D \approx 0$), exactly when it most needs a signal, while $-\log D$ has a small gradient only when the generator is good (the probability of real is high).

19.4 GAN architectures and variants

A simple GAN might map $z \in \mathbb{R}^{64}$ through fully connected layers of 512 units to an image, and the discriminator back through layers of 512 units to [True/False]; a convolutional GAN maps $z \in \mathbb{R}^{256}$ to 1024 units and then through transpose convolutions to an image, with a convolutional discriminator. Real architectures such as BigGAN use residual blocks with upsampling in G and downsampling in D.

Conditional GANs (Mirza and Osindero, 2014) append the conditioning, such as a class label y, to the inputs of both the generator, $G(z \mid y)$, and the discriminator, $D(x \mid y)$; for example, they generate MNIST digits of a chosen class.

CycleGAN (Zhu et al., 2017) translates between image domains (Monet and photos, zebras and horses, summer and winter) without paired data: we do not know which images "go together." It has two conditional generators, G turning X into Y (horse into zebra) and F turning Y into X, and two discriminators, D_X ("is it a realistic horse?") and D_Y ("is it a realistic zebra?"). But why should the translated zebra look anything like the original horse? If I turn a horse into a zebra and then back into a horse, I should get the same horse; this is the CYCLE-CONSISTENCY LOSS

$$\mathcal{L}_{\text{cyc}}(G, F) = E_{x \sim p_{\text{data}}(x)}\left[\|F(G(x)) - x\|_1\right] + E_{y \sim p_{\text{data}}(y)}\left[\|G(F(y)) - y\|_1\right].$$

19.5 Improved GAN training

Why is training GANs hard? Suppose the real data and the generated data are far apart (Figure 39). The discriminator easily separates them, and $D(x)$ is flat (near 0) around all generated samples. What is the generator gradient there? The values of $\log(1 - D_\phi(G_\theta(z)))$ are all basically the same, so the gradient is almost zero. A better discriminator would slope smoothly from the data toward the generated samples; similarly, wider (noisier) versions of p_{data} and p_G would overlap and give a useful gradient.

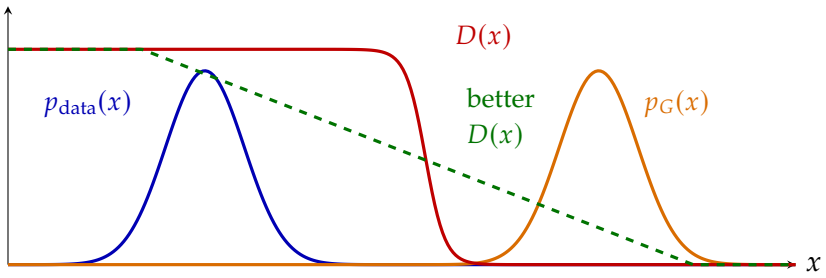


Figure 39: A saturated discriminator gives the generator almost no gradient; a smoother discriminator points the generated samples toward the data. Schematic.

Improved GAN techniques (in no particular order):

- *least-squares GAN* (LSGAN): the discriminator outputs a real-valued number;
- *Wasserstein GAN* (WGAN): the discriminator is constrained to be Lipschitz-continuous;
- *gradient penalty*: the discriminator is constrained to be continuous even harder;
- *spectral norm*: the discriminator is *really* constrained to be continuous;
- *instance noise*: add noise to the data and generated samples.

Gradient penalty, spectral norm, and instance noise are pretty good choices today (Mescheder et al., *Which Training Methods for GANs do actually Converge?*). The theory behind these tricks is quite complex, but the methods are very simple in practice, and such GANs are much easier to train.

19.6 Wasserstein GAN

The high-level intuition is that the JS divergence used by the classic GAN does not account for "distance." In $V(D_G^\star, G)$, when $p_{\text{data}}(x) \approx 0$ wherever $p_G(x) \neq 0$ and vice versa, the value is the same whether the distributions are "really far apart" or "really close together": the JSD (and the KL divergence, and most other divergences) are almost identical in the two cases. This is why GANs are so hard to train.

A better metric considers how far apart, in Euclidean space, all the "bits" of probability are. More precisely, OPTIMAL TRANSPORT (the EARTH MOVER'S DISTANCE) asks how far one has to go to "transport" one distribution into another:

$$W(p_{\text{data}}, p_G) = \inf_{\gamma} E_{(x,y)\sim\gamma(x,y)}\big[\|x - y\|\big],$$

where $\gamma(x, y)$ is a distribution over x, y with marginals $\gamma_X(x) = p_{\text{data}}(x)$ and $\gamma_Y(y) = p_G(y)$. The intuition is that correlations between x and y in $\gamma(x, y)$ indicate which x should be transported to which y. (This definition is not needed to implement WGAN.)

We could learn γ directly, but that is very hard: $p_{\text{data}}(x)$ is unknown and $\gamma(x, y)$ is really complex. A theorem based on Kantorovich-Rubinstein duality (proved with tools from duality, as in Lagrangian duality in convex optimization) gives

$$W(p_{\text{data}}, p_G) = \sup_{\|f\|_L \leq 1} E_{p_{\text{data}}}[f(x)] - E_{p_G(x)}[f(x)],$$

the supremum over the set of all 1-Lipschitz scalar functions, $|f(x) - f(y)| \leq |x - y|$: functions with bounded slope, which are not too steep. This is a difference of expectations under $p_G(x)$ and $p_{\text{data}}(x)$, just like a regular GAN.

How do we enforce the constraint? If f is a neural network with ReLU activations, we can bound the weights. For one layer, $f_\theta(x) = \text{ReLU}(W_1 x + b_1)$ with $W_{1,i,j} \in [-0.01, 0.01]$, the slope cannot exceed $0.01 \times D$. For two layers, $f_\theta(x) = W_2\text{ReLU}(W_1 x + b_1) + b_2$ with $W_{\ell,i,j} \in [-0.01, 0.01]$, the slope is bounded, but by more than 0.01. This does not guarantee 1-Lipschitz unless the bounds are picked very carefully, but it does guarantee K-Lipschitz for some finite K, which suffices up to scale.

Algorithm 19.1: Wasserstein GAN (Arjovsky et al., 2017)

repeat
  update f_θ using the gradient of $E_{x\sim p_{\text{data}}}[f_\theta(x)] - E_{z\sim p(z)}[f_\theta(G(z))]$
  clip all weight matrices in θ to $[-c, c]$
  update the generator to *maximize* $E_{z\sim p(z)}[f_\theta(G(z))]$
until done

The discriminator (the *critic*) outputs real numbers, not a true/false probability, for example $f_\theta(x) = W_2\text{ReLU}(W_1 x + b_1) + b_2$, and uses weight clipping. Where a regular GAN's discriminator saturates and its gradients vanish, the WGAN critic gives linear gradients. Its authors themselves

called weight clipping "a clearly terrible way to enforce a Lipschitz constraint": with a large clipping parameter, weights take a long time to reach their limit, and with a small one, gradients vanish when there are many layers or no batch normalization.

19.7 Better discriminator regularization

Gradient penalty (Gulrajani et al., *Improved Training of Wasserstein GANs*, 2017). Want a bounded slope? Penalize it directly: update f_θ using the gradient of

$$E_{x\sim p_{\text{data}}}\left[f_\theta(x) - \lambda(\|\nabla_x f_\theta(x)\|_2 - 1)^2\right] - E_{z\sim p(z)}\left[f_\theta(G(z))\right],$$

which makes the norm of the gradient close to 1.

Spectral norm (Miyato et al., 2018). The idea is to bound the Lipschitz constant in terms of the singular values of each W_ℓ. For a network $f(x) = f_3 \circ f_2 \circ f_1(x)$ of layers (linear, convolution, ReLU, and so on),

$$\|f\|_{\text{Lip}} = \|f_3 \circ f_2 \circ f_1\|_{\text{Lip}} \le \|f_3\|_{\text{Lip}} \cdot \|f_2\|_{\text{Lip}} \cdot \|f_1\|_{\text{Lip}}$$

(for intuition, imagine these are linear functions, whose slopes multiply). $\text{ReLU}(x) = \max(0, x)$ has maximum slope 1. The maximum slope of $Wx + b$ is the SPECTRAL NORM, the largest singular value of W:

$$\sigma(W) = \max_{h:\, h\neq 0} \frac{\|Wh\|}{\|h\|} = \max_{\|h\|\le 1} \|Wh\|.$$

The method is to force $W_\ell \leftarrow W_\ell/\sigma(W_\ell)$ after each gradient step (the paper shows how to do this efficiently).

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 19 (Spring 2021).

Lecture 20: Adversarial Examples

20.1 Do deep nets generalize?

What a strange question: on ImageNet, the error of the best networks fell from 28.2% (2010) to 3.57% (ResNet, 2015), below human performance of about 5%. But what kinds of mistakes do they make? Sometimes even the mistakes make sense: a bee on a hummingbird feeder is called a hummingbird, and an academic gown in front of a gate is called a mosque. Sometimes they do not: a model that classifies a husky as a wolf turns out, when its prediction is explained, to be looking at the snow in the background (Ribeiro et al., "Why Should I Trust You?", 2016).

A metaphor for machine learning is *Clever Hans*, the horse that appeared to do arithmetic but was reading cues from its audience: the training/test paradigm goes wrong when everything "works as intended" but we still do not get what we want. That is not the network's fault but the fault of asking the wrong question.

Distribution shift. One source of trouble is that test inputs may come from a different distribution than training inputs, which is especially problematic if the training data has spurious correlations. A traffic sign dataset of tightly cropped signs differs from signs in a real street scene. More realistic examples:

- medical imaging: different hospitals have different machines; even worse, different hospitals have different positive rates (some get more sick patients), which induces a machine-label correlation;
- selection biases: center crops, canonical poses, and so on;
- feedback: using the system causes users to change their behavior, which changes the input distribution (the classic example is spam classification).

Calibration. A model is CALIBRATED if its predicted probabilities reflect the actual frequencies of the predicted events. Data are generated as $x \sim p(x)$ (someone takes a photo) and $y \sim p(y \mid x)$ (a person labels it). An output of husky 0.3, wolf 0.7 should mean that 7 times out of 10 a person would say this is a wolf. On an out-of-distribution input, such as a road sign, a calibrated model would say husky 0.5, wolf 0.5: "I don't know what this is." Does this happen? Usually not: such models typically give confident but wrong predictions on out-of-distribution inputs (though not always), since nothing in training constrains their outputs there. Are in-distribution predictions calibrated? Usually not either, but there are many methods for improving calibration.

20.2 Adversarial examples

Adversarial examples are a particularly vivid illustration of how learned models may or may not generalize correctly. A panda image classified as "panda" with 57.7% confidence, plus 0.007 times a special pattern (not random noise, but a pattern designed to "fool" the model, itself classified as "nematode" with 8.2% confidence), is classified as "gibbon" with 99.3% confidence. Very special patterns, almost imperceptible to people, can change a model's classification drastically. The direct issue is that this is a potential way to "attack" learned classifiers; the bigger issue is that it implies strange things about generalization.

Some facts about adversarial examples:

- it is not just pandas and gibbons: with enough effort, basically anything can be turned into anything else;
- they are not easy to defend against: obvious fixes help, but nothing provides a bulletproof defense (that we know of);
- they transfer across networks: the same adversarial example can fool both AlexNet and ResNet;
- they work in the real world, not just as precise pixel patterns: an unaltered photo of a physically modified stop sign is classified as a "speed limit 45" sign;
- they are not specific to artificial neural networks: virtually all learned models are susceptible, including the brain, which is a type of learned model.

Even a linear model (logistic regression) on MNIST can be pushed to classify images of a 9 as "0" or "1" with 90% confidence, so adversarial examples appear to be a general phenomenon for most learned models, and all high-capacity models that we know of.

20.3 Overfitting or linearity?

The overfitting hypothesis: neural networks have a huge number of parameters and tend to overfit, making it easy to find inputs with crazy outputs; in the mental picture, adversarial examples are small pockets of the wrong class carved out by a wiggly decision boundary. The implication would be to stop using neural networks. Most evidence suggests this hypothesis is false.

- If it were true, different models would have very different adversarial examples (high variance). This is conclusively not the case.
- Low-capacity models, such as linear models, would not have the issue. They do.
- The decision boundaries around adversarial examples would be highly non-linear. This appears not to be true.

The linear models hypothesis: neural networks (and many other models) tend to be locally linear, so they extrapolate in counterintuitive ways when moving away from the data, the manifold of "realistic images." Away from the data has a counterintuitive meaning in high dimensions: a small change in every pixel moves far off the manifold. Why so linear? Rectified linear units are piecewise linear, and even carefully tuned sigmoids are operated in their nearly linear regime. This hypothesis is consistent with the transferability of adversarial examples and with the fact that reducing "overfitting" does not fix the problem. Two experiments support it.

1. Varying a car image along one direction, $x + \epsilon d$ for $\epsilon \in [-30, 30]$, the arguments to the softmax of all ten CIFAR-10 classes change almost linearly in ϵ, so far from $\epsilon = 0$ some class wins with huge confidence.
2. Varying images along two directions, an adversarial one (the FGSM direction) and a random one orthogonal to it, the predicted class shows not much variation in the orthogonal direction and a clean "shift" on one side of the adversarial direction, suggesting a mostly linear decision boundary.

Real-world adversarial examples include stop signs with subtle posters or camouflage stickers, which are classified as 45 mph speed limit signs at various distances and angles (Eykholt et al., 2018), and a 3D-printed turtle classified as a rifle from most viewpoints (Athalye et al., 2017). Humans have adversarial examples too: concentric circles of tilted squares look like intertwined spirals (Pinna and Gregory, 2002), and a cat image perturbed adversarially toward "dog" fools time-limited human observers as well as computer vision models (Elsayed et al., 2018).

What does this have to do with generalization? The linear hypothesis matters not just for adversarial examples but for how neural networks do and do not generalize. A model trained to classify cats versus dogs is not learning what cats and dogs look like; it is learning the patterns in the dataset, and from there it extrapolates in potentially weird ways. Put another way, adversarial examples are not bugs, they are features of the learning algorithm (literally: Ilyas et al., *Adversarial Examples Are Not Bugs, They Are Features*, 2019): networks pay attention to "adversarial directions" because doing so helps them get the right answer on the training data.

20.4 Adversarial attacks

Formally defining an adversarial attack is helpful for mathematicians but can hide important real-world considerations: real attackers do not care about our definitions (what is ϵ for an unaltered photo of a modified stop sign?). With a *relation* $R(x, x')$ between the original image x and the altered

image x', for example $R_\infty(x, x') = \|x - x'\|_\infty$ (each pixel changed by at most ϵ):

$$\begin{aligned} \text{attack:} \quad & x^\star \leftarrow \operatorname*{arg\,max}_{x' : R(x,x') \le \epsilon} \mathcal{L}_\theta(x', y), \\ \text{defense:} \quad & \theta^\star \leftarrow \operatorname*{arg\,min}_\theta \sum_{(x,y) \in \mathcal{D}} \max_{x' : R(x,x') \le \epsilon} \mathcal{L}_\theta(x', y). \end{aligned}$$

The attack picks an image x' close to x that *maximizes* the loss of our choice, for example $\mathcal{L}_\theta(x, y) = -\log p_\theta(y \mid x)$. The inner maximum of the defense is the ROBUST LOSS.

The fast gradient sign method (FGSM) is a very simple approximate method for the infinity-norm relation $R(x, x') = \|x - x'\|_\infty$. Make the "first-order" assumption $\mathcal{L}(x', y) \approx \mathcal{L}(x, y) + (x' - x)^T \nabla_x \mathcal{L}$, so the attack becomes $x^\star \leftarrow \arg\max_{x' : \|x - x'\|_\infty \le \epsilon} (x' - x)^T \nabla_x \mathcal{L}$. The optimal solution moves each dimension of x in the direction of $\nabla_x \mathcal{L}$ by ϵ:

$$x^\star = x + \epsilon \operatorname{sign}(\nabla_x \mathcal{L}).$$

Ordinarily we might expect this to make a very weak attack, but neural networks behave locally linearly. It works very well against standard (naive) networks; it can be defeated with simple defenses, but more advanced attacks are more resilient.

A more general formulation replaces the constraint by a penalty with a Lagrange multiplier λ (chosen heuristically, or optimized, for example by dual gradient descent):

$$x^\star \leftarrow \operatorname*{arg\,max}_{x'} \mathcal{L}_\theta(x', y) - \lambda R(x, x'),$$

or, with $\delta = x' - x$,

$$\delta^\star \leftarrow \operatorname*{arg\,max}_{\delta} \mathcal{L}_\theta(x + \delta, y) - \lambda \|\delta\|_\infty,$$

optimized to convergence, for example with Adam. A variety of penalties can be used, including perceptual losses. In general such attacks are very hard to defeat.

20.5 Black-box attacks

Adversarial examples often just transfer: an example constructed against one model fools another (Papernot et al., 2016). The success rate (%) of examples crafted on a source model at fooling a target model:

	Target					
Source	DNN	LR	SVM	DT	kNN	Ens.
DNN	38.27	23.02	64.32	79.31	8.36	20.72
LR	6.31	91.64	91.43	87.42	11.29	44.14
SVM	2.51	36.56	100.0	80.03	5.19	15.67
DT	0.82	12.22	8.85	89.29	3.31	5.11
kNN	11.75	42.89	82.16	82.95	41.65	31.92

Between five different DNNs the rates range from 49% to 86%. So we often do not need gradient access to the network we are attacking: we can construct the adversarial example with another network. A *zero-shot black-box attack* (Liu et al., 2017) attacks one white-box model, or better an ensemble of them, and transfers the resulting adversarial examples to the black-box system (Figure 40).

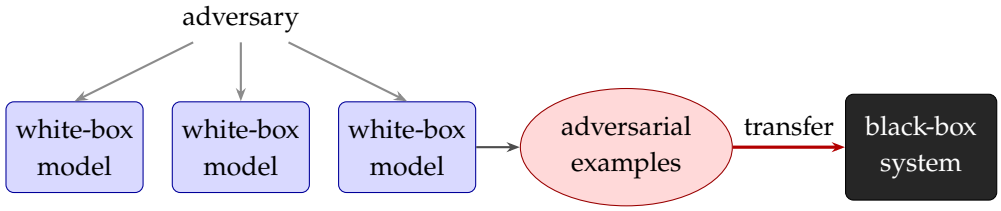


Figure 40: A zero-shot black-box attack: examples crafted against an ensemble of white-box models transfer to the black-box system.

It is also possible to estimate the gradient with a moderate number of queries to a model (for example on a web server) without access to its gradient. For FGSM all we need is the sign of the gradient. With finite differences, for each dimension i of x, get $v_i \leftarrow \mathcal{L}(x + 10^{-3}e_i, y)$, where e_i is the ith canonical vector and 10^{-3} a small number, and set $(\nabla_x \mathcal{L})_i \approx (v_i - \mathcal{L}(x, y))/10^{-3}$. There are fancy tricks to further reduce the number of queries.

20.6 Defenses

There are many methods for "robustifying" models against adversarial examples, all aimed at the robust loss. A simple recipe is ADVERSARIAL TRAINING:

1. sample a minibatch $\{(x_i, y_i)\}$ from the dataset $\mathcal{D}$;
2. for each x_i, compute an adversarial x_i', for example by FGSM, $x_i' \leftarrow x_i + \epsilon \operatorname{sign}(\nabla_{x_i} \mathcal{L}_\theta(x_i, y_i))$;

3. take an SGD step $\theta \leftarrow \theta - \alpha \sum_i \nabla_\theta \mathcal{L}_\theta(x'_i, y_i)$, usually also adding the gradient of the original loss $\nabla_\theta \mathcal{L}_\theta(x_i, y_i)$;

and repeat. This usually does not come for free: it increases robustness to adversarial attacks (a lower fooling rate) but decreases the overall test accuracy compared to a naive network. Defenses are a heavily studied topic, but a very hard one.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 20 (Spring 2021).

Lecture 21: Meta-Learning

21.1 What is meta-learning?

If you have already learned 100 tasks, can you figure out how to *learn* more efficiently? Then having multiple tasks is a huge advantage. Meta-learning is learning to learn; in practice it is very closely related to multi-task learning. There are many formulations: learning an optimizer (one that recognizes "I've seen this before" instead of oscillating like plain gradient descent), learning an RNN that ingests experience, and learning a representation.

Why is this a good idea? Deep learning works very well but requires large datasets. Often we have only a small amount of data for some specific task (say a particular computer vision problem), but lots of data of a similar type for other tasks (other object classification tasks). A meta-learner uses the plentiful prior tasks to meta-train a model that can learn a new task quickly from only a few examples: we then collect a small amount of labeled data for the new task and learn a model on it that generalizes broadly.

21.2 Meta-learning with supervised learning

In *meta-training*, each task has its own small training set (for example five images of five classes) and test set; in *meta-testing*, a new task with new classes is learned from its small training set and evaluated on its test set. Supervised learning learns $f(x) \to y$, from an input (an image) to an output (a label). Supervised meta-learning learns

$$f(\mathcal{D}^{\text{tr}}, x) \to y,$$

which also reads in a training set. Many options can read in a training set; an RNN can work (Figure 41): it reads the (few-shot) training set $(x_1, y_1), (x_2, y_2), (x_3, y_3)$ and then the test input x_{test}, and outputs the test label y_{test}.

What is being "learned"? "Generic" learning is

$$\theta^\star = \arg\min_\theta \mathcal{L}(\theta, \mathcal{D}^{\text{tr}}) = f_{\text{learn}}(\mathcal{D}^{\text{tr}}),$$

and "generic" meta-learning, over tasks $i = 1, \ldots, n$ with training and test sets $\mathcal{D}_i^{\text{tr}}$ and $\mathcal{D}_i^{\text{ts}}$, is

$$\theta^\star = \arg\min_\theta \sum_{i=1}^{n} \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{ts}}), \qquad \text{where } \phi_i = f_\theta(\mathcal{D}_i^{\text{tr}}).$$

For the RNN, $\phi_i = [h_i, \theta_p]$: the RNN hidden state h_i after reading $\mathcal{D}_i^{\text{tr}}$ together with the meta-learned

weights θ_p of the prediction head, which outputs $p_{\phi_i}(y \mid x)$.

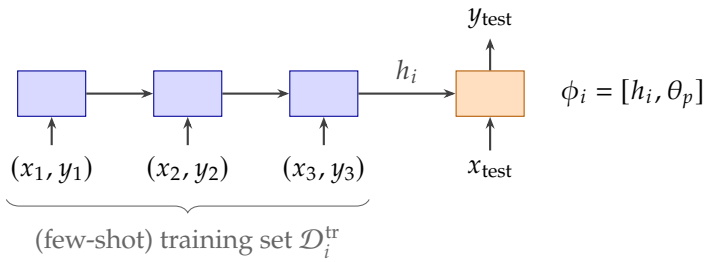


Figure 41: Black-box meta-learning: an RNN reads the training set and then predicts the label of a test input.

There are three families of meta-learning methods:

- *black-box* meta-learning: some kind of network that can read in an entire (few-shot) training set (Santoro et al., memory-augmented neural networks, 2016; Mishra et al., a simple neural attentive meta-learner, 2018);
- *non-parametric* meta-learning (Vinyals et al., matching networks; Snell et al., prototypical networks);
- *gradient-based* meta-learning (Finn et al., model-agnostic meta-learning).

21.3 Non-parametric meta-learning

The basic idea is to embed every training image with features ϕ and label a test image like its nearest neighbor under these features. Why does the nearest neighbor have the right class? Because we *meta-train* the features so that this produces the right answer. With a soft nearest neighbor,

$$p_{\text{nearest}}(x_k^{\text{tr}} \mid x_j^{\text{ts}}) \propto \exp\big(\phi(x_k^{\text{tr}})^T \phi(x_j^{\text{ts}})\big), \qquad p_\theta(y_j^{\text{ts}} \mid x_j^{\text{ts}}, \mathcal{D}^{\text{tr}}) = \sum_{k:\, y_k^{\text{tr}} = y_j^{\text{ts}}} p_{\text{nearest}}(x_k^{\text{tr}} \mid x_j^{\text{ts}}),$$

summing over all training points that have this label. The learned (soft) nearest-neighbor classifier is trained by

$$\theta^\star = \arg\min_\theta \sum_{i=1}^n \mathcal{L}\big(f_\theta(\mathcal{D}_i^{\text{tr}}), \mathcal{D}_i^{\text{ts}}\big) = -\sum_{i=1}^n \sum_{j=1}^m \log p_\theta(y_j^{\text{ts}} \mid x_j^{\text{ts}}, \mathcal{D}_i^{\text{tr}}).$$

Matching networks (Vinyals et al., 2016) use different networks to embed x^{tr} and x^{ts}, both conditioned on the entire set $\mathcal{D}_i^{\text{tr}}$:

$$p_{\text{nearest}}(x_k^{\text{tr}} \mid x_j^{\text{ts}}) \propto \exp\big(g(x_k^{\text{tr}}, \mathcal{D}_i^{\text{tr}})^T f(x_j^{\text{ts}}, \mathcal{D}_i^{\text{tr}})\big),$$

where g is a bidirectional LSTM embedding over the training set and f an attentional LSTM embedding that attends over the $g(x_k^{\text{tr}}, \mathcal{D}_i^{\text{tr}})$.

Prototypical networks (Snell et al., 2017) make two simple changes. First, instead of a soft nearest neighbor, construct a prototype for each class and classify by the nearest prototype:

$$p_\theta(y \mid x_j^{\text{ts}}, \mathcal{D}_i^{\text{tr}}) \propto \exp\left(c_y^T f(x_j^{\text{ts}})\right), \qquad c_y = \frac{1}{N_y} \sum_{k:\, y_k^{\text{tr}} = y} g(x_k^{\text{tr}}).$$

Second, get rid of all the complex machinery (the bidirectional and attentional LSTM embeddings).

Back to representations: pretraining on ImageNet gives features that make new tasks, such as detection, faster to learn. Is pretraining a type of meta-learning? Better features mean faster learning of new tasks.

21.4 Model-agnostic meta-learning

Meta-learning can be cast as an optimization problem, $\theta^\star = \arg\min_\theta \sum_{i=1}^n \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{ts}})$ with $\phi_i = f_\theta(\mathcal{D}_i^{\text{tr}})$. What if $f_\theta(\mathcal{D}_i^{\text{tr}})$ is just a *fine-tuning* algorithm,

$$f_\theta(\mathcal{D}_i^{\text{tr}}) = \theta - \alpha \nabla_\theta \mathcal{L}(\theta, \mathcal{D}_i^{\text{tr}})$$

(or a few gradient steps in general)? This can be trained like any other neural network, by implementing gradient descent as a computation graph and backpropagating *through* gradient descent. Instead of ordinary training, $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta, \mathcal{D}^{\text{tr}})$, MODEL-AGNOSTIC META-LEARNING (MAML; Finn, Abbeel, and Levine) updates

$$\theta \leftarrow \theta - \beta \sum_i \nabla_\theta \mathcal{L}\left(\theta - \alpha \nabla_\theta \mathcal{L}(\theta, \mathcal{D}_i^{\text{tr}}), \mathcal{D}_i^{\text{ts}}\right).$$

It finds parameters θ from which one adaptation step on any task's gradient $\nabla \mathcal{L}_i$ reaches close to that task's optimum $\theta_i^\star$ (Figure 42).

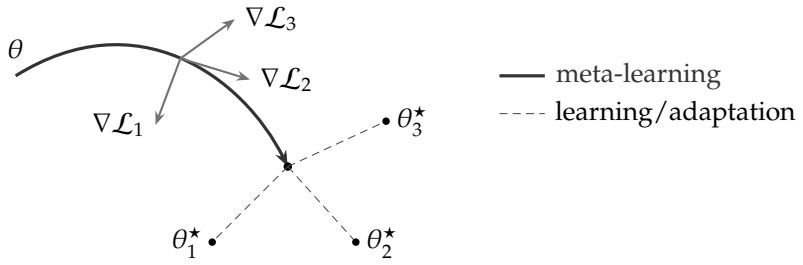


Figure 42: MAML moves θ (solid) to a point from which a gradient step adapts quickly to each task (dashed). Schematic.

What did we just do? Supervised learning is $f(x) \to y$, supervised meta-learning is $f(\mathcal{D}^{\text{tr}}, x) \to y$, and model-agnostic meta-learning is $f_{\text{MAML}}(\mathcal{D}^{\text{tr}}, x) \to y$ with

$$f_{\text{MAML}}(\mathcal{D}^{\text{tr}}, x) = f_{\theta'}(x), \qquad \theta' = \theta - \alpha \sum_{(x,y)\in\mathcal{D}^{\text{tr}}} \nabla_\theta \mathcal{L}(f_\theta(x), y).$$

It is just another computation graph, which can be implemented with any autodiff package (for example TensorFlow), but with a favorable inductive bias.

Why does it work? Black-box meta-learning implements the "learned learning algorithm" inside the network. Does it converge? Kind of. What does it converge to? Who knows. What to do if it is not good enough? Nothing. MAML runs gradient descent: it converges (it is gradient descent), to a local optimum (it is gradient descent), and if the result is not good enough we can keep taking gradient steps. Did we lose anything? No: UNIVERSALITY (Finn and Levine, *Meta-Learning and Universality*) says that both black-box meta-learning and MAML can learn any "algorithm," more precisely represent any function $f(\mathcal{D}_{\text{train}}, x)$.

Black-box	Non-parametric	Gradient-based
+ conceptually very simple; benefits from advances in sequence models (such as transformers)	+ can work very well, combining some inductive bias with easy end-to-end optimization	+ easy to apply to any architecture or loss function (including RL and regression); good generalization to out-of-domain tasks
− minimal inductive bias (everything has to be meta-learned); hard to scale to "medium" shot (long "sequences")	− restricted to classification, hard to extend to regression or reinforcement learning; somewhat specialized architectures	− meta-training optimization is harder and requires more tuning; requires second derivatives

21.5 Meta-reinforcement learning

The same pattern applies to reinforcement learning:

$$\text{reinforcement learning:} \quad \theta^\star = \arg\max_\theta E_{\pi_\theta(\tau)}[R(\tau)] = f_{\text{RL}}(\mathcal{M}), \qquad \mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, r\},$$

$$\text{meta-reinforcement learning:} \quad \theta^\star = \arg\max_\theta \sum_{i=1}^{n} E_{\pi_{\phi_i}(\tau)}[R(\tau)], \qquad \phi_i = f_\theta(\mathcal{M}_i),$$

where $\mathcal{M}$ is an MDP and $\mathcal{M}_i$ the MDP for task i. The assumption is that the meta-training MDPs $\{\mathcal{M}_1, \ldots, \mathcal{M}_n\}$ are drawn as $\mathcal{M}_i \sim p(\mathcal{M})$; at meta-test time, sample $\mathcal{M}_{\text{test}} \sim p(\mathcal{M})$ and get $\phi_i = f_\theta(\mathcal{M}_{\text{test}})$. For example, the tasks might be household chores in different homes, or running at different target speeds (0.5, 0.7, −0.2, and at test time −0.7 m/s).

The main question is how to implement $f_\theta(\mathcal{M}_i)$. What should it do?

1. improve the policy with experience from $\mathcal{M}_i$, $\{(s_1, a_1, s_2, r_1), \ldots, (s_T, a_T, s_{T+1}, r_T)\}$;
2. (new in RL) choose how to interact, that is, choose a_t: meta-RL must also *choose how to explore*.

As before, $\phi_i = [h_i, \theta_\pi]$: an RNN reads the transitions (s_t, a_t, s_{t+1}, r_t), and a policy head $\pi_{\phi_i}(a \mid s)$ uses its hidden state. So we just train an RNN policy? Yes. Crucially, the RNN hidden state is *not* reset between episodes.

Why do recurrent policies learn to explore? Consider a maze where a mouse must find cheese. In the first episode it tries a few directions and gets reward +0; once it finds the cheese (+1), later episodes, sharing the same hidden state, go straight there. Optimizing the total reward over the entire *meta-episode* (several episodes in the same MDP),

$$\theta^\star = \arg\max_\theta E_{\pi_\theta}\Big[\sum_{t=0}^{T} r(s_t, a_t)\Big],$$

with an RNN policy automatically learns to explore. Recurrent meta-RL policies have learned memory-based control (Heess et al., 2015), navigation in 3D mazes (Wang et al., *Learning to Reinforcement Learn*, 2016), and fast exploration strategies (Duan et al., RL^2, 2016). Architectures for meta-RL include a standard RNN (LSTM) whose state persists across the episodes of a trial (RL^2), attention with temporal convolution (Mishra et al.), and a parallel, permutation-invariant context encoder that infers a latent task variable $q_\phi(z \mid c)$ from transitions (s, a, s', r) (Rakelly et al.).

MAML for RL. With policy-gradient objectives J_i, MAML updates

$$\theta \leftarrow \theta + \beta \sum_i \nabla_\theta J_i\big[\theta + \alpha \nabla_\theta J_i(\theta)\big]$$

instead of $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$. For a simulated quadruped, after MAML training the policy is poised between tasks, and a single gradient step with a forward-running reward makes it run forward, while a single step with a backward-running reward makes it run backward.

Source. Sergey Levine, *CS W182/282A: Designing, Visualizing and Understanding Deep Neural Networks*, Lecture 21 (Spring 2021).